

# Java OOP — Interview Master Notes

**Target:** Beginner → Strong Intermediate → Interview Expert

**Language:** Java (8+, with modern features noted)

**Purpose:** Complete preparation for Java OOP interviews at product-based companies

---

## How to Use These Notes

### Recommended Study Order

1. **Unit 1** — Foundations first. You cannot understand polymorphism without understanding objects, references, constructors, and `static`.
2. **Unit 2** — Encapsulation, Inheritance, Polymorphism. The heart of OOP interviews.
3. **Unit 3** — Abstraction, Interfaces, Advanced OOP. Where interviewers test depth.
4. **Unit 4** — Advanced Object Behavior. Where most candidates fail.
5. **Unit 5** — Design, SOLID, Patterns. Where senior-level questions live.

### How to Practice Code

- **Type every example** yourself. Do not just read.
- **Predict the output** before running. Write your prediction on paper.
- **If your prediction is wrong**, stop and understand why. This is where learning happens.
- **Modify examples** — change access modifiers, add `static`, remove `@Override`, use `null`. See what breaks.

### How to Approach Output Questions

1. Identify the **reference type** and **object type**.
2. Determine what is resolved at **compile time** (fields, static methods, overloaded methods).
3. Determine what is resolved at **runtime** (overridden instance methods).
4. Trace the **constructor chain** (parent first, then child).

5. Watch for **initialization order** (static blocks → instance blocks → constructors).

## What Must Be Memorized

- Access modifier rules (private, default, protected, public)
- Overriding rules (access, return type, exceptions)
- Overload resolution order (widening → boxing → varargs)
- `equals()`/`hashCode()` contract
- Initialization order in inheritance

## What Must Be Understood

- Why Java is pass-by-value (not pass-by-reference)
- Why fields are not polymorphic
- Why `static` methods are hidden, not overridden
- How dynamic method dispatch works
- Why composition is preferred over inheritance

## What Should Be Practiced Repeatedly

- Output-based questions (minimum 100)
- Constructor chaining problems
- Casting problems (upcasting, downcasting, `ClassCastException`)
- `equals()`/`hashCode()` implementation
- Design problems using SOLID

---

## Java OOP Interview Roadmap





---

# UNIT 1 — Java OOP Foundations + Object & Class Internals

---

## 1.1 OOP Fundamentals

### What is OOP?

Object-Oriented Programming is a programming paradigm that organizes software around **objects** — entities that bundle **data** (state) and **behavior** (methods) together.

### Why OOP?

- **Modularity** — Each object is a self-contained unit. You can develop, test, and debug objects independently.
- **Reusability** — Through inheritance and composition, you reuse existing code without rewriting.
- **Maintainability** — Changes to one object don't ripple through the entire codebase.
- **Modeling real-world** — Objects map naturally to real-world entities (Student, Account, Order).
- **Scalability** — Large systems can be built by composing smaller objects.

### Procedural vs OOP

| Aspect      | Procedural                       | OOP                         |
|-------------|----------------------------------|-----------------------------|
| Focus       | Functions/procedures             | Objects                     |
| Data        | Shared, passed between functions | Encapsulated inside objects |
| Reusability | Copy-paste or function libraries | Inheritance, composition    |

|          |                       |                                  |
|----------|-----------------------|----------------------------------|
| Security | Data exposed globally | Data hidden via access modifiers |
| Example  | C                     | Java, C++, Python                |

## The Four Pillars of OOP

1. **Encapsulation** — Wrapping data and methods together, hiding internal state
2. **Inheritance** — Acquiring properties and behavior from a parent class
3. **Polymorphism** — Same method name, different behavior depending on the object
4. **Abstraction** — Hiding implementation complexity, exposing only what's necessary

## Class

A **class** is a blueprint/template. It defines what data an object will hold and what behavior it will have. A class does NOT occupy heap memory by itself — objects do.

```
class Student {  
    String name;    // what data  
    int age;  
  
    void study() { // what behavior  
        System.out.println(name + " is studying");  
    }  
}
```

## Object

An **object** is a specific instance created from a class. Each object has its own copy of instance variables.

```
Student s1 = new Student(); // object 1  
Student s2 = new Student(); // object 2 – separate memory  
s1.name = "Alice";  
s2.name = "Bob";
```

```
// s1 and s2 have DIFFERENT name values
```

## State, Behavior, Identity

- **State** — The values of an object's fields at any point in time (name = "Alice", age = 20)
- **Behavior** — The methods an object can perform (study(), getName())
- **Identity** — A unique internal identifier for each object (its memory address, conceptually). Even two objects with identical state are different objects.

```
Student s1 = new Student();  
Student s2 = new Student();  
s1.name = "Alice";  
s2.name = "Alice";  
// s1 and s2 have same STATE but different IDENTITY  
System.out.println(s1 == s2); // false – different objects
```

## Java and OOP

- Java is an **object-oriented** language, but NOT purely object-oriented because it has **primitive types** (int, char, boolean, etc.) that are not objects.
- Everything in Java that is not a primitive is an **object** (including arrays, strings).
- Every class implicitly extends `java.lang.Object`.

## Primitive vs Reference Types

| Aspect   | Primitive                  | Reference                                 |
|----------|----------------------------|-------------------------------------------|
| Examples | int, char, boolean, double | String, Student, int[]                    |
| Stored   | Directly holds the value   | Holds a reference (address) to the object |

|                  |                                        |                                    |
|------------------|----------------------------------------|------------------------------------|
| Default          | 0, false, '\u0000'                     | null                               |
| Memory           | Stack (local) or part of object (heap) | Reference on stack, object on heap |
| == compares      | Values                                 | References (addresses)             |
| Passed to method | Copy of value                          | Copy of reference                  |

```
int a = 10;           // primitive – holds value 10 directly
Student s = new Student(); // reference – s holds address of Student object
```

## 1.2 Class Anatomy

A Java class can contain the following components:

```
package com.example;           // 1. Package declaration

import java.util.List;         // 2. Import statements

public class Employee {        // 3. Class declaration

    // 4. Static variables (class-level, shared)
    static int employeeCount = 0;

    // 5. Instance variables (object-level, separate per object)
    private String name;
    private int age;

    // 6. Static block (runs once when class is loaded)
    static {
        System.out.println("Class loaded");
    }
}
```

```

// 7. Instance initializer block (runs every time an object is created)
{
    System.out.println("Instance block");
}

// 8. Constructor (initializes objects)
public Employee(String name, int age) {
    this.name = name;
    this.age = age;
    employeeCount++;
}

// 9. Instance method
public void work() {
    System.out.println(name + " is working");
}

// 10. Static method
public static int getCount() {
    return employeeCount;
}

// 11. Inner class (nested non-static)
class Department {
    String deptName;
}

// 12. Static nested class
static class IDCard {
    int cardNumber;
}
}

```

## Instance Variables vs Static Variables



```

class Counter {
    int instanceCount = 0;    // each object gets its own copy
    static int staticCount = 0; // shared across ALL objects

    Counter() {
        instanceCount++;
        staticCount++;
    }
}

Counter c1 = new Counter();
Counter c2 = new Counter();
Counter c3 = new Counter();

System.out.println(c1.instanceCount); // 1 – c1's own copy
System.out.println(c2.instanceCount); // 1 – c2's own copy
System.out.println(Counter.staticCount); // 3 – shared

```

## Instance Methods vs Static Methods

```

class MathUtil {
    int base = 10; // instance variable

    // Instance method – can access instance AND static members
    void showBase() {
        System.out.println(base); // OK
        System.out.println(getPI()); // OK – instance can call static
    }

    // Static method – can ONLY access static members directly
    static double getPI() {
        // System.out.println(base); // COMPILE ERROR – cannot access i
        return 3.14159;
    }
}

```

**Key Rule:** Static methods cannot access instance members directly because static methods belong to the class, not to any object. There is no `this` in a static context.

---

## 1.3 Object Creation

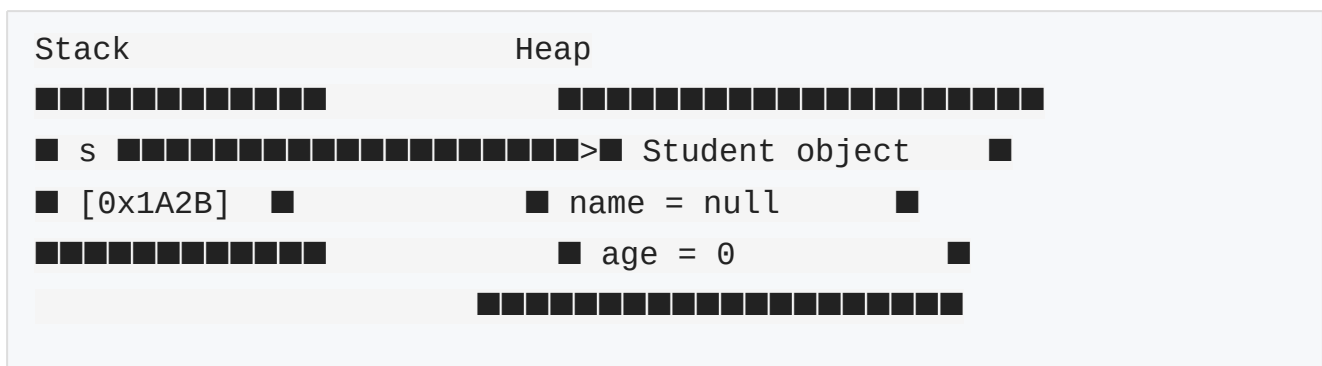
### The `new` Keyword

```
Student s = new Student();
```

This single line does multiple things:

1. **Student s** — Declares a reference variable `s` of type `Student` (on the stack)
2. **new Student()** — Creates a new `Student` object on the **heap** and calls the constructor
3. **=** — Assigns the memory address of the new object to reference `s`

### Conceptual Memory Model



**Note:** This is a conceptual model. The actual JVM memory layout is more complex (Eden space, Survivor spaces, etc.). For interview purposes, understand that references are on the stack and objects are on the heap.

### Reference Variable vs Object

- The **reference variable** (`s`) is NOT the object. It is a variable that holds the address of the object.
- The **object** is the actual data on the heap.
- You can have a reference with no object: `Student s = null;`

- You can have an object with no reference (eligible for garbage collection).

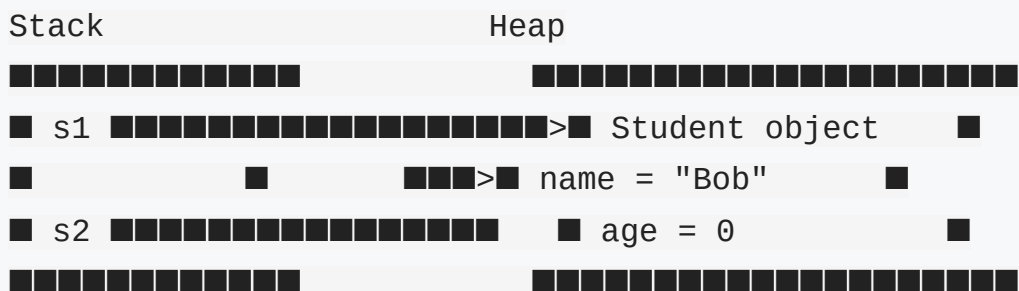
**null**

```
Student s = null; // reference exists, but points to nothing
// s.name;        // NullPointerException at RUNTIME (not compile time)
```

`null` means "this reference does not point to any object." Any attempt to use a `null` reference to access a member will throw `NullPointerException`.

## Multiple References, Same Object (Aliasing)

```
Student s1 = new Student();  
s1.name = "Alice";  
  
Student s2 = s1; // s2 now points to the SAME object as s1  
  
s2.name = "Bob";  
  
System.out.println(s1.name); // "Bob" – because s1 and s2 point to the SAME object
```



**Interview Trap:** "If I change `s2.name`, does `s1.name` also change?"

**Answer:** Yes, because both references point to the same object. There is only ONE object in memory.

## Reassigning References

```
Student s1 = new Student();
s1.name = "Alice";

Student s2 = new Student();
s2.name = "Bob";

s1 = s2; // s1 now points to the SAME object as s2
        // The original "Alice" object has NO references → eligible for GC

System.out.println(s1.name); // "Bob"
```

---

## 1.4 Constructors

### What is a Constructor?

A constructor is a special block of code that initializes a newly created object. It has: - The **same name** as the class - **No return type** (not even `void`) - Is called automatically when `new` is used

### Default Constructor

If you write **no constructor at all**, Java provides a **default constructor** automatically. It: - Takes no arguments - Has an empty body (effectively) - Calls `super ( )` implicitly

```
class Dog {
    String name;
    // Java provides: Dog() { super(); } automatically
}

Dog d = new Dog(); // uses default constructor
// d.name is null (default for String)
```

**Critical:** The moment you write ANY constructor, Java stops providing the default constructor.

```

class Dog {
    String name;
    Dog(String name) {        // parameterized constructor
        this.name = name;
    }
}

// Dog d = new Dog();        // COMPILE ERROR – no no-arg constructor exists
Dog d = new Dog("Rex");      // OK

```

## No-Argument Constructor

```

class Dog {
    String name;

    Dog() {                    // explicit no-arg constructor
        this.name = "Unknown";
    }

    Dog(String name) {
        this.name = name;
    }
}

Dog d1 = new Dog();           // OK – uses no-arg constructor
Dog d2 = new Dog("Rex");      // OK – uses parameterized constructor

```

## Constructor Overloading

Multiple constructors in the same class with different parameter lists.

```

class Student {
    String name;
    int age;
    String college;
}

```

```

Student() {
    this("Unknown", 0, "N/A"); // calls 3-param constructor
}

Student(String name) {
    this(name, 0, "N/A");
}

Student(String name, int age) {
    this(name, age, "N/A");
}

Student(String name, int age, String college) {
    this.name = name;
    this.age = age;
    this.college = college;
}
}

```

## Constructor Chaining with `this()`

- `this()` calls another constructor **in the same class**
- Must be the **first statement** in the constructor
- Cannot have both `this()` and `super()` in the same constructor
- Cannot create circular chaining (compile error)

```

class Box {
    int length, width, height;

    Box() {
        this(1, 1, 1); // chains to 3-param constructor
        // this(1);    // COMPILE ERROR if both this() calls existed
    }

    Box(int side) {

```

```

        this(side, side, side);
    }

    Box(int l, int w, int h) {
        this.length = l;
        this.width = w;
        this.height = h;
    }
}

```

## Constructor Chaining with `super()`

- `super()` calls the parent class constructor
- Must be the **first statement**
- If you don't write `this()` or `super()`, Java inserts `super()` automatically
- If the parent has no no-arg constructor, you **MUST** call `super(args)` explicitly

```

class Animal {
    String name;
    Animal(String name) {
        this.name = name;
        System.out.println("Animal constructor: " + name);
    }
}

class Dog extends Animal {
    String breed;

    Dog(String name, String breed) {
        super(name); // MUST explicitly call super(name) – parent has no no-arg constructor
        this.breed = breed;
        System.out.println("Dog constructor: " + breed);
    }
}

```

```
Dog d = new Dog("Rex", "Labrador");  
// Output:  
// Animal constructor: Rex  
// Dog constructor: Labrador
```

## Constructor Inheritance — Constructors Are NOT Inherited

```
class Parent {  
    Parent(int x) {  
        System.out.println("Parent: " + x);  
    }  
}  
  
class Child extends Parent {  
    Child() {  
        super(0); // Must explicitly call parent constructor  
    }  
    // Child does NOT inherit Parent(int x)  
}  
  
// new Child(5); // COMPILE ERROR – Child has no constructor that takes i
```

## Constructor Overriding — Constructors CANNOT Be Overridden

Overriding requires the same method name in parent and child. Since constructors have the class name, and parent/child have different class names, constructors cannot be overridden. This is a common interview question.

## Private Constructor

A constructor can be `private`. This is used in: - **Singleton pattern** — prevent external instantiation - **Utility classes** — classes with only static methods (like `Math`) - **Factory methods** — control object creation

```
class Singleton {  
    private static Singleton instance;
```



```

    private Singleton() {} // cannot be called from outside

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

// new Singleton(); // COMPILE ERROR – constructor is private
Singleton s = Singleton.getInstance(); // OK

```

## Constructor Execution Order (Tricky!)

```

class Parent {
    int x = initX(); // instance initializer expression

    static { System.out.println("1. Parent static block"); }
    { System.out.println("3. Parent instance block"); }

    Parent() {
        System.out.println("4. Parent constructor");
    }

    int initX() {
        System.out.println("2. Parent field init");
        return 10;
    }
}

class Child extends Parent {
    int y = initY();

    static { System.out.println(" 1b. Child static block"); }
    { System.out.println(" 5. Child instance block"); }
}

```

```
    Child() {  
        System.out.println(" 7. Child constructor");  
    }  
  
    int initY() {  
        System.out.println(" 6. Child field init");  
        return 20;  
    }  
}  
  
new Child();
```

### Output:

1. Parent static block
  - 1b. Child static block
2. Parent field init
3. Parent instance block
4. Parent constructor
6. Child field init
5. Child instance block
7. Child constructor

**Note:** Instance field initializers and instance blocks are woven into the constructor by the compiler, executing in textual order BEFORE the constructor body but AFTER `super()`.

---

## 1.5 The `this` Keyword

`this` is a reference to the **current object** — the object on which the method or constructor was invoked.

### `this.field` — Disambiguating Fields

```
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name; // this.name = instance variable, name = parameter  
    }  
}
```

Without `this`, the parameter `name` shadows the field `name`, and the field would not be set.

### **`this.method()` — Calling Another Method**

```
class Calculator {  
    void add(int a, int b) {  
        System.out.println(a + b);  
    }  
  
    void addAndPrint(int a, int b) {  
        this.add(a, b); // same as just add(a, b)  
    }  
}
```

Using `this.method()` is optional for instance methods — it's implied.

### **`this()` — Constructor Chaining**

```
class Point {  
    int x, y;  
  
    Point() {  
        this(0, 0); // calls Point(int, int)  
    }  
  
    Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

```
    }  
}
```

## Passing `this` as an Argument

```
class Printer {  
    void print(Student s) {  
        System.out.println(s.name);  
    }  
}  
  
class Student {  
    String name = "Alice";  
  
    void printMe(Printer p) {  
        p.print(this); // passing the current Student object  
    }  
}
```

## Returning `this` — Fluent API / Method Chaining

```
class Builder {  
    String name;  
    int age;  
  
    Builder setName(String name) {  
        this.name = name;  
        return this; // returns the current object  
    }  
  
    Builder setAge(int age) {  
        this.age = age;  
        return this;  
    }  
}
```

```
Builder b = new Builder().setName("Alice").setAge(25); // method chaining
```

## Why `this` Cannot Be Used in Static Context

```
class Example {  
    int x = 10;  
  
    static void show() {  
        // System.out.println(this.x); // COMPILE ERROR  
        // 'this' does not exist in static context because  
        // static methods belong to the CLASS, not any object.  
        // There is no "current object" to refer to.  
    }  
}
```

---

## 1.6 `static` Keyword

### Static Variables (Class Variables)

- Shared across ALL objects of the class
- Only one copy exists, regardless of how many objects are created
- Can be accessed via class name (recommended) or object reference
- Initialized when the class is loaded

```
class Student {  
    String name; // instance variable – one per object  
    static String college = "MIT"; // static variable – one per class  
  
    Student(String name) {  
        this.name = name;  
    }  
}  
  
Student s1 = new Student("Alice");
```

```
Student s2 = new Student("Bob");  
  
System.out.println(s1.college);    // "MIT"  
System.out.println(Student.college); // "MIT" – preferred access  
Student.college = "Stanford";  
System.out.println(s2.college);    // "Stanford" – changed for ALL
```

## Static Methods

- Belong to the class, not to any object
- Cannot access instance variables or instance methods directly
- Cannot use `this` or `super`
- Can be called without creating an object

```
class MathUtil {  
    static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
int sum = MathUtil.add(3, 5); // no object needed
```

## Static Blocks

- Execute once when the class is loaded into memory
- Used for static initialization logic
- Multiple static blocks execute in order of appearance

```
class Config {  
    static String dbUrl;  
  
    static {  
        System.out.println("Loading config...");  
        dbUrl = "jdbc:mysql://localhost:3306/mydb";  
    }  
}
```

```
    }  
}  
// Just referencing Config triggers class loading and the static block
```

## Why Static Cannot Access Instance Members Directly

```
class Demo {  
    int x = 10;        // instance  
    static int y = 20; // static  
  
    void instanceMethod() {  
        System.out.println(x); // OK – instance can access instance  
        System.out.println(y); // OK – instance can access static  
    }  
  
    static void staticMethod() {  
        // System.out.println(x);        // COMPILE ERROR  
        // instanceMethod();              // COMPILE ERROR  
        System.out.println(y);            // OK – static can access static  
  
        // Workaround: create an object  
        Demo d = new Demo();  
        System.out.println(d.x);          // OK – accessing through an obj  
    }  
}
```

**Reason:** A static method can be called without any object. Since instance members belong to objects, and there may be no object (or multiple objects), the compiler cannot know which object's `x` you mean.

---

## 1.7 Initialization Order

This is one of the most asked topics in interviews.

### Single Class Initialization Order

1. Static variables and static blocks (in textual order) – when class is loaded
2. Instance variables and instance blocks (in textual order) – when object is created
3. Constructor body – after instance initialization

```
class Demo {
    static int a = initA();
    int b = initB();

    static { System.out.println("2. Static block"); }
    { System.out.println("4. Instance block"); }

    Demo() {
        System.out.println("5. Constructor");
    }

    static int initA() { System.out.println("1. Static var init"); return 1; }
    int initB() { System.out.println("3. Instance var init"); return 2; }
}

// new Demo();
```

### Output:

1. Static var init
2. Static block
3. Instance var init
4. Instance block
5. Constructor

### Inheritance Initialization Order

1. Parent static variables + static blocks (textual order)
2. Child static variables + static blocks (textual order)  
--- static init done (class loading phase) ---
3. Parent instance variables + instance blocks (textual order)



4. Parent constructor body
5. Child instance variables + instance blocks (textual order)
6. Child constructor body

### Output Question 1

```
class A {  
    static { System.out.println("A static"); }  
    { System.out.println("A instance"); }  
    A() { System.out.println("A constructor"); }  
}  
  
class B extends A {  
    static { System.out.println("B static"); }  
    { System.out.println("B instance"); }  
    B() { System.out.println("B constructor"); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        new B();  
        System.out.println("---");  
        new B();  
    }  
}
```

### Output:

```
A static  
B static  
A instance  
A constructor  
B instance  
B constructor  
---  
A instance
```

A constructor  
B instance  
B constructor

**Why?** Static blocks run only ONCE (when the class is first loaded). Instance blocks and constructors run every time an object is created.

## Output Question 2

```
class Parent {  
    int x = 10;  
  
    Parent() {  
        printX(); // What does this call?  
    }  
  
    void printX() {  
        System.out.println("Parent x = " + x);  
    }  
}  
  
class Child extends Parent {  
    int x = 20;  
  
    @Override  
    void printX() {  
        System.out.println("Child x = " + x);  
    }  
}  
  
new Child();
```

**Output:**

Child x = 0

**Why?** 1. `new Child()` → calls `Child()` → implicit `super()` → calls `Parent()` 2. Inside `Parent()`, `printX()` is called. But `printX()` is overridden in `Child`. 3. Dynamic dispatch calls `Child.printX()`. 4. But `Child`'s instance variable `x` has NOT been initialized yet (we're still in `Parent`'s constructor). So `Child.x` is still `0` (default for `int`).

**Interview Trap:** Calling overridable methods from a constructor is dangerous. The child's fields may not be initialized yet.

### Output Question 3

```
class A {  
    static int x = 10;  
    int y = 20;  
  
    static { x = 30; System.out.println("A static block: x=" + x); }  
    { y = 40; System.out.println("A instance block: y=" + y); }  
  
    A() { System.out.println("A constructor: x=" + x + " y=" + y); }  
}  
  
new A();  
new A();
```

**Output:**

```
A static block: x=30  
A instance block: y=40  
A constructor: x=30 y=40  
A instance block: y=40  
A constructor: x=30 y=40
```

### Output Question 4

```
class Base {  
    Base() { System.out.println("Base"); }  
}  
class Mid extends Base {
```

```
    Mid() { System.out.println("Mid"); }  
}  
class Top extends Mid {  
    Top() { System.out.println("Top"); }  
}  
new Top();
```

**Output:**

```
Base  
Mid  
Top
```

**Why?** Top() → implicit super() → Mid() → implicit super() → Base() → prints "Base" → returns → prints "Mid" → returns → prints "Top".

### Output Question 5

```
class A {  
    static { System.out.println("A static"); }  
    A() { System.out.println("A()"); }  
}  
  
class B extends A {  
    static { System.out.println("B static"); }  
    B() { System.out.println("B()"); }  
}  
  
class C extends B {  
    static { System.out.println("C static"); }  
    C() { System.out.println("C()"); }  
}  
  
new C();
```

**Output:**

```
A static
B static
C static
A()
B()
C()
```

---

## 1.8 Access Modifiers

### The Four Levels

| Modifier             | Same Class | Same Package | Subclass (Different Package) | Different Package (Non-subclass) |
|----------------------|------------|--------------|------------------------------|----------------------------------|
| private              | ■          | ■            | ■                            | ■                                |
| default (no keyword) | ■          | ■            | ■                            | ■                                |
| protected            | ■          | ■            | ■*                           | ■                                |
| public               | ■          | ■            | ■                            | ■                                |

\*protected in a different package subclass: accessible only through inheritance, NOT through a reference of the parent type.

### private — Most Restrictive

```
class BankAccount {
    private double balance = 1000;

    public double getBalance() {
        return balance; // accessible within same class
    }
}
```

```

    }
}

class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        // System.out.println(acc.balance); // COMPILE ERROR – private
        System.out.println(acc.getBalance()); // OK – public method
    }
}

```

## protected — Tricky Across Packages

```

// File: package1/Parent.java
package package1;
public class Parent {
    protected int x = 10;
    protected void show() {
        System.out.println("Parent show");
    }
}

// File: package2/Child.java
package package2;
import package1.Parent;

public class Child extends Parent {
    void test() {
        System.out.println(this.x); // OK – inherited
        this.show(); // OK – inherited

        Parent p = new Parent();
        // System.out.println(p.x); // COMPILE ERROR – accessing through
        // p.show(); // COMPILE ERROR – not through inher
    }
}

```

**Interview Trap:** protected members from a different package are accessible ONLY through inheritance (`this.x`), NOT through a parent reference (`p.x`).

## Default (Package-Private) — No Keyword

```
class Helper {           // default access – only visible in same package
    void assist() {}     // default access
}
```

If no access modifier is specified, access is limited to the same package.

---

## 1.9 The Object Class

Every class in Java implicitly extends `java.lang.Object`. If a class doesn't explicitly extend another class, it directly extends `Object`.

```
class Student {}
// is equivalent to:
class Student extends Object {}
```

## Key Methods of Object

### `toString()`

```
class Student {
    String name = "Alice";
}

Student s = new Student();
System.out.println(s); // calls s.toString() implicitly
// Output: Student@1b6d3586 (class name + @ + hashCode in hex)
```

Overriding `toString()`:

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return "Student{name='" + name + "', age=" + age + "}";
    }
}

System.out.println(new Student("Alice", 20));
// Output: Student{name='Alice', age=20}
```

### **equals(Object obj)**

Default implementation in `Object` uses `==` (reference comparison):

```
// Object's equals:
public boolean equals(Object obj) {
    return (this == obj);
}
```

### **hashCode()**

Returns an integer hash code. Default implementation is based on memory address (implementation-dependent).

### **getClass()**

Returns the runtime class of the object. Cannot be overridden (it's `final`).

```
Student s = new Student("Alice", 20);
System.out.println(s.getClass().getName()); // "Student"
```



## clone()

Creates a shallow copy. The class must implement `Cloneable`. Covered deeply in Unit 4.

## `finalize()` — Deprecated

Called by GC before object is collected. Deprecated since Java 9. Do not use.

## 1.10 == vs equals()

## For Primitives

```
int a = 10;
int b = 10;
System.out.println(a == b); // true - compares VALUES
```

## For References — == Compares Addresses

```
String s1 = new String("Hello");
String s2 = new String("Hello");
System.out.println(s1 == s2);          // false – different objects
System.out.println(s1.equals(s2));    // true – String overrides equals() to
```

## String Pool Behavior

```
String a = "Hello";           // goes to String pool
String b = "Hello";           // reuses same pool object
String c = new String("Hello"); // creates new object on heap

System.out.println(a == b);    // true – same pool object
System.out.println(a == c);    // false – pool vs heap
System.out.println(a.equals(c)); // true – same content
```

| String Pool (in Heap)                                                                |  | Regular Heap |
|--------------------------------------------------------------------------------------|--|--------------|
|  |  |              |



## Custom Objects Without Overriding equals()

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
System.out.println(p1 == p2);          // false – different objects
System.out.println(p1.equals(p2));     // false – Object's equals uses ==
```

## Custom Objects With Proper equals()

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;           // same reference
        if (obj == null) return false;         // null check
        if (getClass() != obj.getClass()) return false; // type check
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y;
    }
}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
System.out.println(p1.equals(p2)); // true – content comparison
```

## Output Questions

### Question 1:

```
String s1 = "Java";  
String s2 = "Java";  
String s3 = new String("Java");  
String s4 = s3.intern();  
  
System.out.println(s1 == s2);    // ?  
System.out.println(s1 == s3);    // ?  
System.out.println(s1 == s4);    // ?  
System.out.println(s1.equals(s3)); // ?
```

### Answer:

```
true    - same pool object  
false   - pool vs heap  
true    - intern() returns pool reference  
true    - same content
```

### Question 2:

```
Integer a = 127;  
Integer b = 127;  
Integer c = 128;  
Integer d = 128;  
  
System.out.println(a == b);  // ?  
System.out.println(c == d);  // ?
```

### Answer:

```
true    - Integer caches values -128 to 127  
false   - 128 is outside cache range, different objects
```

**Interview Trap:** Java caches `Integer` values from -128 to 127. Within this range, `==` returns `true` for autoboxed values. Outside this range, new objects are created.

---

## 1.11 equals() + hashCode() Contract

### The Contract

1. If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` MUST be true
2. If `a.hashCode() == b.hashCode()`, `a.equals(b)` may or may not be true (hash collision)
3. If `a.equals(b)` is false, `hashCode()` values may or may not be different

**In simple terms:** Equal objects MUST have equal hash codes. But equal hash codes do NOT guarantee equal objects.

### Why This Matters — HashMap/HashSet

```
class Student {
    String name;
    Student(String name) { this.name = name; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Student)) return false;
        return this.name.equals(((Student) obj).name);
    }
    // NO hashCode() override – BUG!
}

Set<Student> set = new HashSet<>();
set.add(new Student("Alice"));
System.out.println(set.contains(new Student("Alice"))); // likely false!
```

**Why false?** `HashSet` first checks `hashCode()` to find the bucket. Since `hashCode()` is not overridden, the two `Student("Alice")` objects have different hash codes (from `Object`), so `HashSet` looks in the wrong bucket and never calls `equals()`.

### Correct Implementation

```

class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Student other = (Student) obj;
        return age == other.age && Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }
}

Set<Student> set = new HashSet<>();
set.add(new Student("Alice", 20));
System.out.println(set.contains(new Student("Alice", 20))); // true!

```

## Common Mistake — Mutable Keys

```

Student s = new Student("Alice", 20);
Set<Student> set = new HashSet<>();
set.add(s);

s.name = "Bob"; // MUTATED the key!
System.out.println(set.contains(s)); // likely false!
System.out.println(set.contains(new Student("Bob", 20))); // likely false!

```

```
System.out.println(set.contains(new Student("Alice", 20))); // likely false
```

**Why?** The object was stored in a bucket based on `hashCode("Alice", 20)`. After mutation, `hashCode("Bob", 20)` points to a different bucket. The object is "lost" in the set.

**Rule:** Never mutate an object after using it as a key in a `HashMap` or adding it to a `HashSet`.

---

## Unit 1 — Interview Section

### 20 Interview Questions

1. What is the difference between a class and an object?
2. Can you create an object without `new`? (Yes — reflection, `clone()`, deserialization, factory methods)
3. What happens if you don't write any constructor?
4. Can a constructor be `private`? When would you use it?
5. Can a constructor call another constructor? How?
6. What is the difference between `this()` and `super()`?
7. Can you have both `this()` and `super()` in the same constructor? (No)
8. Why can't `this` be used in a static method?
9. What is the difference between static and instance variables?
10. Can a static method access instance variables? (Not directly)
11. What is the initialization order in Java?
12. What is the initialization order with inheritance?
13. Explain the access levels of `private`, `default`, `protected`, and `public`.
14. What is the `protected` access rule for different packages?
15. Why does every class in Java extend `Object`?
16. What is the difference between `==` and `equals()`?
17. What happens when you print an object directly? (`toString()` is called)

18. What is the `equals()` and `hashCode()` contract?
19. Why must `hashCode()` be overridden when `equals()` is overridden?
20. What happens if two objects have the same `hashCode()` but are not `equals()`?  
(Hash collision)

## 10 Output Questions

**Q1:**

```
class A {  
    static { System.out.print("1 "); }  
    { System.out.print("2 "); }  
    A() { System.out.print("3 "); }  
}  
new A();  
new A();
```

**Output:** 1 2 3 2 3

**Concept:** Static block runs once; instance block and constructor run per object.

**Q2:**

```
String s1 = "abc";  
String s2 = "abc";  
String s3 = new String("abc");  
System.out.println(s1 == s2);  
System.out.println(s1 == s3);  
System.out.println(s1.equals(s3));
```

**Output:** true, false, true

**Concept:** String pool reuse, heap object, `equals()` content comparison.

**Q3:**

```
class Test {  
    int x;  
    Test(int x) { this.x = x; }
```

```
Test() { this(10); System.out.println(x); }  
}  
new Test();
```

**Output:** 10

**Concept:** `this(10)` calls parameterized constructor first, sets `x` to 10.

**Q4:**

```
class A {  
    A() { this(5); System.out.print("A() "); }  
    A(int x) { System.out.print("A(int) "); }  
}  
class B extends A {  
    B() { System.out.print("B() "); }  
}  
new B();
```

**Output:** A(int) A() B()

**Concept:** B() → super() → A() → this(5) → A(int) prints → A() prints → B() prints.

**Q5:**

```
class Parent {  
    Parent() { show(); }  
    void show() { System.out.println("Parent"); }  
}  
class Child extends Parent {  
    int x = 10;  
    void show() { System.out.println("Child: " + x); }  
}  
new Child();
```

**Output:** Child: 0

**Concept:** During Parent constructor, `show()` dispatches to Child's override, but `x` is not yet initialized (still 0).

**Q6:**



```
class Test {
    static int a;
    int b;
    static { a = 5; }
    { b = a + 10; }
    Test() { System.out.println("a=" + a + " b=" + b); }
}
new Test();
```

**Output:** a=5 b=15

**Q7:**

```
Integer x = 100;
Integer y = 100;
Integer p = 200;
Integer q = 200;
System.out.println(x == y);
System.out.println(p == q);
```

**Output:** true, false

**Concept:** Integer cache range is -128 to 127.

**Q8:**

```
class A { A() { System.out.print("A "); } }
class B extends A { B() { System.out.print("B "); } }
class C extends B { C() { System.out.print("C "); } }
new C();
```

**Output:** A B C

**Q9:**

```
class Demo {
    static Demo d = new Demo();
    static { System.out.println("static block"); }
    { System.out.println("instance block"); }
```

```
Demo() { System.out.println("constructor"); }  
}  
new Demo();
```

**Output:**

```
instance block  
constructor  
static block  
instance block  
constructor
```

**Concept:** `static Demo d = new Demo();` triggers instance creation during static initialization — before the static block runs.

**Q10:**

```
class Test {  
    int x = 10;  
    Test() { x = 20; }  
    { x = 15; }  
}  
Test t = new Test();  
System.out.println(t.x);
```

**Output:** 20

**Concept:** Field init (x=10) → instance block (x=15) → constructor (x=20). Final value is 20.

## 5 Debugging Questions

**D1:**

```
class Student {  
    String name;  
    Student(String name) {  
        name = name; // BUG: assigns parameter to itself  
    }  
}
```

```
}  
Student s = new Student("Alice");  
System.out.println(s.name); // prints null
```

**Fix:** `this.name = name;`

**D2:**

```
class Parent {  
    Parent(int x) {}  
}  
class Child extends Parent {  
    // COMPILE ERROR: no default constructor in Parent  
}
```

**Fix:** Add `Child() { super(0); }` or add no-arg constructor in Parent.

**D3:**

```
class A {  
    A() { this(5); }  
    A(int x) { this(); } // COMPILE ERROR: recursive constructor invocation  
}
```

**Fix:** Remove circular `this()` call.

**D4:**

```
class Test {  
    static void show() {  
        System.out.println(this); // COMPILE ERROR: 'this' in static context  
    }  
}
```

**Fix:** Remove `this` reference or make the method non-static.

**D5:**

```
class Box {  
    final int size;  
    // COMPILER ERROR: final variable 'size' might not have been initialized  
}
```

**Fix:** Initialize `size` at declaration, in an instance block, or in ALL constructors.

## 5 "Why" Questions

**Why doesn't Java have a destructor like C++?** — Java has garbage collection. The GC automatically reclaims memory. `finalize()` existed but was unreliable and is deprecated.

**Why must `this()` or `super()` be the first statement?** — The parent object must be fully initialized before the child can execute any code. Java enforces this at compile time to prevent accessing an uninitialized parent.

**Why are constructors not inherited?** — A constructor is specific to its class. A `Dog(String name)` constructor makes sense for `Dog` but might not make sense for `Puppy` extends `Dog` which might need additional parameters.

**Why is `String` immutable?** — Security (used in class loading, network connections), thread safety (shared without synchronization), caching (String pool, hashCode caching), and performance.

**Why can a static method not be overridden?** — Overriding is runtime polymorphism based on the object type. Static methods belong to the class, not the object. They are resolved at compile time based on the reference type. They can be *hidden* but not *overridden*.

---

## UNIT 2 — Encapsulation + Inheritance + Polymorphism

---

### 2.1 Encapsulation

#### What is Encapsulation?

Encapsulation is the practice of: 1. **Bundling** data (fields) and methods that operate on that data into a single class 2. **Hiding** the internal state from the outside world using access

modifiers 3. **Providing controlled access** through public getters/setters

## Why Encapsulation?

- **Data protection** — Prevent invalid states
- **Flexibility** — Change internal implementation without affecting external code
- **Validation** — Enforce rules when setting values
- **Debugging** — Control all access points to data

## Bad Design (No Encapsulation)

```
class BankAccount {  
    double balance; // public by default in same package  
}  
  
BankAccount acc = new BankAccount();  
acc.balance = -5000; // INVALID STATE – no validation!
```

## Good Design (With Encapsulation)

```
class BankAccount {  
    private double balance;  
  
    public BankAccount(double initialBalance) {  
        if (initialBalance < 0) throw new IllegalArgumentException("Negative  
        this.balance = initialBalance;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void deposit(double amount) {  
        if (amount <= 0) throw new IllegalArgumentException("Deposit must b  
        balance += amount;  
    }  
}
```

```
    public void withdraw(double amount) {  
        if (amount <= 0) throw new IllegalArgumentException("Withdrawal must be positive");  
        if (amount > balance) throw new IllegalArgumentException("Insufficient funds");  
        balance -= amount;  
    }  
}
```

## Encapsulation and Immutability Connection

An immutable object is the ultimate form of encapsulation — the state cannot be changed at all after creation.

```
final class ImmutablePoint {  
    private final int x;  
    private final int y;  
  
    public ImmutablePoint(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int getX() { return x; }  
    public int getY() { return y; }  
    // No setters – state cannot change  
}
```

---

## 2.2 Inheritance

### What is Inheritance?

Inheritance is a mechanism where a new class (child/subclass) acquires the properties and behavior of an existing class (parent/superclass).

```
class Animal {  
    String name;
```

```

    void eat() { System.out.println(name + " eats"); }
}

class Dog extends Animal {
    void bark() { System.out.println(name + " barks"); }
}

Dog d = new Dog();
d.name = "Rex";
d.eat();    // inherited from Animal
d.bark();   // defined in Dog

```

## IS-A Relationship

Inheritance creates an IS-A relationship: Dog IS-A Animal.

```

Dog d = new Dog();
System.out.println(d instanceof Dog);    // true
System.out.println(d instanceof Animal); // true
System.out.println(d instanceof Object); // true

```

## Types of Inheritance in Java

```

Single:      A → B
Multilevel:  A → B → C
Hierarchical: A → B, A → C

NOT supported with classes:
Multiple:    A, B → C      // COMPILE ERROR
Hybrid:      combinations  // NOT supported

```

## Why Multiple Class Inheritance is NOT Supported

```

// Hypothetical – does NOT compile
class A {

```

```

    void show() { System.out.println("A"); }
}
class B {
    void show() { System.out.println("B"); }
}
class C extends A, B { // COMPILE ERROR
    // Which show() to inherit? Ambiguity!
}

```

Java avoids this ambiguity (the "Diamond Problem") by not allowing multiple class inheritance. However, Java supports multiple inheritance through **interfaces** (covered in Unit 3).

### What is Inherited?

| Member                                   | Inherited?                                    |
|------------------------------------------|-----------------------------------------------|
| public fields/methods                    | ■ Yes                                         |
| protected fields/methods                 | ■ Yes                                         |
| default (package-private) fields/methods | ■ Only in same package                        |
| private fields/methods                   | ■ No (exist in parent, but not accessible)    |
| Constructors                             | ■ Never                                       |
| Static members                           | Accessible, but belong to the declaring class |

**Important distinction:** `private` members are NOT inherited (not accessible), but they DO exist in the child object's memory (the parent's part of the object still has them).



```
class Parent {
    private int secret = 42;
    public int getSecret() { return secret; }
}

class Child extends Parent {
    void test() {
        // System.out.println(secret); // COMPILER ERROR – not inherited
        System.out.println(getSecret()); // OK – accesses parent's private
    }
}
```

---

## 2.3 super Keyword

### **super.field** — Access Parent's Field

```
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;

    void show() {
        System.out.println(x); // 20 – child's x
        System.out.println(super.x); // 10 – parent's x
    }
}
```

### **super.method()** — Call Parent's Method

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}

class Dog extends Animal {
    @Override
```

```

    void sound() {
        super.sound(); // calls Animal's sound()
        System.out.println("Bark");
    }
}

new Dog().sound();
// Output:
// Some sound
// Bark

```

## **super ( ) — Call Parent's Constructor**

```

class Person {
    String name;
    Person(String name) {
        this.name = name;
    }
}

class Student extends Person {
    int rollNo;
    Student(String name, int rollNo) {
        super(name); // calls Person(String)
        this.rollNo = rollNo;
    }
}

```

## **Automatic super ( ) Insertion**

If you don't explicitly write `this()` or `super()`, the compiler inserts `super()` (no-arg) as the first statement.

```

class A {
    A() { System.out.println("A"); }
}

```

```

class B extends A {
    B() {
        // super(); ← inserted by compiler
        System.out.println("B");
    }
}
new B(); // Output: A B

```

**Trap:** If the parent does NOT have a no-arg constructor, the implicit `super()` fails:

```

class A {
    A(int x) { } // only parameterized constructor
}
class B extends A {
    B() {
        // super(); ← compiler inserts this, but A has no no-arg constructor
        // COMPILE ERROR!
    }
}

```

**Fix:** Either add a no-arg constructor in `A`, or explicitly call `super(someValue)` in `B`.

---

## 2.4 Method Overloading

### What is Method Overloading?

Same method name in the same class (or inherited), but different parameter lists. This is **compile-time polymorphism** (also called static polymorphism).

```

class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}

```

### What Makes Overloading Valid?

Methods are overloaded if they differ in: - **Number** of parameters - **Type** of parameters - **Order** of parameter types

Methods are NOT overloaded if they differ only in: - Return type (COMPILE ERROR if same name and params but different return) - Access modifier - Exception declarations - Parameter names

```
// VALID overloading
void show(int a) {}
void show(String a) {}
void show(int a, String b) {}
void show(String a, int b) {} // different ORDER of types

// INVALID – compile error
int show(int a) {} // same params as void show(int a)
void show(int x) {} // same as show(int a) – name doesn't matter
```

## Overload Resolution Order (VERY IMPORTANT for interviews)

When Java encounters an overloaded method call, it resolves using this priority:

1. Exact match
2. Widening (implicit type promotion)
3. Autoboxing/unboxing
4. Varargs

Java will NOT combine widening AND boxing in a single step.

### Exact Match

```
void show(int x) { System.out.println("int"); }
void show(long x) { System.out.println("long"); }

show(5); // Output: int – exact match with int
```

### Widening

```
void show(long x) { System.out.println("long"); }
void show(float x) { System.out.println("float"); }

show(5); // Output: long – int widens to long (before float)
```

Widening order: byte → short → int → long → float → double

## Autoboxing

```
void show(Integer x) { System.out.println("Integer"); }

show(5); // Output: Integer – int autoboxed to Integer
```

## Varargs (Lowest Priority)

```
void show(int... x) { System.out.println("varargs"); }

show(5); // Output: varargs – only if no other match
```

## Combined Resolution Example

```
class Demo {
    void method(int x) { System.out.println("int"); }
    void method(long x) { System.out.println("long"); }
    void method(Integer x) { System.out.println("Integer"); }
    void method(int... x) { System.out.println("varargs"); }
}

new Demo().method(5);
// Output: int (exact match wins)

// If you remove method(int x):
// Output: long (widening wins over boxing)

// If you also remove method(long x):
// Output: Integer (boxing wins over varargs)
```

```
// If you also remove method(Integer x):  
// Output: varargs
```

## The `null` Ambiguity Trap

```
class Demo {  
    void show(String s) { System.out.println("String"); }  
    void show(Object o) { System.out.println("Object"); }  
}  
new Demo().show(null); // Output: String
```

**Why?** When `null` matches multiple overloads, Java picks the **most specific** type. `String` is more specific than `Object`.

```
class Demo {  
    void show(String s) { System.out.println("String"); }  
    void show(Integer i) { System.out.println("Integer"); }  
}  
new Demo().show(null); // COMPILE ERROR – ambiguous!
```

**Why error?** Neither `String` nor `Integer` is more specific than the other. They are siblings in the type hierarchy.

## Overloading with Widening + Boxing Combined

```
void show(Long x) { System.out.println("Long"); }  
  
show(5); // COMPILE ERROR!  
// Java will NOT widen int to long AND THEN box long to Long.  
// Widening + boxing is NOT allowed in a single step.
```

But boxing THEN widening IS allowed:

```
void show(Object x) { System.out.println("Object"); }

show(5); // int → Integer (boxing) → Object (widening) – OK!
// Output: Object
```

---

## 2.5 Method Overriding

### What is Method Overriding?

A subclass provides a specific implementation for a method already defined in the parent class. This is **runtime polymorphism** (dynamic polymorphism).

```
class Animal {
    void sound() {
        System.out.println("Some sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow");
    }
}

Animal a = new Dog();
a.sound(); // Output: Bark – runtime polymorphism

a = new Cat();
```

```
a.sound(); // Output: Meow
```

## Rules for Overriding

| Rule                         | Requirement                                 |
|------------------------------|---------------------------------------------|
| Method name                  | Must be exactly the same                    |
| Parameters                   | Must be exactly the same                    |
| Return type                  | Same or covariant (subtype)                 |
| Access modifier              | Same or wider (more accessible)             |
| Exceptions (checked)         | Same, narrower, or none — CANNOT be broader |
| Exceptions (unchecked)       | Any — no restriction                        |
| <code>static</code> methods  | CANNOT be overridden (they are hidden)      |
| <code>final</code> methods   | CANNOT be overridden                        |
| <code>private</code> methods | NOT visible to child — not overriding       |
| Constructors                 | CANNOT be overridden                        |

## @Override Annotation

```
class Parent {  
    void show() { System.out.println("Parent"); }  
}
```



```

class Child extends Parent {
    @Override
    void show() { System.out.println("Child"); } // compiler checks this I
}

// @Override
// void shwo() { } // COMPILE ERROR – no such method in Parent (typo c
}

```

Always use `@Override`. It catches errors at compile time.

## Access Modifier Rules

```

class Parent {
    protected void show() {}
}

class Child extends Parent {
    // @Override public void show() {} // OK – wider access
    // @Override protected void show() {} // OK – same access
    // @Override void show() {} // COMPILE ERROR – default is na
    // @Override private void show() {} // COMPILE ERROR – narrower acce
}

```

**Memory aid:** Access can be widened but never narrowed: `private` → `default` → `protected` → `public`

## Covariant Return Types

The overriding method can return a **subtype** of the original return type.

```

class Animal {
    Animal create() {
        return new Animal();
    }
}

class Dog extends Animal {

```

```

    @Override
    Dog create() {        // returns Dog instead of Animal – covariant return
        return new Dog();
    }
}

```

This works because `Dog` IS-A `Animal`, so returning a `Dog` where an `Animal` is expected is safe.

## Exception Rules During Overriding

```

class Parent {
    void method() throws IOException { }
}

class Child extends Parent {
    // @Override void method() throws IOException { }           // OK – same
    // @Override void method() throws FileNotFoundException { } // OK – nar
    // @Override void method() { }                               // OK – no exc
    // @Override void method() throws Exception { }             // COMPILE ERR
    // @Override void method() throws SQLException { }          // COMPILE ERR

    @Override
    void method() throws RuntimeException { } // OK – unchecked exceptions
}

```

**Rule:** Overriding method CANNOT throw NEW or BROADER checked exceptions.  
 Unchecked exceptions are unrestricted.

---

## 2.6 Overloading vs Overriding — Detailed Comparison

| Aspect | Overloading               | Overriding           |
|--------|---------------------------|----------------------|
| Where? | Same class (or inherited) | Parent–Child classes |

|                              |                                 |                                    |
|------------------------------|---------------------------------|------------------------------------|
| Method name                  | Same                            | Same                               |
| Parameters                   | MUST be different               | MUST be same                       |
| Return type                  | Can be anything                 | Same or covariant                  |
| Access modifier              | Can be anything                 | Same or wider                      |
| Binding                      | Compile-time (static binding)   | Runtime (dynamic binding)          |
| Which decides?               | Reference type + argument types | Object type                        |
| <code>static</code> methods  | Can be overloaded               | Cannot be overridden (hidden)      |
| <code>private</code> methods | Can be overloaded               | Cannot be overridden (not visible) |
| <code>final</code> methods   | Can be overloaded               | Cannot be overridden               |
| Polymorphism type            | Compile-time                    | Runtime                            |
| Exceptions                   | No rules                        | Checked exception restrictions     |

## Critical Distinction with Code

```
class Parent {
    void show(int x) {
        System.out.println("Parent int: " + x);
    }
    void show(String s) {
```

```

        System.out.println("Parent String: " + s);
    }
}

class Child extends Parent {
    @Override
    void show(int x) {                                // OVERRIDING
        System.out.println("Child int: " + x);
    }
    void show(double d) {                             // OVERLOADING (new parameter)
        System.out.println("Child double: " + d);
    }
}

Parent p = new Child();
p.show(5);      // Child int: 5      – overridden, runtime dispatch
p.show("Hi");   // Parent String: Hi – not overridden in Child
// p.show(3.14); // COMPILE ERROR – Parent reference, Parent has no show(double)

```

---

## 2.7 Dynamic Method Dispatch

This is the mechanism by which Java resolves overridden methods at **runtime**.

```

Animal a = new Dog();
a.sound();

```

### Step-by-step:

1. **Compile time:** Compiler checks `Animal` class. Does `Animal` have a `sound()` method? YES → compiles.
2. **Runtime:** JVM looks at the ACTUAL OBJECT type (`Dog`). Does `Dog` override `sound()`? YES → executes `Dog.sound()`.

```

class Animal {
    void sound() { System.out.println("Generic sound"); }
}

```

```

}
class Dog extends Animal {
    @Override void sound() { System.out.println("Bark"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Meow"); }
}

// Polymorphic behavior
Animal[] animals = { new Dog(), new Cat(), new Animal() };
for (Animal a : animals) {
    a.sound(); // dispatched at runtime based on actual object type
}
// Output:
// Bark
// Meow
// Generic sound

```

## The Two Key Questions

For ANY polymorphic call `ref.method()`:

1. **Compile time:** Does the **reference type** have this method? (Determines if it compiles)
2. **Runtime:** Does the **object type** override this method? (Determines what executes)

---

## 2.8 Reference Type vs Object Type

```

class Parent {
    int x = 10;
    void show() { System.out.println("Parent show"); }
    void parentOnly() { System.out.println("Only in Parent"); }
}

class Child extends Parent {
    int x = 20;
}

```

```
@Override void show() { System.out.println("Child show"); }
void childOnly() { System.out.println("Only in Child"); }
}

Parent p = new Child(); // Reference: Parent, Object: Child
```

| What                        | Resolved by              | Result                                       |
|-----------------------------|--------------------------|----------------------------------------------|
| <code>p.x</code>            | Reference type (Parent)  | <code>10</code>                              |
| <code>p.show()</code>       | Object type (Child)      | <code>"Child show"</code>                    |
| <code>p.parentOnly()</code> | Compiles (Parent has it) | <code>"Only in Parent"</code>                |
| <code>p.childOnly()</code>  | COMPILE ERROR            | Parent doesn't have <code>childOnly()</code> |

## The Golden Rule

|   |                             |                                             |   |
|---|-----------------------------|---------------------------------------------|---|
| ■ | FIELDS                      | → resolved by REFERENCE type (compile time) | ■ |
| ■ | STATIC                      | → resolved by REFERENCE type (compile time) | ■ |
| ■ | OVERRIDDEN INSTANCE METHODS | → resolved by OBJECT type                   | ■ |
| ■ |                             | (runtime – dynamic dispatch)                | ■ |
| ■ | ACCESSIBLE MEMBERS          | → determined by REFERENCE type              | ■ |

## Output Question — The Classic

```
class Parent {
    int x = 10;
    void show() { System.out.println("Parent"); }
}
```

```

class Child extends Parent {
    int x = 20;
    @Override void show() { System.out.println("Child"); }
}

Parent p = new Child();
System.out.println(p.x); // ?
p.show();                // ?

```

**Output:**

```

10
Child

```

**Why?** - `p.x` → 10 because **fields are NOT polymorphic**. Reference type is `Parent`, so `Parent.x = 10`. - `p.show()` → "Child" because **instance methods ARE polymorphic**. Object type is `Child`, so `Child.show()` runs.

**Interview Trap:** Many candidates assume `p.x` will give 20 because the object is `Child`. **WRONG**. Fields are resolved at compile time by reference type.

## 2.9 Upcasting

Upcasting is assigning a child object to a parent reference. It is **automatic** and **safe**.

```

Animal a = new Dog();    // upcasting – implicit, no cast needed
Object o = new String("hello"); // upcasting

```

After upcasting: - You can only access members declared in the **reference type** (`Parent`) - Overridden methods will execute the **child's version** at runtime - You CANNOT access child-specific methods

```

Animal a = new Dog();
a.eat();    // OK – Animal has eat()
a.sound();  // OK – Animal has sound(), but Dog's override runs
// a.fetch(); // COMPILER ERROR – Animal doesn't have fetch()

```

---

## 2.10 Downcasting

Downcasting is assigning a parent reference to a child reference. It requires an **explicit cast** and is **dangerous**.

```
Animal a = new Dog();           // upcasting
Dog d = (Dog) a;                 // downcasting – OK at runtime because a actually is a Dog
d.fetch();                       // OK – now we can access Dog-specific methods
```

### ClassCastException

```
Animal a = new Cat();
Dog d = (Dog) a;    // RUNTIME ERROR: ClassCastException
                   // a points to Cat, not Dog!
```

The compiler allows the cast (since Dog IS-A Animal makes it theoretically possible), but at runtime, the JVM checks and throws `ClassCastException`.

### Safe Downcasting with instanceof

```
Animal a = new Cat();

if (a instanceof Dog) {
    Dog d = (Dog) a; // safe – only executes if a is actually a Dog
    d.fetch();
} else {
    System.out.println("Not a Dog"); // this runs
}
```

### Java 16+ Pattern Matching for instanceof

```
Animal a = new Dog();

if (a instanceof Dog d) { // checks AND casts in one step
    d.fetch();             // d is automatically a Dog reference
}
```



```
}
```

---

## 2.11 Overriding Rules — Complete Reference

### private Methods — Not Overridden

```
class Parent {
    private void secret() { System.out.println("Parent secret"); }

    void callSecret() { secret(); } // calls Parent's secret
}

class Child extends Parent {
    // This is NOT overriding – it's a completely new method
    private void secret() { System.out.println("Child secret"); }
}

Parent p = new Child();
p.callSecret(); // Output: "Parent secret"
// callSecret() in Parent calls Parent.secret() – no dispatch
```

### static Methods — Hidden, Not Overridden

```
class Parent {
    static void greet() { System.out.println("Parent greet"); }
}

class Child extends Parent {
    static void greet() { System.out.println("Child greet"); }
    // This HIDES Parent.greet(), does NOT override it
}

Parent p = new Child();
p.greet(); // Output: "Parent greet"
Child.greet(); // Output: "Child greet"
```

**Why "Parent greet"?** Static methods are resolved at **compile time** using the **reference type**. `p` is of type `Parent`, so `Parent.greet()` is called. There is no dynamic dispatch for static methods.

### **final Methods — Cannot Be Overridden**

```
class Parent {
    final void show() { System.out.println("Cannot override me"); }
}

class Child extends Parent {
    // @Override void show() {} // COMPILE ERROR – show() is final
}
```

### **protected → public Is Allowed**

```
class Parent {
    protected void show() {}
}

class Child extends Parent {
    @Override
    public void show() {} // OK – widened access
}
```

### **Constructor — Cannot Be Overridden**

Constructors have the class name. `Parent` and `child` have different class names. Therefore, constructors cannot be overridden. This is a fundamental concept, not just a rule.

---

## **2.12 Static Method Hiding — Deep Dive**

```
class Animal {
    static void info() { System.out.println("Animal info"); }
    void sound() { System.out.println("Animal sound"); }
}
```

```

class Dog extends Animal {
    static void info() { System.out.println("Dog info"); } // HIDING
    @Override void sound() { System.out.println("Dog sound"); } // OVERRIDING
}

Animal a = new Dog();
a.info(); // "Animal info" – static: reference type decides (compile time)
a.sound(); // "Dog sound" – instance: object type decides (runtime)

Dog d = new Dog();
d.info(); // "Dog info" – reference type is Dog
d.sound(); // "Dog sound"

```

---

## 2.13 Field Hiding

Fields are NEVER polymorphic. If parent and child both declare a field with the same name, the child's field HIDES the parent's field.

```

class Parent {
    int x = 10;
    String type = "Parent";
}

class Child extends Parent {
    int x = 20;
    String type = "Child";
}

Parent p = new Child();
System.out.println(p.x); // 10 – reference type is Parent
System.out.println(p.type); // "Parent"

Child c = new Child();
System.out.println(c.x); // 20 – reference type is Child
System.out.println(c.type); // "Child"

```

```
// Accessing parent's hidden field from child
System.out.println(((Parent) c).x);    // 10
System.out.println(((Parent) c).type); // "Parent"
```

**Key Rule:** The reference type at **compile time** determines which field is accessed. Always.

---

## Unit 2 — Interview Section

### 25 Interview Questions

1. What is encapsulation? Give a real-world example.
2. What is inheritance? What keyword is used?
3. Why is multiple class inheritance not supported in Java?
4. What is the difference between IS-A and HAS-A?
5. Are private members inherited?
6. What does `super` do?
7. What happens if the parent has no no-arg constructor and the child doesn't call `super(args)`?
8. What is method overloading?
9. What is method overriding?
10. What is the difference between overloading and overriding?
11. What is compile-time polymorphism?
12. What is runtime polymorphism?
13. Explain dynamic method dispatch.
14. What determines which overloaded method is called?
15. What determines which overridden method is called?
16. Can we override a static method? (No — it's hidden)
17. Can we override a private method? (No — not visible)
18. Can we override a final method? (No)

19. What is upcasting? Is it safe?
20. What is downcasting? What can go wrong?
21. What is `ClassCastException`? When does it occur?
22. What is covariant return type?
23. Can overriding method throw broader checked exceptions? (No)
24. What is field hiding? Are fields polymorphic?
25. What is the overload resolution order in Java?

## 20 Output Questions

**Q1:**

```
class A { void show() { System.out.println("A"); } }
class B extends A { void show() { System.out.println("B"); } }
class C extends B { void show() { System.out.println("C"); } }
A obj = new C();
obj.show();
```

**Output:** C — Dynamic dispatch goes to the actual object type (C).

**Q2:**

```
class A {
    int x = 10;
    void show() { System.out.println(x); }
}
class B extends A {
    int x = 20;
}
A obj = new B();
System.out.println(obj.x);
obj.show();
```

**Output:** 10, 10

`obj.x` → reference type A → 10. `obj.show()` → not overridden in B → A's `show()` → prints A's x (10).

**Q3:**

```
class Parent {  
    void show(int x) { System.out.println("Parent int"); }  
}  
class Child extends Parent {  
    void show(String x) { System.out.println("Child String"); }  
}  
Parent p = new Child();  
p.show(5);  
// p.show("hi"); // would this compile?
```

**Output:** Parent int

p.show("hi") would NOT compile — Parent has no show(String). Child's show(String) is an overload, not override.

**Q4:**

```
class A {  
    static void show() { System.out.println("A"); }  
}  
class B extends A {  
    static void show() { System.out.println("B"); }  
}  
A obj = new B();  
obj.show();
```

**Output:** A — Static methods use reference type. obj is type A.

**Q5:**

```
class A {  
    void show() { System.out.println("A"); }  
}  
class B extends A {  
    void show() { System.out.println("B"); }  
}  
class C extends B {
```

```

    void show() { System.out.println("C"); }
}
B obj = new C();
obj.show();

```

**Output:** C — Object type is C, C overrides show().

**Q6:**

```

class Parent {
    int x = 5;
    int getX() { return x; }
}
class Child extends Parent {
    int x = 10;
    int getX() { return x; }
}
Parent p = new Child();
System.out.println(p.x);
System.out.println(p.getX());

```

**Output:** 5, 10

p.x → reference type Parent → 5. p.getX() → dynamic dispatch → Child's getX() → Child's x → 10.

**Q7:**

```

class A { void m(int x) { System.out.println("A-int"); } }
class B extends A {
    void m(int x) { System.out.println("B-int"); }
    void m(long x) { System.out.println("B-long"); }
}
A a = new B();
a.m(5);

```

**Output:** B-int

Compile time: A has m(int) → compiles. Runtime: B overrides m(int) → B-int. Note: a.m(5L) would NOT compile because A has no m(long).

**Q8:**

```
class Animal {  
    void eat() { System.out.println("Animal eats"); }  
}  
class Dog extends Animal {  
    void eat() { System.out.println("Dog eats"); }  
    void bark() { System.out.println("Dog barks"); }  
}  
Animal a = new Dog();  
a.eat();  
// a.bark(); // ?
```

**Output:** Dog eats

`a.bark()` would NOT compile — Animal reference does not know about `bark()`.

**Q9:**

```
class A {  
    A() { print(); }  
    void print() { System.out.println("A"); }  
}  
class B extends A {  
    int x = 5;  
    void print() { System.out.println("B: " + x); }  
}  
new B();
```

**Output:** B: 0

A's constructor calls `print()`, which dispatches to B's `print()`. But B's `x` is not initialized yet (still 0).

**Q10:**

```
class A {  
    String name = "A";  
    String getName() { return name; }  
}
```



```
class B extends A {  
    String name = "B";  
    String getName() { return name; }  
}  
A obj = new B();  
System.out.println(obj.name);  
System.out.println(obj.getName());
```

**Output:** A, B

Field: reference type (A). Method: object type (B).

**Q11:**

```
void show(Object o) { System.out.println("Object"); }  
void show(String s) { System.out.println("String"); }  
  
show(null);
```

**Output:** String — Most specific type is chosen.

**Q12:**

```
class P {  
    protected void show() { System.out.println("P"); }  
}  
class C extends P {  
    public void show() { System.out.println("C"); } // wider access – OK  
}  
P p = new C();  
p.show();
```

**Output:** C

**Q13:**

```
class A {  
    void show(long x) { System.out.println("long"); }  
}
```

```

class B extends A {
    void show(int x) { System.out.println("int"); }
}
A a = new B();
a.show(5);

```

**Output:** long

Compile time: A has `show(long)`. int 5 widens to long. B's `show(int)` is an overload not visible through A reference.

**Q14:**

```

class A {
    static int x = 10;
    int y = 20;
}
class B extends A {
    static int x = 30;
    int y = 40;
}
A a = new B();
System.out.println(a.x + " " + a.y);

```

**Output:** 10 20 — Both static and instance fields use reference type.

**Q15:**

```

Animal a = new Animal();
Dog d = (Dog) a; // what happens?

```

**Answer:** Compiles (compiler allows it), but throws `ClassCastException` at runtime. The actual object is `Animal`, not `Dog`.

**Q16:**

```

class A {
    void show() throws Exception { System.out.println("A"); }
}

```

```
class B extends A {  
    void show() { System.out.println("B"); } // valid?  
}  
A a = new B();  
a.show();
```

**Output: B**

Valid — overriding method can throw NO exception (narrower than parent's Exception).

**Q17:**

```
class A {  
    private void show() { System.out.println("A"); }  
    void call() { show(); }  
}  
class B extends A {  
    void show() { System.out.println("B"); }  
}  
A a = new B();  
a.call();
```

**Output: A**

private methods are not overridden. call() in A invokes A's show().

**Q18:**

```
class A {  
    int x = 10;  
}  
class B extends A {  
    int x = 20;  
    void print() {  
        System.out.println(x);  
        System.out.println(super.x);  
        System.out.println(((A)this).x);  
    }  
}
```

```
new B().print();
```

**Output:** 20, 10, 10

**Q19:**

```
class Base {  
    void show(int x) { System.out.println("Base int"); }  
}  
class Der extends Base {  
    void show(Integer x) { System.out.println("Der Integer"); }  
}  
Base b = new Der();  
b.show(5);
```

**Output:** Base int

show(Integer) in Der is an **overload**, not an override. Base reference → only show(int) visible → exact match.

**Q20:**

```
class A {  
    final void show() { System.out.println("A"); }  
}  
class B extends A {  
    // void show() {} // COMPILE ERROR – final method  
}  
A a = new B();  
a.show();
```

**Output:** A — final methods cannot be overridden.

## 10 Tricky Inheritance Questions

**T1:** Can an interface extend a class? → No. Interfaces extend interfaces, classes extend classes.

**T2:** If a child class does not override a parent method, which version runs? → The parent's version (inherited).

**T3:** Can you call a parent constructor from a child's instance method? → No. `super()` can only be called from a constructor, and must be the first statement.

**T4:** What happens if both parent and child have a static method with the same signature and you call it on a child reference? → Child's static method runs (method hiding based on reference type, and reference is Child).

**T5:** Is `super.super.method()` valid? → No. Java does not allow skipping levels. You can only access the immediate parent with `super`.

**T6:** Can a child's overriding method be `synchronized` even if the parent's is not? → Yes. `synchronized` is not part of the method signature for overriding purposes.

**T7:** Can you override `equals()` with a different parameter type? → That would be overloading, not overriding. `equals(Object)` must take `Object` to override.

**T8:** If Parent has `void m(int x)` and Child has `int m(int x)`, is this overriding? → No. COMPILE ERROR. Same name and parameters but different return type (int vs void). Not covariant.

**T9:** Can an abstract method be `static`? → No. abstract methods need to be overridden. `static` methods can't be overridden.

**T10:** Can you narrow the return type when overriding? → Only for reference types (covariant). You cannot change `int` to `short` (primitives don't have inheritance).

## 5 Casting Problems

**C1:**

```
Object o = "Hello";
String s = (String) o;      // OK – o actually is a String
Integer i = (Integer) o;    // ClassCastException at runtime
```

**C2:**

```
Object o = new Integer(5);
Number n = (Number) o;      // OK – Integer IS-A Number
Double d = (Double) o;      // ClassCastException – Integer is not a Double
```

**C3:**

```
class A {}
class B extends A {}
class C extends A {}

A a = new B();
C c = (C) a; // ClassCastException – actual object is B, not C
```

**C4:**

```
List<String> list = new ArrayList<>();
Object o = list;
// ArrayList<String> al = o; // COMPILER ERROR – needs cast
ArrayList<String> al = (ArrayList<String>) o; // OK at runtime
```

**C5:**

```
int[] arr = {1, 2, 3};
Object o = arr; // OK – arrays are objects
int[] arr2 = (int[]) o; // OK
// String[] sa = (String[]) o; // ClassCastException
```

## 5 Compile-Time vs Runtime Questions

**CR1:** `Parent p = new Child(); p.childOnlyMethod();` → **Compile-time error**.  
Reference type (`Parent`) doesn't have `childOnlyMethod()`.

**CR2:** `Animal a = new Dog(); a.sound();` where `Dog` overrides `sound()`. →  
**Compiles** (`Animal` has `sound()`). At **runtime**, `Dog.sound()` executes.

**CR3:** `Animal a = new Dog(); Dog d = (Dog) a;` → **Compiles** and succeeds at  
runtime (`a` is actually `Dog`).

**CR4:** `Animal a = new Cat(); Dog d = (Dog) a;` → **Compiles** but throws  
**ClassCastException** at runtime.

**CR5:** `Parent p = new Child(); System.out.println(p.x);` where both have  
`int x`. → **Compiles** and prints `Parent's x`. Fields resolved at **compile time**.

---

# UNIT 3 — Abstraction + Interfaces + Advanced OOP

---

## 3.1 Abstraction

### What is Abstraction?

Abstraction means **hiding the implementation complexity** and exposing only the essential features to the user.

- You know **what** an object does, but not **how** it does it.
- Example: You drive a car using the steering wheel and pedals. You don't need to know how the engine works internally.

### Why Abstraction Exists

- **Reduces complexity** — Users interact with a simple interface
- **Loose coupling** — Implementation can change without affecting users
- **Security** — Internal details are hidden
- **Code organization** — Forces a clean separation between "what" and "how"

### Java Mechanisms for Abstraction

1. **Abstract classes** — Partial abstraction (can have both abstract and concrete methods)
2. **Interfaces** — Full abstraction (traditionally all abstract; Java 8+ allows defaults)

---

## 3.2 Abstract Classes

### What is an Abstract Class?

An abstract class is a class that: - Is declared with the `abstract` keyword - **Cannot be instantiated** directly - Can contain abstract methods (no body) and concrete methods (with body) - Is meant to be extended by subclasses

```

abstract class Shape {
    String color;

    Shape(String color) {
        this.color = color;
    }

    // Abstract method – no body, MUST be overridden by concrete subclasses
    abstract double area();

    // Concrete method – has a body, inherited as-is
    void displayColor() {
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}

// Shape s = new Shape("Red"); // COMPILE ERROR – cannot instantiate abstract
Shape s = new Circle("Red", 5);
System.out.println(s.area()); // 78.53...
s.displayColor(); // Color: Red

```

## What Can an Abstract Class Contain?



| Member                     | Allowed?                                              |
|----------------------------|-------------------------------------------------------|
| Abstract methods           | ■ Yes                                                 |
| Concrete methods           | ■ Yes                                                 |
| Constructors               | ■ Yes (called via <code>super()</code> by subclasses) |
| Instance variables         | ■ Yes                                                 |
| Static variables           | ■ Yes                                                 |
| Static methods             | ■ Yes                                                 |
| Final methods              | ■ Yes (cannot be overridden by subclasses)            |
| Private methods            | ■ Yes (Java 9+)                                       |
| <code>main()</code> method | ■ Yes                                                 |

## Illegal Combinations

```
// abstract + final – COMPILE ERROR
// abstract final class X {}
// Reason: abstract must be extended, final prevents extension

// abstract + private – COMPILE ERROR
// abstract class X { abstract private void m(); }
// Reason: abstract must be overridden, private prevents access

// abstract + static – COMPILE ERROR
// abstract class X { abstract static void m(); }
```

```
// Reason: abstract must be overridden, static methods can't be overridden
```

## Abstract Class with No Abstract Methods

This is legal. It simply prevents direct instantiation.

```
abstract class Base {  
    void show() { System.out.println("Base"); }  
    // No abstract methods – still valid  
}  
  
// new Base(); // COMPILER ERROR – still can't instantiate  
class Child extends Base {}  
new Child();    // OK
```

## If a Subclass Doesn't Override All Abstract Methods

The subclass must also be declared `abstract`.

```
abstract class A {  
    abstract void m1();  
    abstract void m2();  
}  
  
abstract class B extends A {  
    void m1() { System.out.println("m1 implemented"); }  
    // m2() not implemented → B must be abstract  
}  
  
class C extends B {  
    void m2() { System.out.println("m2 implemented"); }  
    // Now all abstract methods are implemented → C is concrete  
}
```

---

## 3.3 Interfaces

## What is an Interface?

An interface is a contract that defines **what** a class must do, but not **how**.

```
interface Flyable {  
    void fly(); // implicitly public and abstract  
}  
  
class Bird implements Flyable {  
    @Override  
    public void fly() {  
        System.out.println("Bird flies with wings");  
    }  
}  
  
class Airplane implements Flyable {  
    @Override  
    public void fly() {  
        System.out.println("Airplane flies with engines");  
    }  
}  
  
Flyable f = new Bird(); // interface reference  
f.fly(); // "Bird flies with wings"
```

## Key Properties

- All methods are implicitly `public abstract` (before Java 8)
- All variables are implicitly `public static final`
- Cannot have constructors
- Cannot be instantiated
- A class can implement multiple interfaces
- An interface can extend multiple interfaces

---

## 3.4 Interface Evolution (Java 8, 9+)

## Java 8 — Default Methods

**Why?** Before Java 8, adding a new method to an interface broke ALL existing implementations. Default methods allow adding methods with a body.

```
interface Printable {
    void print();

    default void printInColor() {
        System.out.println("Printing in color...");
    }
}

class Document implements Printable {
    @Override
    public void print() {
        System.out.println("Document printed");
    }
    // printInColor() is inherited with its default body
}

new Document().printInColor(); // "Printing in color..."
```

## Java 8 — Static Methods

```
interface MathOps {
    static int add(int a, int b) {
        return a + b;
    }
}

int result = MathOps.add(3, 5); // called via interface name
// NOT inherited by implementing classes
```

**Important:** Interface static methods are NOT inherited by implementing classes. You must call them on the interface directly.

## Java 9 — Private Interface Methods

**Why?** To share code between default methods without exposing it.

```
interface Logger {  
    default void logInfo(String msg) {  
        log("INFO", msg);  
    }  
  
    default void logError(String msg) {  
        log("ERROR", msg);  
    }  
  
    private void log(String level, String msg) {  
        System.out.println "[" + level + "] " + msg);  
    }  
}
```

---

## 3.5 Interface Variables

All fields in an interface are automatically `public static final`.

```
interface Constants {  
    int MAX = 100;           // implicitly public static final  
    String NAME = "Java";    // implicitly public static final  
}  
  
// Constants.MAX = 200;      // COMPILE ERROR – final  
System.out.println(Constants.MAX); // 100
```

**Why?** Interfaces define contracts. Variables in interfaces are constants shared by all implementations. They cannot vary per instance (no instance state in interfaces).

---

## 3.6 Multiple Inheritance Through Interfaces

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {
    @Override
    public void fly() { System.out.println("Duck flies"); }

    @Override
    public void swim() { System.out.println("Duck swims"); }
}

Flyable f = new Duck();
f.fly();

Swimmable s = new Duck();
s.swim();
```

## Why Classes Can't Extend Multiple Classes But Can Implement Multiple Interfaces

- **Classes have state** (fields, constructors). Multiple inheritance of state creates ambiguity about which field/constructor to use.
- **Interfaces (traditional) have no state**. They only define method signatures, so there's no ambiguity about which data to inherit.
- Java 8 default methods reintroduce some ambiguity, which is resolved by explicit rules.

---

## 3.7 Default Method Conflict (Diamond Problem in Interfaces)

```
interface A {
    default void show() { System.out.println("A"); }
```

```

}

interface B {
    default void show() { System.out.println("B"); }
}

class C implements A, B {
    // COMPILER ERROR if show() is not overridden!
    // Must resolve the conflict explicitly:

    @Override
    public void show() {
        A.super.show(); // explicitly choose A's version
        // OR
        // B.super.show();
        // OR provide entirely new implementation
    }
}

```

## Resolution Rules

1. **Class wins over interface.** If a class inherits a method from both a superclass and an interface, the class method wins.
2. **Subtype wins over supertype.** If two interfaces have a subtype relationship, the more specific one wins.
3. **If still ambiguous,** the implementing class **MUST** override the method.

```

interface A {
    default void show() { System.out.println("A"); }
}

class B {
    public void show() { System.out.println("B"); }
}

class C extends B implements A {

```

```
// No need to override – B (class) wins over A (interface)
}  
  
new C().show(); // Output: "B" – class always wins
```

### 3.8 Abstract Class vs Interface — Deep Comparison

| Feature              | Abstract Class              | Interface                               |
|----------------------|-----------------------------|-----------------------------------------|
| Keyword              | <code>abstract class</code> | <code>interface</code>                  |
| Instantiation        | Cannot                      | Cannot                                  |
| Constructors         | ■ Yes                       | ■ No                                    |
| Instance fields      | ■ Yes                       | ■ Only <code>public static final</code> |
| Abstract methods     | ■ Yes                       | ■ Yes (implicitly)                      |
| Concrete methods     | ■ Yes                       | ■ Default methods (Java 8+)             |
| Static methods       | ■ Yes (inherited)           | ■ Yes (NOT inherited)                   |
| Private methods      | ■ Yes                       | ■ Yes (Java 9+)                         |
| Final methods        | ■ Yes                       | ■ No                                    |
| Multiple inheritance | ■ Single class only         | ■ Multiple interfaces                   |



|                  |     |                                                      |
|------------------|-----|------------------------------------------------------|
| Access modifiers | Any | public (methods),<br>public static final<br>(fields) |
|------------------|-----|------------------------------------------------------|

## When to Use Each

**Use abstract class when:** - You want to share **state** (fields) among related classes - You need **constructors** - You want to provide a **common base implementation** that subclasses refine - The classes have a strong IS-A relationship - Example: `abstract class Vehicle` with fields `speed`, `fuel`

**Use interface when:** - You want to define a **capability/behavior** that unrelated classes can implement - You need **multiple inheritance** - You want to define a **contract** without dictating any implementation - Example: `interface Serializable`, `interface Comparable`

**Interview answer:** "Use abstract classes for IS-A relationships with shared state. Use interfaces for CAN-DO relationships (capabilities) that cut across the class hierarchy."

---

## 3.9 Composition vs Inheritance

### Inheritance (IS-A)

```
class Engine {
    void start() { System.out.println("Engine starts"); }
}

class Car extends Engine { // CAR IS-A ENGINE? Semantically wrong!
    void drive() { start(); }
}
```

### Composition (HAS-A) — Preferred

```
class Engine {
    void start() { System.out.println("Engine starts"); }
}
```

```

class Car {
    private Engine engine; // CAR HAS-A ENGINE – correct!

    Car(Engine engine) {
        this.engine = engine;
    }

    void drive() {
        engine.start();
        System.out.println("Car is driving");
    }
}

```

## Why Composition is Preferred

| Aspect        | Inheritance                               | Composition                       |
|---------------|-------------------------------------------|-----------------------------------|
| Coupling      | Tight (child depends on parent internals) | Loose (depends only on interface) |
| Flexibility   | Fixed at compile time                     | Can change at runtime             |
| Reuse         | Only in subclass hierarchy                | Any class can compose             |
| Encapsulation | Breaks (child sees protected members)     | Preserves (only public API used)  |
| Testing       | Harder (need parent)                      | Easier (can mock components)      |

**Rule of thumb:** "Favor composition over inheritance" (from Effective Java by Joshua Bloch).

---

### 3.10 Association

Association is a relationship between two independent classes. Neither owns the other.

```
class Teacher {
    String name;
    Teacher(String name) { this.name = name; }
}

class Student {
    String name;
    Teacher teacher; // Student is associated with a Teacher

    Student(String name, Teacher teacher) {
        this.name = name;
        this.teacher = teacher;
    }
}
// Both can exist independently
```

### 3.11 Aggregation (Weak HAS-A)

Aggregation is a special form of association where one class contains a reference to another, but both can exist independently. The contained object can outlive the container.

```
class Department {
    String name;
    List<Employee> employees; // Department HAS employees

    Department(String name, List<Employee> employees) {
        this.name = name;
        this.employees = employees;
    }
}
// If Department is destroyed, Employees still exist
```

Department  Employee (hollow diamond = aggregation)

### 3.12 Composition (Strong HAS-A)

Composition is a stronger form where the contained object cannot exist without the container. If the container is destroyed, the contained objects are also destroyed.

```
class Room {
    String name;
    Room(String name) { this.name = name; }
}

class House {
    private List<Room> rooms; // House HAS Rooms

    House() {
        // Rooms are created INSIDE House – they don't exist independently
        rooms = new ArrayList<>();
        rooms.add(new Room("Bedroom"));
        rooms.add(new Room("Kitchen"));
    }
}

// If House is destroyed, Rooms are also destroyed
```

House   Room (filled diamond = composition)

---

### 3.13 final Keyword

#### final Variable

```
final int MAX = 100;
// MAX = 200; // COMPILE ERROR – cannot reassign
```

#### final Reference vs Immutable Object

```
final Student s = new Student("Alice");
s.name = "Bob";           // OK – you can change the object's state
// s = new Student("Eve"); // COMPILER ERROR – cannot reassign the reference
```

**Critical distinction:** - `final` reference → You cannot point `s` to a **different object** - `final` reference does NOT make the object **immutable** — you can still modify its fields - To make an object truly immutable, you need: `final` class, `private final` fields, no setters, defensive copying

## `final` Method

```
class Parent {
    final void show() { System.out.println("Cannot override"); }
}

class Child extends Parent {
    // void show() {} // COMPILER ERROR – final method cannot be overridden
}
```

## `final` Class

```
final class Utility {
    // Cannot be extended
}

// class SubUtility extends Utility {} // COMPILER ERROR
```

Examples: `String`, `Integer`, `Math` are all `final` classes.

## `final` Parameter

```
void process(final int x) {
    // x = 10; // COMPILER ERROR – cannot reassign final parameter
    System.out.println(x);
}
```

---

## 3.14 Nested Classes

### Static Nested Class

- Declared with `static` inside another class
- Can access outer class's static members only
- Does NOT need an outer class instance

```
class Outer {
    static int x = 10;
    int y = 20;

    static class Inner {
        void show() {
            System.out.println(x);    // OK – static member
            // System.out.println(y); // COMPILER ERROR – instance member
        }
    }
}

Outer.Inner inner = new Outer.Inner(); // no outer instance needed
inner.show();
```

### Inner Class (Non-static Nested)

- Declared without `static`
- Can access ALL outer class members (including private)
- Needs an outer class instance

```
class Outer {
    private int x = 10;

    class Inner {
        void show() {
            System.out.println(x);    // OK – can access private members of outer class
        }
    }
}
```

```

    }
}

Outer outer = new Outer();
Outer.Inner inner = outer.new Inner(); // needs outer instance
inner.show(); // 10

```

## Local Class

Defined inside a method. Can access local variables only if they are effectively final.

```

class Outer {
    void doSomething() {
        final int x = 10; // must be effectively final

        class Local {
            void show() { System.out.println(x); }
        }

        new Local().show();
    }
}

```

## Anonymous Class

A class without a name, declared and instantiated in one expression.

## 3.15 Anonymous Classes

```

interface Greeting {
    void greet();
}

// Anonymous class implementing Greeting
Greeting g = new Greeting() {

```

```
    @Override
    public void greet() {
        System.out.println("Hello!");
    }
};

g.greet(); // "Hello!"
```

## Practical Example — Event Handling Style

```
abstract class Task {
    abstract void execute();
}

// Create and use without a named subclass
Task t = new Task() {
    @Override
    void execute() {
        System.out.println("Task executed");
    }
};

t.execute();
```

## Anonymous Class vs Lambda (Java 8+)

For functional interfaces (single abstract method), lambdas are preferred:

```
// Anonymous class
Runnable r1 = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};
```



```
// Lambda – shorter and cleaner
Runnable r2 = () -> System.out.println("Running");
```

Anonymous classes are still needed when: - The interface has multiple methods - You need to extend a class (not just implement an interface) - You need `this` to refer to the anonymous class itself

---

## Unit 3 — Interview Section

### 25 Interview Questions

1. What is abstraction? How is it different from encapsulation?
2. What is an abstract class? Can it have a constructor?
3. Can an abstract class have no abstract methods?
4. Can you instantiate an abstract class? (No, but you can use it as a reference type)
5. What is an interface? What is its purpose?
6. What are default methods in interfaces? Why were they introduced?
7. What are static methods in interfaces? Are they inherited?
8. What are private methods in interfaces (Java 9+)?
9. Why are interface variables `public static final`?
10. Can a class implement multiple interfaces?
11. Can an interface extend multiple interfaces?
12. What is the diamond problem? How does Java handle it with interfaces?
13. How are default method conflicts resolved?
14. When should you use an abstract class vs an interface?
15. What is composition? How is it different from inheritance?
16. Why is composition preferred over inheritance?
17. What is association, aggregation, and composition? How do they differ?
18. What does `final` mean for variables, methods, and classes?
19. Does `final` make an object immutable? (No — only the reference is final)

20. What is a static nested class? How is it different from an inner class?
21. What is an anonymous class? When would you use one?
22. Can an abstract method be `static`? (No)
23. Can an abstract method be `final`? (No)
24. Can an abstract class be `final`? (No)
25. Can an interface extend a class? (No)

## 15 Output Questions

**Q1:**

```
interface A {  
    default void show() { System.out.println("A"); }  
}  
interface B {  
    default void show() { System.out.println("B"); }  
}  
class C implements A, B {  
    public void show() { A.super.show(); }  
}  
new C().show();
```

**Output:** A — Explicitly delegates to A's default method.

**Q2:**

```
abstract class Base {  
    Base() { System.out.println("Base"); }  
}  
class Child extends Base {  
    Child() { System.out.println("Child"); }  
}  
new Child();
```

**Output:** Base, Child — Abstract class constructors are called via `super()`.

**Q3:**

```

interface I {
    int X = 10;
}
class C implements I {
    void show() {
        // X = 20;  // COMPILER ERROR — X is final
        System.out.println(X);
    }
}
new C().show();

```

**Output:** 10

**Q4:**

```

interface A {
    default void show() { System.out.println("A"); }
}
class B {
    public void show() { System.out.println("B"); }
}
class C extends B implements A {}
new C().show();

```

**Output:** B — Class method wins over interface default method.

**Q5:**

```

abstract class Shape {
    abstract double area();
    void describe() { System.out.println("Area = " + area()); }
}
class Circle extends Shape {
    double area() { return 3.14 * 5 * 5; }
}
Shape s = new Circle();
s.describe();

```

**Output:** Area = 78.5 — area() dispatches to Circle's implementation.

**Q6:**

```
final int x;  
x = 10;  
// x = 20; // COMPILE ERROR  
System.out.println(x);
```

**Output:** 10 — Blank final can be assigned once.

**Q7:**

```
final Student s = new Student("Alice");  
s.name = "Bob";  
System.out.println(s.name);
```

**Output:** Bob — final prevents reference reassignment, not object mutation.

**Q8:**

```
interface Drawable {  
    void draw();  
}  
Drawable d = new Drawable() {  
    public void draw() { System.out.println("Drawing"); }  
};  
d.draw();  
System.out.println(d.getClass().getName());
```

**Output:** Drawing, then something like Main\$1 — Anonymous class has a compiler-generated name.

**Q9:**

```
interface A { default void m() { System.out.println("A"); } }  
interface B extends A { default void m() { System.out.println("B"); } }  
class C implements A, B { }
```

```
new C().m();
```

**Output:** B — B is more specific than A (subtype wins).

**Q10:**

```
interface I {  
    static void show() { System.out.println("Interface"); }  
}  
class C implements I { }  
// C.show();      // COMPILE ERROR – static interface methods not inherited  
I.show();
```

**Output:** Interface

**Q11:**

```
abstract class A {  
    A() { show(); }  
    abstract void show();  
}  
class B extends A {  
    int x = 10;  
    void show() { System.out.println("x = " + x); }  
}  
new B();
```

**Output:** x = 0 — Same trap as before: field not yet initialized during parent constructor.

**Q12:**

```
class Outer {  
    private int x = 10;  
    class Inner {  
        void show() { System.out.println(x); }  
    }  
}  
new Outer().new Inner().show();
```

**Output:** 10 — Inner class can access outer's private members.

**Q13:**

```
interface A { void m(); }
interface B { void m(); }
class C implements A, B {
    public void m() { System.out.println("C"); }
}
A a = new C();
B b = new C();
a.m();
b.m();
```

**Output:** C, C — Same method signature, one implementation satisfies both interfaces.

**Q14:**

```
class Outer {
    static int x = 10;
    static class Nested {
        void show() { System.out.println(x); }
    }
}
new Outer.Nested().show();
```

**Output:** 10 — Static nested class can access outer's static members.

**Q15:**

```
final class A {}
// class B extends A {} // COMPILE ERROR
System.out.println("A is final");
```

**Output:** A is final

## 10 Interface Questions

1. Can an interface have instance variables? → No, all are `public static final`.

2. Can you create an object of an interface? → No, but you can create anonymous implementations.
3. Can an interface have a constructor? → No.
4. Can an interface extend a class? → No, interfaces extend interfaces.
5. Can a class extend an interface? → No, classes implement interfaces.
6. Can an interface extend multiple interfaces? → Yes.
7. Can default methods be `final`? → No (they must be overridable).
8. Can static interface methods be overridden? → No (they're not inherited).
9. Are interface methods `public` by default? → Yes.
10. Can an interface have a `main()` method? → Yes (static method).

## 10 Abstract Class/Interface Trap Questions

1. **Trap:** "Abstract classes can't have constructors." → **Wrong.** They can. Called via `super()`.
2. **Trap:** "You always need abstract methods in an abstract class." → **Wrong.** An abstract class with zero abstract methods is legal.
3. **Trap:** "Interfaces can't have method bodies." → **Wrong since Java 8.** Default and static methods have bodies.
4. **Trap:** "You can create an instance of an interface with `new Interface()`." → **Wrong.** What you're creating is an anonymous class implementing the interface.
5. **Trap:** "All interface methods must be overridden." → **Wrong.** Default methods are optional to override.
6. **Trap:** "`final` makes the object immutable." → **Wrong.** Only the reference is final.
7. **Trap:** "Static nested classes need an outer class instance." → **Wrong.** Only inner (non-static) classes do.
8. **Trap:** "Interface static methods are inherited." → **Wrong.** They must be called on the interface name.
9. **Trap:** "An abstract class with all abstract methods is the same as an interface." → **Wrong.** An abstract class can have constructors, instance fields, and supports single inheritance only.

10. **Trap:** "Composition means the same thing as aggregation." → **Wrong.** Composition implies ownership and lifecycle dependency; aggregation does not.

---

## UNIT 4 — Advanced Object Behavior + Java-Specific Traps

---

### 4.1 Immutable Classes

#### What is an Immutable Class?

An immutable class is a class whose objects **cannot be modified after creation**. Once constructed, the state never changes.

#### Why Immutability?

- **Thread-safe** — No synchronization needed
- **Cacheable** — Safe to cache and reuse
- **Predictable** — No unexpected state changes
- **Good hash keys** — `hashCode()` never changes

#### Rules to Create an Immutable Class

1. Declare the class as `final` (prevent subclassing)
2. Make all fields `private final`
3. Don't provide setters
4. Initialize all fields via constructor
5. If fields include mutable objects, perform **defensive copying**

#### Correct Immutable Class

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
```



```
final class Student {
    private final String name;
    private final int age;
    private final List<String> courses;

    public Student(String name, int age, List<String> courses) {
        this.name = name;
        this.age = age;
        // Defensive copy – don't store the original mutable list
        this.courses = new ArrayList<>(courses);
    }

    public String getName() { return name; }
    public int getAge() { return age; }

    public List<String> getCourses() {
        // Return unmodifiable copy – prevent external modification
        return Collections.unmodifiableList(courses);
    }
}
```

### Common Mistake — No Defensive Copy

```
// BAD – mutable field exposed
final class Broken {
    private final List<String> items;

    Broken(List<String> items) {
        this.items = items; // stores the SAME reference!
    }

    List<String> getItems() { return items; }
}

List<String> list = new ArrayList<>();
list.add("A");
```

```
Broken b = new Broken(list);  
list.add("B"); // modifies the "immutable" object!  
System.out.println(b.getItems()); // [A, B] – broken immutability!
```

---

## 4.2 String Immutability

### String is Immutable

Once a `String` object is created, its content cannot be changed. Any operation that appears to modify a string actually creates a **new** `String` object.

```
String s = "Hello";  
s.concat(" World"); // creates a new String, but result is NOT assigned  
System.out.println(s); // "Hello" – original unchanged  
  
s = s.concat(" World"); // now s points to the NEW string  
System.out.println(s); // "Hello World"
```

### String Pool

Java maintains a **String pool** (in the heap, within the Metaspace area) to save memory by reusing string literals.

```
String a = "Java"; // creates in pool (or reuses existing)  
String b = "Java"; // reuses the SAME pool object  
String c = new String("Java"); // creates NEW object on heap (outside pool)  
  
System.out.println(a == b); // true – same pool reference  
System.out.println(a == c); // false – pool vs heap  
System.out.println(a.equals(c)); // true – same content  
  
String d = c.intern(); // returns pool reference for "Java"  
System.out.println(a == d); // true
```

### Why is String Immutable?

1. **Security** — Strings are used in class loading, network connections, file paths. If mutable, an attacker could change a file path after security checks.
  2. **String Pool** — Sharing is safe only because strings can't change. If `a` and `b` share a pool string, mutating through `a` would corrupt `b`.
  3. **Hashing** — `hashCode()` can be cached (computed once, reused). `HashMap` keys work reliably.
  4. **Thread safety** — Immutable objects are inherently thread-safe.
- 

## 4.3 Pass-by-Value (VERY IMPORTANT)

### The Rule

Java is **ALWAYS pass-by-value**. There is no pass-by-reference in Java. Period.

- For **primitives**: A copy of the **VALUE** is passed.
- For **objects**: A copy of the **REFERENCE** (address) is passed. NOT the object itself.

### Primitive Pass-by-Value

```
void change(int x) {  
    x = 100; // modifies the LOCAL copy  
}  
  
int a = 10;  
change(a);  
System.out.println(a); // 10 – unchanged
```

### Object Pass-by-Value (Tricky!)

```
void modify(Student s) {  
    s.name = "Bob"; // modifies the OBJECT that s points to  
}  
  
Student s1 = new Student("Alice");  
modify(s1);
```

```
System.out.println(s1.name); // "Bob" – changed!
```

### Wait, if Java is pass-by-value, why did it change?

Because the VALUE that was passed is the REFERENCE (address). Both `s1` and the method parameter `s` point to the SAME object. So modifying the object through either reference affects the same object.

### The Key Distinction — Reassigning vs Mutating

```
void reassign(Student s) {  
    s = new Student("Charlie"); // reassigns the LOCAL copy of the reference  
    // This does NOT affect the original reference  
}  
  
Student s1 = new Student("Alice");  
reassign(s1);  
System.out.println(s1.name); // "Alice" – unchanged!
```

Before `reassign()`:

```
s1 ■■■■■■> [Student: "Alice"]  
s  ■■■■■■> [Student: "Alice"] (same object)
```

After `s = new Student("Charlie")`:

```
s1 ■■■■■■> [Student: "Alice"] (s1 unchanged)  
s  ■■■■■■> [Student: "Charlie"] (local copy reassigned)
```

### Complete Confusion-Clearing Example

```
void process(int[] arr, int x) {  
    arr[0] = 999; // MUTATES the array object – visible to caller  
    x = 999;      // modifies local copy of primitive – NOT visible  
    arr = new int[]{1, 2, 3}; // reassigns local reference – NOT visible  
}  
  
int[] myArr = {10, 20, 30};
```

```
int myX = 10;
process(myArr, myX);

System.out.println(myX);      // 10 – primitive unchanged
System.out.println(myArr[0]); // 999 – mutation visible
System.out.println(myArr.length); // 3 – reassignment NOT visible
```

## Swap Test — The Classic

```
void swap(Student a, Student b) {
    Student temp = a;
    a = b;
    b = temp;
    // This only swaps the LOCAL copies of the references!
}

Student s1 = new Student("Alice");
Student s2 = new Student("Bob");
swap(s1, s2);
System.out.println(s1.name); // "Alice" – NOT swapped
System.out.println(s2.name); // "Bob" – NOT swapped
```

**Interview Rule:** "You can modify the object a reference points to, but you cannot make the caller's reference point to a different object."

---

## 4.4 Shallow vs Deep Copy

### Shallow Copy

Copies the object but NOT the objects it references. The copy shares references to nested objects.

```
class Address {
    String city;
    Address(String city) { this.city = city; }
```

```

}

class Person {
    String name;
    Address address;

    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }

    // Shallow copy
    Person shallowCopy() {
        return new Person(this.name, this.address); // same Address reference
    }
}

Address addr = new Address("NYC");
Person p1 = new Person("Alice", addr);
Person p2 = p1.shallowCopy();

p2.address.city = "LA";
System.out.println(p1.address.city); // "LA" – p1 is affected!

```

## Deep Copy

Copies the object AND all nested objects recursively.

```

class Person {
    String name;
    Address address;

    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }
}

```

```
// Deep copy
Person deepCopy() {
    return new Person(this.name, new Address(this.address.city));
}

Person p1 = new Person("Alice", new Address("NYC"));
Person p2 = p1.deepCopy();

p2.address.city = "LA";
System.out.println(p1.address.city); // "NYC" – p1 is NOT affected
```

---

## 4.5 clone()

### How clone() Works

1. The class must implement Cloneable (marker interface)
2. Override clone() from Object
3. Call super.clone() which performs a **shallow copy**

```
class Student implements Cloneable {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    protected Student clone() throws CloneNotSupportedException {
        return (Student) super.clone();
    }
}
```

```
Student s1 = new Student("Alice", 20);
Student s2 = s1.clone();
s2.name = "Bob";
System.out.println(s1.name); // "Alice" – separate objects
```

## Limitations of `clone()`

- `super.clone()` performs **shallow copy** — nested mutable objects are shared
- `CloneNotSupportedException` if `Cloneable` is not implemented
- No constructor is called — object is created by copying bits
- Considered broken by many experts (Joshua Bloch: "clone is deeply broken")
- **Recommended alternatives:** Copy constructors or factory methods

```
// Copy constructor – preferred alternative
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Copy constructor
    Student(Student other) {
        this.name = other.name;
        this.age = other.age;
    }
}
```

---

## 4.6 `equals()` Deep Dive

### The Contract (from JavaDoc)

`equals()` must be:



1. **Reflexive:** `x.equals(x)` returns `true`
2. **Symmetric:** If `x.equals(y)`, then `y.equals(x)`
3. **Transitive:** If `x.equals(y)` and `y.equals(z)`, then `x.equals(z)`
4. **Consistent:** Multiple calls return the same result (if objects don't change)
5. **Non-null:** `x.equals(null)` returns `false`

## `instanceof` vs `getClass()` — Symmetry Problem

```
class Animal {
    String name;
    Animal(String name) { this.name = name; }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Animal)) return false; // uses instanceof
        return name.equals(((Animal) o).name);
    }
}

class Dog extends Animal {
    String breed;
    Dog(String name, String breed) {
        super(name);
        this.breed = breed;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Dog)) return false;
        Dog d = (Dog) o;
        return name.equals(d.name) && breed.equals(d.breed);
    }
}

Animal a = new Animal("Rex");
```

```
Dog d = new Dog("Rex", "Lab");

System.out.println(a.equals(d)); // true – Animal sees Dog as Animal (ins
System.out.println(d.equals(a)); // false – Dog doesn't see Animal as Dog
// SYMMETRY BROKEN!
```

**Fix:** Use `getClass()` instead of `instanceof` when subclasses add state to `equals()`:

```
@Override
public boolean equals(Object o) {
    if (o == null || getClass() != o.getClass()) return false;
    // ...
}
```

---

## 4.7 hashCode() Deep Dive

### The Contract

1. If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` MUST be true
2. If `a.hashCode() != b.hashCode()`, then `a.equals(b)` MUST be false
3. If `a.hashCode() == b.hashCode()`, `a.equals(b)` MAY or MAY NOT be true (collision)

### Broken hashCode() Example

```
class Key {
    int id;
    Key(int id) { this.id = id; }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Key)) return false;
        return this.id == ((Key) o).id;
    }
}
```

```

    // NO hashCode override – uses Object's default (memory-based)
}

Map<Key, String> map = new HashMap<>();
map.put(new Key(1), "One");
System.out.println(map.get(new Key(1))); // null – different hashCode!

```

## Mutable Keys — Silent Data Loss

```

class MutableKey {
    int value;
    MutableKey(int v) { this.value = v; }

    @Override public boolean equals(Object o) {
        return o instanceof MutableKey && value == ((MutableKey) o).value;
    }
    @Override public int hashCode() { return value; }
}

Map<MutableKey, String> map = new HashMap<>();
MutableKey key = new MutableKey(1);
map.put(key, "Hello");

key.value = 2; // MUTATED!
System.out.println(map.get(key)); // null – hashCode changed, wrong bucket
System.out.println(map.get(new MutableKey(1))); // null – right bucket but wrong key
System.out.println(map.get(new MutableKey(2))); // null – right bucket but wrong key
// The entry is LOST – can't retrieve it

```

---

## 4.8 toString()

### Default Implementation

```
// In Object class:
public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
}
```

```
class Student { String name = "Alice"; }
System.out.println(new Student()); // Student@1b6d3586
```

## Override for Meaningful Output

```
class Student {
    String name;
    int age;

    @Override
    public String toString() {
        return "Student{name='" + name + "', age=" + age + "}";
    }
}
System.out.println(new Student()); // Student{name='null', age=0}
```

## Implicit toString() Calls

```
Student s = new Student();
System.out.println(s);           // calls toString()
String msg = "Student: " + s;    // calls toString() via string concatenation
System.out.println(msg);        // calls toString()
```

---

## 4.9 Garbage Collection

### What is Garbage Collection?

GC automatically reclaims memory occupied by objects that are no longer reachable.

## When is an Object Eligible for GC?

An object becomes eligible when there are **no live references** pointing to it.

```
// Case 1: Reference set to null
Student s = new Student();
s = null; // object eligible for GC

// Case 2: Reference reassigned
Student s = new Student("A");
s = new Student("B"); // "A" object eligible for GC

// Case 3: Object created in a method (local reference goes out of scope)
void method() {
    Student s = new Student(); // eligible after method returns
}

// Case 4: Island of isolation
class Node {
    Node next;
}
Node a = new Node();
Node b = new Node();
a.next = b;
b.next = a;
a = null;
b = null;
// Both objects reference each other but are unreachable from any live thr
```

## Key Points

- `System.gc()` is a **request**, not a command. JVM may ignore it.
  - GC is NOT guaranteed to run immediately.
  - `finalize()` is deprecated (Java 9+). Don't rely on it.
  - Memory leaks can still happen (e.g., static collections holding references, unclosed resources).
-

## 4.10 Object Lifecycle

1. Class Loading → Class loaded into memory (ClassLoader)
2. Memory Allocation → new keyword allocates heap memory
3. Initialization → Constructor + initializers run
4. Usage → Object used via references
5. Dereferencing → All references removed/nulled
6. GC Eligibility → Object is unreachable
7. Garbage Collection → GC reclaims memory (timing not guaranteed)

---

## 4.11 Class Loading

### Overview (Interview Level)

Loading → Linking → Initialization

■■■ Verification

■■■ Preparation

■■■ Resolution

1. **Loading:** ClassLoader reads the `.class` file bytecode
2. **Verification:** Bytecode is checked for correctness and security
3. **Preparation:** Static variables are allocated and set to default values
4. **Resolution:** Symbolic references resolved to actual references
5. **Initialization:** Static initializers and static blocks execute

### ClassLoader Hierarchy

Bootstrap ClassLoader (loads `java.lang.*`, `java.util.*`, etc.)

■■■ Platform ClassLoader (loads extension classes)

■■■ Application ClassLoader (loads your classes from classpath)

### Connection to Static Initialization

```

class Demo {
    static int x = compute();
    static { System.out.println("Static block: x=" + x); }

    static int compute() {
        System.out.println("Computing x");
        return 42;
    }
}

// First reference to Demo triggers class loading → initialization
Demo d = new Demo();
// Output:
// Computing x
// Static block: x=42

```

Static initialization happens ONCE, when the class is first loaded.

---

## 4.12 instanceof

### Basic Usage

```

class Animal {}
class Dog extends Animal {}
class Cat extends Animal {}

Dog d = new Dog();
System.out.println(d instanceof Dog);    // true
System.out.println(d instanceof Animal); // true
System.out.println(d instanceof Object); // true

Animal a = new Cat();
System.out.println(a instanceof Cat);    // true
System.out.println(a instanceof Dog);    // false
System.out.println(a instanceof Animal); // true

```

## null with instanceof

```
System.out.println(null instanceof Object); // false
System.out.println(null instanceof String); // false
// null is never an instance of anything
```

## With Interfaces

```
interface Flyable {}
class Bird implements Flyable {}

Bird b = new Bird();
System.out.println(b instanceof Flyable); // true
System.out.println(b instanceof Bird);    // true
```

## Pattern Matching (Java 16+)

```
Object obj = "Hello";

if (obj instanceof String s) {
    System.out.println(s.length()); // s is already cast to String
}
```

---

## 4.13 Covariant Return Type

The overriding method can return a subtype of the declared return type.

```
class Producer {
    Object produce() { return new Object(); }
}

class StringProducer extends Producer {
    @Override
    String produce() { return "Hello"; } // String is subtype of Object – v
}
```



```
Producer p = new StringProducer();  
Object result = p.produce(); // returns "Hello"
```

**Rules:** - Applies only to **reference types**, not primitives - `int` cannot be covariant with `Long` (primitives don't have inheritance) - The return type must be a **subtype** of the parent's return type

---

## 4.14 Exception Rules During Overriding

### Checked Exceptions

```
class Parent {  
    void m() throws IOException {}  
}  
  
class Child extends Parent {  
    // void m() throws IOException {}          // OK – same  
    // void m() throws FileNotFoundException {} // OK – subclass (narrower)  
    // void m() {}                             // OK – no exception  
    // void m() throws Exception {}            // COMPILE ERROR – broader  
    // void m() throws SQLException {}         // COMPILE ERROR – new/unrelated  
}
```

### Unchecked Exceptions — No Restrictions

```
class Parent {  
    void m() {}  
}  
  
class Child extends Parent {  
    @Override  
    void m() throws RuntimeException {} // OK  
    // void m() throws ArithmeticException {} // OK  
    // void m() throws NullPointerException {} // OK
```

```
}
```

### Why This Rule Exists

If a caller has `Parent p = new Child(); p.m();` and handles `IOException`, it would be wrong for `Child.m()` to throw `SQLException` — the caller isn't prepared for it.

---

## 4.15 Private Methods and Inheritance

Private methods are NOT visible to subclasses. They cannot be overridden.

```
class Parent {
    private void helper() {
        System.out.println("Parent helper");
    }

    public void process() {
        helper(); // calls Parent's helper – always
    }
}

class Child extends Parent {
    // This is a NEW method, not an override
    private void helper() {
        System.out.println("Child helper");
    }
}

Parent p = new Child();
p.process(); // "Parent helper" – no dynamic dispatch for private methods
```

---

## 4.16 Constructors + Inheritance — Complete Deep Dive

### Combined Example

```

class A {
    static { System.out.println("A static block"); }
    { System.out.println("A instance block"); }

    A() {
        System.out.println("A()");
    }
    A(int x) {
        this(); // calls A()
        System.out.println("A(int): " + x);
    }
}

class B extends A {
    static { System.out.println("B static block"); }
    { System.out.println("B instance block"); }

    B() {
        super(10); // calls A(int)
        System.out.println("B()");
    }
}

new B();

```

### Output:

```

A static block
B static block
A instance block
A()
A(int): 10
B instance block
B()

```

**Trace:** 1. Class loading: A static → B static 2. new B() → B() → super(10) → A(int) → this() → A() 3. Before A() body: A instance block runs → prints "A"

instance block" 4. A( ) body: prints "A()" 5. Returns to A(int): prints "A(int): 10" 6. Returns to B(): B instance block runs → prints "B instance block" 7. B( ) body: prints "B()"

---

## Unit 4 — Interview Section

### 30 Output Questions

Q1:

```
String a = "Hello";
String b = "Hello";
String c = new String("Hello");
System.out.println(a == b);
System.out.println(a == c);
System.out.println(a.equals(c));
```

**Output:** true, false, true

**Concept:** String pool vs heap object.

Q2:

```
void change(int[] arr) {
    arr[0] = 100;
    arr = new int[]{9, 8, 7};
}
int[] nums = {1, 2, 3};
change(nums);
System.out.println(nums[0] + " " + nums.length);
```

**Output:** 100 3

**Concept:** Mutation visible, reassignment not.

Q3:

```
void modify(StringBuilder sb) {
    sb.append(" World");
    sb = new StringBuilder("New");
}
```

```
}  
StringBuilder sb = new StringBuilder("Hello");  
modify(sb);  
System.out.println(sb);
```

**Output:** Hello World

**Concept:** `.append()` mutates; reassignment is local.

**Q4:**

```
String s = "Java";  
s.concat(" Rocks");  
s.toUpperCase();  
System.out.println(s);
```

**Output:** Java

**Concept:** String is immutable; results not assigned.

**Q5:**

```
Integer a = 127, b = 127;  
Integer c = 128, d = 128;  
System.out.println(a == b);  
System.out.println(c == d);  
System.out.println(a.equals(b));  
System.out.println(c.equals(d));
```

**Output:** true, false, true, true

**Concept:** Integer cache [-128, 127].

**Q6:**

```
void swap(Integer a, Integer b) {  
    Integer temp = a;  
    a = b;  
    b = temp;  
}  
Integer x = 1, y = 2;
```

```
swap(x, y);  
System.out.println(x + " " + y);
```

**Output:** 1 2

**Concept:** Pass-by-value — references not swapped.

**Q7:**

```
class A {  
    static { System.out.print("1 "); }  
    { System.out.print("2 "); }  
    A() { System.out.print("3 "); }  
}  
class B extends A {  
    static { System.out.print("4 "); }  
    { System.out.print("5 "); }  
    B() { System.out.print("6 "); }  
}  
new B();  
new B();
```

**Output:** 1 4 2 3 5 6 2 3 5 6

**Q8:**

```
final int[] arr = {1, 2, 3};  
arr[0] = 10;  
// arr = new int[]{4, 5, 6}; // COMPILE ERROR  
System.out.println(arr[0]);
```

**Output:** 10

**Concept:** final prevents reassignment, not mutation.

**Q9:**

```
String s1 = "ab" + "cd";  
String s2 = "abcd";  
System.out.println(s1 == s2);
```

**Output:** true

**Concept:** Compile-time constant folding — "ab" + "cd" is resolved to "abcd" at compile time.

**Q10:**

```
String a = "ab";
String b = a + "cd";
String c = "abcd";
System.out.println(b == c);
```

**Output:** false

**Concept:** a is a variable (not a compile-time constant), so a + "cd" creates a new object at runtime.

**Q11:**

```
final String a = "ab";
String b = a + "cd";
String c = "abcd";
System.out.println(b == c);
```

**Output:** true

**Concept:** a is a compile-time constant (final String literal), so a + "cd" is resolved at compile time to "abcd".

**Q12:**

```
class Test {
    int x;
    public boolean equals(Test t) { // WRONG signature!
        return this.x == t.x;
    }
}
Test a = new Test(); a.x = 5;
Test b = new Test(); b.x = 5;
Object c = b;
System.out.println(a.equals(b));
System.out.println(a.equals(c));
```

**Output:** true, false

**Concept:** equals(Test t) is OVERLOADING, not overriding. a.equals(c) calls Object.equals(Object) which uses ==.

**Q13:**

```
System.out.println(null instanceof Object);  
System.out.println(null instanceof String);
```

**Output:** false, false

**Concept:** null is never an instance of anything.

**Q14:**

```
Object o = "Hello";  
System.out.println(o instanceof String);  
System.out.println(o instanceof Object);  
System.out.println(o instanceof Integer);
```

**Output:** true, true, false

**Q15:**

```
class Parent {  
    int x = 10;  
    Parent() { show(); }  
    void show() { System.out.println("Parent: " + x); }  
}  
class Child extends Parent {  
    int x = 20;  
    void show() { System.out.println("Child: " + x); }  
}  
Parent p = new Child();  
System.out.println(p.x);
```

**Output:**



```
Child: 0  
10
```

**Concept:** Constructor calls overridden method (Child x=0), then p.x is Parent's field.

**Q16:**

```
class A implements Cloneable {  
    int[] data = {1, 2, 3};  
    protected A clone() throws CloneNotSupportedException {  
        return (A) super.clone(); // shallow copy  
    }  
}  
A a1 = new A();  
A a2 = a1.clone();  
a2.data[0] = 99;  
System.out.println(a1.data[0]);
```

**Output:** 99

**Concept:** `super.clone()` is shallow — both share the same array.

**Q17:**

```
String s = null;  
System.out.println("Hello" .equals(s));  
System.out.println(s.equals("Hello"));
```

**Output:** false, then `NullPointerException`

**Concept:** Always call `.equals()` on the known non-null object.

**Q18:**

```
void method(Object o) { System.out.println("Object"); }  
void method(String s) { System.out.println("String"); }  
void method(Integer i) { System.out.println("Integer"); }  
  
method(null);
```

**Output:** COMPILE ERROR — ambiguous. `String` and `Integer` are both equally specific.

**Q19:**

```
class A {  
    void show() { System.out.println("A"); }  
}  
class B extends A {  
    void show() { System.out.println("B"); }  
}  
A a = null;  
// a.show(); // what happens?
```

**Answer:** `NullPointerException` at runtime (compiles fine).

**Q20:**

```
class A {  
    static void show() { System.out.println("A"); }  
}  
A a = null;  
a.show();
```

**Output:** A

**Concept:** Static methods are resolved by reference type at compile time. `null` doesn't matter — no object is needed.

**Q21:**

```
Boolean b = null;  
if (b) { System.out.println("true"); }
```

**Output:** `NullPointerException`

**Concept:** Unboxing `null` to `boolean` throws NPE.

**Q22:**

```
System.out.println(1 + 2 + "3");  
System.out.println("1" + 2 + 3);
```

**Output:** 33, 123

**Concept:** Left-to-right evaluation.  $1+2=3$ , then  $3+"3"="33"$ .  $"1"+2="12"$ , then  $"12"+3="123"$ .

**Q23:**

```
Object[] objs = new String[3];  
objs[0] = "Hello";      // OK  
objs[1] = new Object(); // ?
```

**Output:** `ArrayStoreException` at runtime.

**Concept:** Array covariance in Java. Compiles, but runtime checks actual array type.

**Q24:**

```
class Box {  
    int size;  
    Box(int size) { this.size = size; }  
    public boolean equals(Object o) {  
        if (!(o instanceof Box)) return false;  
        return this.size == ((Box) o).size;  
    }  
}  
  
Set<Box> set = new HashSet<>();  
set.add(new Box(5));  
System.out.println(set.contains(new Box(5)));
```

**Output:** Likely false

**Concept:** `hashCode()` not overridden → different hash codes → different buckets.

**Q25:**

```
class A {  
    A() { this(10); System.out.print("A() "); }  
    A(int x) { System.out.print("A(int) "); }
```

```

}
class B extends A {
    B() { System.out.print("B() "); }
    B(int x) { this(); System.out.print("B(int) "); }
}
new B(5);

```

**Output:** A(int) A() B() B(int)

**Q26:**

```

String s1 = new String("Hello");
String s2 = new String("Hello");
System.out.println(s1 == s2);
System.out.println(s1.intern() == s2.intern());

```

**Output:** false, true

**Q27:**

```

class Base {
    void m() throws IOException { System.out.println("Base"); }
}
class Derived extends Base {
    void m() throws FileNotFoundException { System.out.println("Derived"); }
}
Base b = new Derived();
b.m();

```

**Output:** Derived

**Concept:** Valid override — `FileNotFoundException` is narrower than `IOException`.

**Q28:**

```

class Parent {
    static void show() { System.out.println("Parent"); }
}
class Child extends Parent {

```

```

    static void show() { System.out.println("Child"); }
}
Parent p = new Child();
p.show();
Child c = new Child();
c.show();

```

**Output:** Parent, Child

**Concept:** Static method hiding — reference type determines which runs.

**Q29:**

```

double d = 0.1 + 0.2;
System.out.println(d == 0.3);
System.out.println(d);

```

**Output:** false, 0.30000000000000004

**Concept:** Floating-point precision.

**Q30:**

```

class A {
    void show(Object o) { System.out.println("Object"); }
}
class B extends A {
    void show(String s) { System.out.println("String"); }
}
A a = new B();
a.show("Hello");

```

**Output:** Object

**Concept:** show(String) in B is overloading, not overriding. A reference only sees show(Object).

## 20 Tricky Conceptual Questions

1. **Is Java pass-by-reference?** → No. Always pass-by-value. For objects, the value of the reference (address) is copied.

2. **Can you swap two objects in Java?** → No. You can swap references within a method, but the caller's references are unaffected.
3. **What is the difference between shallow copy and deep copy?** → Shallow copies references; deep copies objects recursively.
4. **Why is `clone()` considered broken?** → It bypasses constructors, performs shallow copy, requires implementing `Cloneable`, and has complex contracts.
5. **Can `hashCode()` return the same value for all objects?** → Yes, it's legal but makes `HashMap`  $O(n)$  — terrible performance.
6. **What happens if you use a mutable object as a `HashMap` key?** → The entry may become unretrievable if the mutation changes the hash code.
7. **Why is `String` final?** → To guarantee immutability (a subclass could break it by adding mutating methods).
8. **What is `String.intern()`?** → Returns a canonical representation from the String pool.
9. **Can a `final` reference point to a mutable object?** → Yes. `final` prevents reassignment, not mutation.
10. **What is compile-time constant folding?** → `"ab" + "cd"` becomes `"abcd"` at compile time. But `variable + "cd"` doesn't.
11. **When is a `final String` resolved at compile time?** → When it's initialized with a literal: `final String s = "abc";`
12. **Can garbage collection cause a memory leak?** → GC prevents most leaks, but references held in static collections, unclosed resources, or listener registrations can still leak.
13. **What does `System.gc()` do?** → Suggests GC. JVM may ignore it.
14. **Is `finalize()` guaranteed to run?** → No. It's also deprecated since Java 9.
15. **Can `instanceof` return true for null?** → No. `null instanceof Anything` is always `false`.
16. **What is covariant return?** → Overriding method returns a subtype of the parent's return type.
17. **Can overriding add new checked exceptions?** → No. Can only narrow or remove checked exceptions.

18. **Why can't private methods be overridden?** → They aren't visible to subclasses. A child's same-named method is a new method.

19. **What happens calling an overridable method from a constructor?** → Dangerous — child's override runs before child's fields are initialized.

20. **What is the equals/hashCode contract?** → Equal objects must have equal hash codes. Unequal hash codes must mean unequal objects.

## 10 Debugging Questions

**D1:**

```
class Employee {
    String name;
    public boolean equals(Employee e) { // BUG: should be Object
        return this.name.equals(e.name);
    }
}
```

**Problem:** Overloads `equals()` instead of overriding it.

**Fix:** `public boolean equals(Object o)` with proper casting.

**D2:**

```
class Point {
    int x, y;
    public boolean equals(Object o) {
        Point p = (Point) o; // BUG: no null check, no type check
        return x == p.x && y == p.y;
    }
}
```

**Fix:** Add `if (o == null || getClass() != o.getClass()) return false;`

**D3:**

```
class Item {
    String name;
```

```

    public boolean equals(Object o) {
        if (!(o instanceof Item)) return false;
        return name.equals(((Item) o).name);
    }
    // BUG: hashCode() not overridden
}
Set<Item> set = new HashSet<>();
set.add(new Item("A"));
System.out.println(set.contains(new Item("A"))); // likely false

```

**Fix:** Override `hashCode()` to match `equals()`.

**D4:**

```

final class Immutable {
    private final List<String> items;
    Immutable(List<String> items) {
        this.items = items; // BUG: no defensive copy
    }
    List<String> getItems() { return items; } // BUG: returns mutable reference
}

```

**Fix:** `this.items = new ArrayList<>(items);` and return `Collections.unmodifiableList(items);`

**D5:**

```

String s = "Hello";
if (s == "Hello") {
    System.out.println("Equal");
}

```

**Problem:** Works coincidentally (string pool), but fragile. Use `.equals()`.

**D6:**

```

void changeValue(int x) {
    x = 100;
}

```



```

    System.out.println("Inside: " + x);
}
int val = 5;
changeValue(val);
System.out.println("Outside: " + val);

```

**Not a bug** but a misunderstanding: `val` is unchanged outside (pass-by-value).

**D7:**

```

class Parent {
    Parent() { init(); }
    void init() { System.out.println("Parent init"); }
}
class Child extends Parent {
    String data;
    void init() { data = "Ready"; }
    void show() { System.out.println(data.length()); }
}
new Child().show();

```

**Problem:** Works here, but if `init()` in `Child` used other uninitialized fields, it could NPE. Fragile design.

**D8:**

```

Map<int[], String> map = new HashMap<>();
int[] key = {1, 2, 3};
map.put(key, "value");
System.out.println(map.get(new int[]{1, 2, 3})); // null

```

**Problem:** Arrays use identity-based `hashCode()` and `equals()`. Different array objects are never equal via `==`.

**Fix:** Use `List<Integer>` as key instead.

**D9:**

```

class A {
    void show() throws Exception { }
}
class B extends A {
    void show() throws Exception, SQLException { } // BUG?
}

```

**Problem:** `SQLException` IS-A `Exception`, so this is actually fine (no new broader exception). But if it were `throws IOException, ClassNotFoundException` and parent had `throws IOException` — that would be a compile error.

**D10:**

```

class Counter {
    static int count = 0;
    Counter() { count++; }
    public boolean equals(Object o) {
        return o instanceof Counter; // BUG: all Counters are "equal"
    }
    public int hashCode() { return 1; } // BUG: all same hash code
}

```

**Problem:** Every `Counter` equals every other `Counter`. `HashMap` with `Counter` keys has only one bucket and every put overwrites.

## 10 "Predict: Compile Error / Runtime Exception / Output" Questions

**P1:** `int[] a = null; System.out.println(a.length);`

**Answer:** Runtime Exception — `NullPointerException`

**P2:** `int[] a = {}; System.out.println(a[0]);`

**Answer:** Runtime Exception — `ArrayIndexOutOfBoundsException`

**P3:** `String s = (String) new Object();`

**Answer:** Runtime Exception — `ClassCastException`

**P4:** `byte b = 128;`

**Answer:** Compile Error — 128 is out of byte range (-128 to 127)

**P5:** `abstract final class X {}`

**Answer:** Compile Error — `abstract` and `final` are contradictory

**P6:** `class A { void m() {} void m() {} }`

**Answer:** Compile Error — duplicate method

**P7:** `System.out.println(10/0);`

**Answer:** Runtime Exception — `ArithmeticException: / by zero`

**P8:** `System.out.println(10.0/0);`

**Answer:** Output — Infinity (floating-point division by zero)

**P9:** `class A { A() { this(1); } A(int x) { this(); } }`

**Answer:** Compile Error — recursive constructor invocation

**P10:** `class A extends B {} class B extends A {}`

**Answer:** Compile Error — cyclic inheritance

---

## UNIT 5 — OOP Design + SOLID + Design Patterns + Modern Java

---

### 5.1 SOLID Principles

#### S — Single Responsibility Principle (SRP)

**Definition:** A class should have only ONE reason to change. It should do ONE thing.

**Bad Code:**

```
class Employee {
    String name;
    double salary;

    void calculatePay() { /* pay logic */ }
    void saveToDatabase() { /* DB logic */ }
    void generateReport() { /* report logic */ }
}
// This class has 3 reasons to change: pay rules, DB schema, report format
```

## Good Code:

```
class Employee {
    String name;
    double salary;
}

class PayCalculator {
    double calculate(Employee e) { return e.salary * 1.0; }
}

class EmployeeRepository {
    void save(Employee e) { /* DB logic */ }
}

class ReportGenerator {
    String generate(Employee e) { return e.name + ": " + e.salary; }
}
```

**Interview Explanation:** "Each class has exactly one reason to change. If pay rules change, only `PayCalculator` changes. If DB changes, only `EmployeeRepository` changes."

**Common Misunderstanding:** SRP doesn't mean one method per class. It means one *responsibility* — one axis of change.

---

## O — Open/Closed Principle (OCP)

**Definition:** Classes should be **open for extension** but **closed for modification**.

## Bad Code:

```
class AreaCalculator {
    double calculate(Object shape) {
        if (shape instanceof Circle c) {
            return Math.PI * c.radius * c.radius;
        } else if (shape instanceof Rectangle r) {
            return r.width * r.height;
        }
    }
}
```

```
        // Adding Triangle means MODIFYING this class
        return 0;
    }
}
```

### Good Code:

```
interface Shape {
    double area();
}

class Circle implements Shape {
    double radius;
    Circle(double r) { this.radius = r; }
    public double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    double width, height;
    Rectangle(double w, double h) { this.width = w; this.height = h; }
    public double area() { return width * height; }
}

// Adding Triangle = new class, NO modification to existing code
class Triangle implements Shape {
    double base, height;
    Triangle(double b, double h) { this.base = b; this.height = h; }
    public double area() { return 0.5 * base * height; }
}
```

---

## L — Liskov Substitution Principle (LSP)

**Definition:** Objects of a superclass should be replaceable with objects of a subclass without breaking the program.

### Bad Code:

```
class Bird {
    void fly() { System.out.println("Flying"); }
}

class Penguin extends Bird {
    @Override
    void fly() {
        throw new UnsupportedOperationException("Penguins can't fly!");
    }
}

// Code that uses Bird:
void makeBirdFly(Bird b) {
    b.fly(); // breaks for Penguin – violates LSP
}
```

### Good Code:

```
interface Bird {
    void eat();
}

interface FlyingBird extends Bird {
    void fly();
}

class Sparrow implements FlyingBird {
    public void eat() { System.out.println("Eating"); }
    public void fly() { System.out.println("Flying"); }
}

class Penguin implements Bird {
    public void eat() { System.out.println("Eating fish"); }
    // No fly() – Penguin doesn't promise it can fly
}
```

---

## I — Interface Segregation Principle (ISP)

**Definition:** No client should be forced to depend on methods it does not use. Prefer smaller, focused interfaces.

### Bad Code:

```
interface Worker {  
    void code();  
    void test();  
    void design();  
    void manage();  
}  
  
class Developer implements Worker {  
    public void code() { /* ok */ }  
    public void test() { /* ok */ }  
    public void design() { /* not my job */ }  
    public void manage() { /* not my job */ }  
}
```

### Good Code:

```
interface Coder { void code(); }  
interface Tester { void test(); }  
interface Designer { void design(); }  
interface Manager { void manage(); }  
  
class Developer implements Coder, Tester {  
    public void code() { /* ok */ }  
    public void test() { /* ok */ }  
}  
  
class TeamLead implements Coder, Manager {  
    public void code() { /* ok */ }  
    public void manage() { /* ok */ }  
}
```

---

## D — Dependency Inversion Principle (DIP)

**Definition:** High-level modules should NOT depend on low-level modules. Both should depend on **abstractions**.

### Bad Code:

```
class MySQLDatabase {
    void save(String data) { System.out.println("Saving to MySQL"); }
}

class UserService {
    private MySQLDatabase db = new MySQLDatabase(); // tightly coupled!

    void createUser(String name) {
        db.save(name);
    }
}

// What if we want to switch to PostgreSQL? Must modify UserService.
```

### Good Code:

```
interface Database {
    void save(String data);
}

class MySQLDatabase implements Database {
    public void save(String data) { System.out.println("MySQL: " + data); }
}

class PostgresDatabase implements Database {
    public void save(String data) { System.out.println("Postgres: " + data); }
}

class UserService {
    private Database db; // depends on abstraction

    UserService(Database db) { // injected via constructor
    }
}
```



```
        this.db = db;
    }

    void createUser(String name) {
        db.save(name);
    }
}

// Usage
UserService service = new UserService(new PostgresDatabase());
```

---

## 5.2 Coupling vs Cohesion

| Aspect     | Coupling                                        | Cohesion                                   |
|------------|-------------------------------------------------|--------------------------------------------|
| Definition | How much one class depends on another           | How focused a class's responsibilities are |
| Goal       | LOW coupling                                    | HIGH cohesion                              |
| Bad        | Class A directly creates and uses B's internals | Class does 10 unrelated things             |
| Good       | Class A uses B through an interface             | Class does one thing well                  |

**Good Design = Low Coupling + High Cohesion**

---

## 5.3 Dependency Injection

### What is Dependency Injection?

Instead of a class creating its own dependencies, they are **injected** from outside.

## Constructor Injection (Preferred)

```
class OrderService {  
    private final PaymentGateway gateway;  
  
    OrderService(PaymentGateway gateway) {  
        this.gateway = gateway;  
    }  
  
    void placeOrder(double amount) {  
        gateway.pay(amount);  
    }  
}
```

## Setter Injection

```
class OrderService {  
    private PaymentGateway gateway;  
  
    void setGateway(PaymentGateway gateway) {  
        this.gateway = gateway;  
    }  
}
```

## Benefits

- **Testability** — Can inject mock objects for testing
- **Flexibility** — Can swap implementations without changing the class
- **Loose coupling** — Class depends on interface, not implementation

---

## 5.4 Singleton Pattern

### Problem

Ensure a class has exactly ONE instance and provide a global access point.

## Basic Implementation

```
class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

**Problem:** Not thread-safe. Two threads could both see `instance == null`.

## Thread-Safe — Synchronized Method

```
class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

**Problem:** Synchronization overhead on EVERY call.

## Double-Checked Locking

```

class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) { // first check (no lock)
            synchronized (Singleton.class) {
                if (instance == null) { // second check (with lock)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

`volatile` prevents instruction reordering that could expose a partially constructed object.

## Eager Initialization

```

class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() { return INSTANCE; }
}

```

Simple and thread-safe, but creates instance even if never used.

## Enum Singleton (RECOMMENDED)

```

enum Singleton {
    INSTANCE;

    public void doSomething() {
        System.out.println("Singleton action");
    }
}

```

```
}  
  
Singleton.INSTANCE.doSomething();
```

**Why recommended:** - Thread-safe by default - Serialization-safe (no duplicate instances) - Reflection-safe (cannot create enum instances via reflection) - Simple and concise

---

## 5.5 Factory Pattern

### Problem

Client code creates objects directly with `new`, coupling it to specific classes.

### Bad Code

```
class NotificationService {  
    void send(String type, String message) {  
        if (type.equals("email")) {  
            new EmailNotification().send(message);  
        } else if (type.equals("sms")) {  
            new SMSNotification().send(message);  
        }  
        // Adding push notification = modifying this class  
    }  
}
```

### Factory Solution

```
interface Notification {  
    void send(String message);  
}  
  
class EmailNotification implements Notification {  
    public void send(String msg) { System.out.println("Email: " + msg); }  
}
```

```

class SMSNotification implements Notification {
    public void send(String msg) { System.out.println("SMS: " + msg); }
}

class PushNotification implements Notification {
    public void send(String msg) { System.out.println("Push: " + msg); }
}

class NotificationFactory {
    static Notification create(String type) {
        return switch (type) {
            case "email" -> new EmailNotification();
            case "sms" -> new SMSNotification();
            case "push" -> new PushNotification();
            default -> throw new IllegalArgumentException("Unknown: " + type);
        };
    }
}

// Usage
Notification n = NotificationFactory.create("email");
n.send("Hello!");

```

---

## 5.6 Builder Pattern

### Problem — Telescoping Constructor

```

class Pizza {
    Pizza(int size) {}
    Pizza(int size, boolean cheese) {}
    Pizza(int size, boolean cheese, boolean pepperoni) {}
    Pizza(int size, boolean cheese, boolean pepperoni, boolean mushrooms) {}
    // This doesn't scale!
}

```

## Builder Solution

```
class Pizza {
    private final int size;
    private final boolean cheese;
    private final boolean pepperoni;
    private final boolean mushrooms;

    private Pizza(Builder builder) {
        this.size = builder.size;
        this.cheese = builder.cheese;
        this.pepperoni = builder.pepperoni;
        this.mushrooms = builder.mushrooms;
    }

    static class Builder {
        private final int size; // required
        private boolean cheese;
        private boolean pepperoni;
        private boolean mushrooms;

        Builder(int size) { this.size = size; }

        Builder cheese(boolean v) { this.cheese = v; return this; }
        Builder pepperoni(boolean v) { this.pepperoni = v; return this; }
        Builder mushrooms(boolean v) { this.mushrooms = v; return this; }

        Pizza build() { return new Pizza(this); }
    }

    @Override
    public String toString() {
        return "Pizza{size=" + size + ", cheese=" + cheese +
            ", pepperoni=" + pepperoni + ", mushrooms=" + mushrooms + "}";
    }
}
```

```
// Fluent API
Pizza p = new Pizza.Builder(12)
    .cheese(true)
    .pepperoni(true)
    .build();
```

---

## 5.7 Strategy Pattern

### Problem

Algorithm needs to vary at runtime.

### Solution

```
interface PaymentStrategy {
    void pay(double amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " via Credit Card");
    }
}

class UPIPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " via UPI");
    }
}

class WalletPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " via Wallet");
    }
}
```



```
class PaymentProcessor {
    private PaymentStrategy strategy;

    void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    void processPayment(double amount) {
        strategy.pay(amount);
    }
}

// Usage
PaymentProcessor processor = new PaymentProcessor();
processor.setStrategy(new UPIPayment());
processor.processPayment(500); // "Paid 500.0 via UPI"

processor.setStrategy(new CreditCardPayment());
processor.processPayment(1000); // "Paid 1000.0 via Credit Card"
```

---

## 5.8 Observer Pattern

### Problem

When one object changes state, multiple dependent objects need to be notified.

### Solution

```
import java.util.ArrayList;
import java.util.List;

interface Observer {
    void update(String event);
}

class EventManager {
```

```

    private List<Observer> observers = new ArrayList<>();

    void subscribe(Observer o) { observers.add(o); }
    void unsubscribe(Observer o) { observers.remove(o); }

    void notify(String event) {
        for (Observer o : observers) {
            o.update(event);
        }
    }
}

class EmailAlert implements Observer {
    public void update(String event) {
        System.out.println("Email alert: " + event);
    }
}

class SMSAlert implements Observer {
    public void update(String event) {
        System.out.println("SMS alert: " + event);
    }
}

// Usage
EventManager manager = new EventManager();
manager.subscribe(new EmailAlert());
manager.subscribe(new SMSAlert());
manager.notify("Order placed");
// Output:
// Email alert: Order placed
// SMS alert: Order placed

```

---

## 5.9 Records (Java 16+)

Records provide a compact syntax for immutable data carriers.

```
record Student(int id, String name) {}
```

This single line automatically generates: - `private final` fields - All-args constructor - `id()` and `name()` accessor methods (NOT `getId()`) - `equals()` based on all fields - `hashCode()` based on all fields - `toString()` like `Student[id=1, name=Alice]`

```
Student s = new Student(1, "Alice");
System.out.println(s.id());           // 1
System.out.println(s.name());         // "Alice"
System.out.println(s);                // Student[id=1, name=Alice]

Student s2 = new Student(1, "Alice");
System.out.println(s.equals(s2));     // true
```

### Records are:

- Implicitly `final` (cannot be extended)
- Cannot extend other classes (implicitly extend `java.lang.Record`)
- Can implement interfaces
- Can have custom methods, static fields, and custom constructors

---

## 5.10 Sealed Classes (Java 17+)

Sealed classes restrict which classes can extend them.

```
sealed class Shape permits Circle, Rectangle, Triangle {}

final class Circle extends Shape {
    double radius;
}

final class Rectangle extends Shape {
    double width, height;
}
```

```
non-sealed class Triangle extends Shape {  
    // non-sealed = any class can extend Triangle  
    double base, height;  
}
```

## Rules

- Permitted subclasses must be `final`, `sealed`, or `non-sealed`
- All permitted subclasses must be in the same module (or package for unnamed modules)
- Enables exhaustive pattern matching in `switch` (Java 21+)

## Why Sealed Classes Exist

- **Controlled inheritance** — You decide exactly who can extend your class
- **Exhaustive checks** — Compiler can verify all cases are handled
- **Better modeling** — Express that only specific types make sense as subtypes

---

## 5.11 Functional Interfaces

A functional interface has exactly ONE abstract method. It can be used with lambdas.

```
@FunctionalInterface  
interface Transformer {  
    String transform(String input);  
    // Can have default and static methods  
    default String transformTwice(String input) {  
        return transform(transform(input));  
    }  
}  
  
// Lambda usage  
Transformer upper = s -> s.toUpperCase();  
System.out.println(upper.transform("hello"));    // "HELLO"  
System.out.println(upper.transformTwice("hello")); // "HELLO"
```

## Built-in Functional Interfaces

| Interface                        | Method                         | Purpose                   |
|----------------------------------|--------------------------------|---------------------------|
| <code>Predicate&lt;T&gt;</code>  | <code>boolean test(T t)</code> | Test a condition          |
| <code>Function&lt;T,R&gt;</code> | <code>R apply(T t)</code>      | Transform input to output |
| <code>Consumer&lt;T&gt;</code>   | <code>void accept(T t)</code>  | Consume input, no return  |
| <code>Supplier&lt;T&gt;</code>   | <code>T get()</code>           | Supply a value, no input  |
| <code>Runnable</code>            | <code>void run()</code>        | Run with no input/output  |

---

## 5.12 OOP Design Problems

### Design 1: Payment System

**Step 1 — Requirements:** Process payments via multiple methods (credit card, UPI, wallet). Apply discounts. Generate receipts.

**Step 2 — Entities:** Payment, CreditCardPayment, UPIPayment, WalletPayment, Discount, Receipt

**Step 3 — Relationships:** Payment HAS-A Discount. Receipt is generated FROM Payment.

**Step 4 — Abstractions:** `PaymentMethod` interface for strategy.

**Step 5 — Composition over inheritance:** `PaymentProcessor` HAS-A `PaymentMethod` (composition/strategy).

**Step 6 — Interfaces:**

```
interface PaymentMethod {
    boolean pay(double amount);
    String getMethodName();
}
```

```
interface DiscountStrategy {
    double applyDiscount(double amount);
}
```

### Step 7 — Class Diagram:

```
PaymentProcessor --uses--> PaymentMethod (interface)
    CreditCardPayment
    UPIPayment
    WalletPayment
PaymentProcessor --uses--> DiscountStrategy (interface)
    PercentageDiscount
    FlatDiscount
PaymentProcessor --creates--> Receipt
```

### Step 8 — Implementation:

```
interface PaymentMethod {
    boolean pay(double amount);
    String getMethodName();
}

class CreditCardPayment implements PaymentMethod {
    private String cardNumber;
    CreditCardPayment(String cardNumber) { this.cardNumber = cardNumber; }
    public boolean pay(double amount) {
        System.out.println("Charged " + amount + " to card " + cardNumber);
        return true;
    }
    public String getMethodName() { return "Credit Card"; }
}

class UPIPayment implements PaymentMethod {
    private String upiId;
    UPIPayment(String upiId) { this.upiId = upiId; }
    public boolean pay(double amount) {
```

```

        System.out.println("Paid " + amount + " via UPI " + upiId);
        return true;
    }

    public String getMethodName() { return "UPI"; }
}

interface DiscountStrategy {
    double apply(double amount);
}

class PercentageDiscount implements DiscountStrategy {
    private double percent;

    PercentageDiscount(double percent) { this.percent = percent; }

    public double apply(double amount) { return amount * (1 - percent / 100); }
}

record Receipt(double originalAmount, double finalAmount, String method, boolean success) {}

class PaymentProcessor {
    private PaymentMethod method;
    private DiscountStrategy discount;

    PaymentProcessor(PaymentMethod method) { this.method = method; }

    void setDiscount(DiscountStrategy discount) { this.discount = discount; }

    Receipt process(double amount) {
        double finalAmount = discount != null ? discount.apply(amount) : amount;
        boolean success = method.pay(finalAmount);
        return new Receipt(amount, finalAmount, method.getMethodName(), success);
    }
}

```

**Step 9 — SOLID Applied:** - SRP: Each class has one job - OCP: New payment methods = new class, no modification - LSP: Any `PaymentMethod` can substitute another - ISP: Interfaces are focused - DIP: `PaymentProcessor` depends on interfaces

**Step 10 — Design Decisions:** Strategy pattern for payment methods. Composition for processor-to-method relationship.

**Step 11 — Interviewer Follow-ups:** - "How would you add refunds?" → Add `refund()` to `PaymentMethod` or create separate `RefundService`. - "How would you log transactions?" → Observer pattern or decorator. - "What if discount stacking is needed?" → Chain of responsibility or composite discount.

---

## Design 2: Notification System

```
interface NotificationChannel {
    void send(String recipient, String message);
}

class EmailChannel implements NotificationChannel {
    public void send(String recipient, String message) {
        System.out.println("Email to " + recipient + ": " + message);
    }
}

class SMSChannel implements NotificationChannel {
    public void send(String recipient, String message) {
        System.out.println("SMS to " + recipient + ": " + message);
    }
}

class PushChannel implements NotificationChannel {
    public void send(String recipient, String message) {
        System.out.println("Push to " + recipient + ": " + message);
    }
}

enum Priority { LOW, MEDIUM, HIGH, URGENT }

record Notification(String recipient, String message, Priority priority,
                    NotificationChannel channel) {
    void send() { channel.send(recipient, message); }
}
```



```

}

class NotificationService {
    private List<NotificationChannel> channels;

    NotificationService(List<NotificationChannel> channels) {
        this.channels = channels;
    }

    void broadcast(String message) {
        for (NotificationChannel ch : channels) {
            ch.send("all", message);
        }
    }
}

```

---

### Design 3: Parking Lot

```

enum VehicleSize { SMALL, MEDIUM, LARGE }

abstract class Vehicle {
    private String licensePlate;
    private VehicleSize size;

    Vehicle(String plate, VehicleSize size) {
        this.licensePlate = plate;
        this.size = size;
    }

    String getLicensePlate() { return licensePlate; }
    VehicleSize getSize() { return size; }
}

class Car extends Vehicle {
    Car(String plate) { super(plate, VehicleSize.MEDIUM); }
}

class Bike extends Vehicle {

```

```

    Bike(String plate) { super(plate, VehicleSize.SMALL); }
}
class Truck extends Vehicle {
    Truck(String plate) { super(plate, VehicleSize.LARGE); }
}

class ParkingSpot {
    private int id;
    private VehicleSize size;
    private Vehicle parkedVehicle;

    ParkingSpot(int id, VehicleSize size) { this.id = id; this.size = size; }

    boolean canFit(Vehicle v) { return parkedVehicle == null && v.getSize() < size; }

    boolean park(Vehicle v) {
        if (!canFit(v)) return false;
        parkedVehicle = v;
        return true;
    }

    Vehicle unpark() {
        Vehicle v = parkedVehicle;
        parkedVehicle = null;
        return v;
    }

    boolean isAvailable() { return parkedVehicle == null; }
}

class ParkingLot {
    private List<ParkingSpot> spots;

    ParkingLot(List<ParkingSpot> spots) { this.spots = spots; }

    ParkingSpot findSpot(Vehicle v) {
        return spots.stream().filter(s -> s.canFit(v)).findFirst().orElse(null);
    }
}

```

```

    }

    boolean parkVehicle(Vehicle v) {
        ParkingSpot spot = findSpot(v);
        return spot != null && spot.park(v);
    }
}

```

---

## Design 4: Library Management System

```

enum BookStatus { AVAILABLE, BORROWED, RESERVED }

class Book {
    private String isbn, title, author;
    private BookStatus status = BookStatus.AVAILABLE;

    Book(String isbn, String title, String author) {
        this.isbn = isbn; this.title = title; this.author = author;
    }
    // getters, setters for status
    String getIsbn() { return isbn; }
    String getTitle() { return title; }
    BookStatus getStatus() { return status; }
    void setStatus(BookStatus s) { this.status = s; }
}

class Member {
    private String id, name;
    private List<Book> borrowedBooks = new ArrayList<>();

    Member(String id, String name) { this.id = id; this.name = name; }

    boolean borrow(Book b) {
        if (borrowedBooks.size() >= 5) return false; // max limit
        if (b.getStatus() != BookStatus.AVAILABLE) return false;
        b.setStatus(BookStatus.BORROWED);
    }
}

```

```

        borrowedBooks.add(b);
        return true;
    }

    boolean returnBook(Book b) {
        b.setStatus(BookStatus.AVAILABLE);
        return borrowedBooks.remove(b);
    }
}

class Library {
    private Map<String, Book> catalog = new HashMap<>(); // isbn -> Book

    void addBook(Book b) { catalog.put(b.getIsbn(), b); }

    List<Book> searchByTitle(String title) {
        return catalog.values().stream()
            .filter(b -> b.getTitle().contains(title))
            .collect(Collectors.toList());
    }
}

```

---

## Design 5: Employee Management System

```

abstract class Employee {
    private String id, name;
    private double baseSalary;

    Employee(String id, String name, double baseSalary) {
        this.id = id; this.name = name; this.baseSalary = baseSalary;
    }

    abstract double calculateSalary();
    String getName() { return name; }
    double getBaseSalary() { return baseSalary; }
}

```

```

class FullTimeEmployee extends Employee {
    private double bonus;
    FullTimeEmployee(String id, String name, double salary, double bonus) {
        super(id, name, salary);
        this.bonus = bonus;
    }
    double calculateSalary() { return getBaseSalary() + bonus; }
}

class ContractEmployee extends Employee {
    private int hoursWorked;
    ContractEmployee(String id, String name, double hourlyRate, int hours) {
        super(id, name, hourlyRate);
        this.hoursWorked = hours;
    }
    double calculateSalary() { return getBaseSalary() * hoursWorked; }
}

class InternEmployee extends Employee {
    InternEmployee(String id, String name, double stipend) {
        super(id, name, stipend);
    }
    double calculateSalary() { return getBaseSalary(); }
}

class Payroll {
    void processPayroll(List<Employee> employees) {
        for (Employee e : employees) {
            System.out.println(e.getName() + ": " + e.calculateSalary());
        }
    }
}

```

---

## Design 6: Food Delivery Order System

```

enum OrderStatus { PLACED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED }

record MenuItem(String name, double price) {}

class Order {
    private String orderId;
    private List<MenuItem> items = new ArrayList<>();
    private OrderStatus status = OrderStatus.PLACED;

    Order(String orderId) { this.orderId = orderId; }

    void addItem(MenuItem item) { items.add(item); }

    double getTotal() {
        return items.stream().mapToDouble(MenuItem::price).sum();
    }

    void updateStatus(OrderStatus status) { this.status = status; }
    OrderStatus getStatus() { return status; }
}

interface DeliveryStrategy {
    double calculateFee(double distance);
}

class StandardDelivery implements DeliveryStrategy {
    public double calculateFee(double dist) { return dist * 5; }
}

class ExpressDelivery implements DeliveryStrategy {
    public double calculateFee(double dist) { return dist * 10; }
}

```

---

## Design 7: Ride/Vehicle System

```

interface Rideable {
    double calculateFare(double distance);
    int getCapacity();
}

class AutoRickshaw implements Rideable {
    public double calculateFare(double dist) { return 20 + dist * 8; }
    public int getCapacity() { return 3; }
}

class Cab implements Rideable {
    public double calculateFare(double dist) { return 50 + dist * 12; }
    public int getCapacity() { return 4; }
}

class PremiumCab implements Rideable {
    public double calculateFare(double dist) { return 100 + dist * 18; }
    public int getCapacity() { return 4; }
}

class RideService {
    Rideable selectVehicle(String type) {
        return switch (type) {
            case "auto" -> new AutoRickshaw();
            case "cab" -> new Cab();
            case "premium" -> new PremiumCab();
            default -> throw new IllegalArgumentException("Unknown type");
        };
    }
}

```

---

## Design 8: Coffee Machine

```

interface Beverage {
    String getDescription();
    double getCost();
}

```

```

}

class Espresso implements Beverage {
    public String getDescription() { return "Espresso"; }
    public double getCost() { return 100; }
}

class Latte implements Beverage {
    public String getDescription() { return "Latte"; }
    public double getCost() { return 150; }
}

// Decorator pattern for add-ons
abstract class BeverageDecorator implements Beverage {
    protected Beverage beverage;
    BeverageDecorator(Beverage b) { this.beverage = b; }
}

class MilkDecorator extends BeverageDecorator {
    MilkDecorator(Beverage b) { super(b); }
    public String getDescription() { return beverage.getDescription() + " + Milk"; }
    public double getCost() { return beverage.getCost() + 20; }
}

class SugarDecorator extends BeverageDecorator {
    SugarDecorator(Beverage b) { super(b); }
    public String getDescription() { return beverage.getDescription() + " + Sugar"; }
    public double getCost() { return beverage.getCost() + 10; }
}

// Usage
Beverage coffee = new SugarDecorator(new MilkDecorator(new Espresso()));
System.out.println(coffee.getDescription()); // Espresso + Milk + Sugar
System.out.println(coffee.getCost()); // 130.0

```

---

## Design 9: Vending Machine



```

enum Product {
    WATER(20), COKE(40), CHIPS(30), CHOCOLATE(50);
    final double price;
    Product(double p) { this.price = p; }
}

class VendingMachine {
    private Map<Product, Integer> inventory = new HashMap<>();
    private double balance = 0;

    VendingMachine() {
        for (Product p : Product.values()) inventory.put(p, 10);
    }

    void insertMoney(double amount) { balance += amount; }

    String dispense(Product product) {
        if (inventory.getOrDefault(product, 0) <= 0) return "Out of stock";
        if (balance < product.price) return "Insufficient balance. Need " +
            balance -= product.price;
        inventory.put(product, inventory.get(product) - 1);
        return "Dispensed " + product + ". Change: " + balance;
    }

    double getBalance() { return balance; }
    void refund() { System.out.println("Refunded: " + balance); balance = 0; }
}

```

---

## Design 10: File Storage System

```

interface StorageService {
    void upload(String fileName, byte[] data);
    byte[] download(String fileName);
    boolean delete(String fileName);
    List<String> listFiles();
}

```

```

class LocalStorage implements StorageService {
    private Map<String, byte[]> store = new HashMap<>();

    public void upload(String name, byte[] data) { store.put(name, data); }
    public byte[] download(String name) { return store.get(name); }
    public boolean delete(String name) { return store.remove(name) != null; }
    public List<String> listFiles() { return new ArrayList<>(store.keySet())
}

class CloudStorage implements StorageService {
    public void upload(String name, byte[] data) {
        System.out.println("Uploading " + name + " to cloud");
    }
    public byte[] download(String name) {
        System.out.println("Downloading " + name + " from cloud");
        return new byte[0];
    }
    public boolean delete(String name) {
        System.out.println("Deleting " + name + " from cloud");
        return true;
    }
    public List<String> listFiles() { return List.of(); }
}

class FileManager {
    private StorageService storage;
    FileManager(StorageService storage) { this.storage = storage; }

    void upload(String name, byte[] data) { storage.upload(name, data); }
    byte[] download(String name) { return storage.download(name); }
}

```

---

## FINAL SECTION — INTERVIEW MASTER PRACTICE

# Java OOP Interview Question Bank

## A. Basic Questions (30 Questions)

**What is Object-Oriented Programming (OOP)?** *Answer:* OOP is a programming paradigm based on the concept of "objects," which can contain data (fields/attributes) and code (methods/behaviors). It focuses on organizing software around data rather than functions and logic.

**What are the four main pillars of OOP?** *Answer:* The four pillars are Encapsulation (data hiding and bundling), Inheritance (code reuse and hierarchies), Polymorphism (one interface, many implementations), and Abstraction (hiding implementation details).

**What is a class in Java?** *Answer:* A class is a blueprint or template from which individual objects are created. It defines the state (fields) and behavior (methods) that its instances will have.

**What is an object in Java?** *Answer:* An object is an instance of a class. It has state, behavior, and identity. It occupies memory on the heap.

**What is the difference between a class and an object?** *Answer:* A class is a logical template/blueprint that does not occupy memory (except class metadata in Metaspace). An object is a physical instance of that blueprint that occupies memory on the heap.

**What is a constructor in Java?** *Answer:* A constructor is a block of code similar to a method that is called when an instance of an object is created. It has no return type and has the same name as the class.

**What is the default constructor?** *Answer:* If no constructor is written in a class, the compiler automatically generates a no-argument constructor that calls the parent class's no-arg constructor via `super ( )`.

**What is constructor overloading?** *Answer:* Defining multiple constructors within the same class, each having a different parameter list (different number, types, or order of parameters).

**Can a constructor be private?** *Answer:* Yes. Private constructors are used to prevent direct instantiation of a class (e.g., in Utility classes, Singleton patterns, or Factory patterns).

**What is the `this` keyword in Java?** *Answer:* `this` is a reference variable that refers to the current object. It is used to access fields, invoke methods, or chain constructors within the same class.

**What is the `super` keyword in Java?** *Answer:* `super` is a reference variable used to refer to the immediate parent class object. It can be used to call parent constructors, fields, or overridden methods.

**What is the difference between `this()` and `super()`?** *Answer:* `this()` invokes another constructor in the same class, while `super()` invokes a constructor of the immediate parent class. Both must be the first statement in a constructor.

**What is a static variable?** *Answer:* A static variable (or class variable) belongs to the class itself rather than instances. Only one copy of the static variable exists in memory, shared by all instances.

**What is a static method?** *Answer:* A static method belongs to the class and can be invoked without creating an instance. It can only access other static members directly and cannot use `this` or `super`.

**What is a static block?** *Answer:* A static block is a block of code inside a class that runs once when the class is first loaded into the JVM. It is commonly used to initialize static variables.

**What is inheritance?** *Answer:* Inheritance is a mechanism where one class (subclass/child) inherits the fields and methods of another class (superclass/parent), promoting code reuse.

**Which keyword is used to implement inheritance in Java?** *Answer:* The `extends` keyword is used for inheritance between classes, and the `implements` keyword is used to implement interfaces.

**Why does Java not support multiple inheritance with classes?** *Answer:* To avoid ambiguity and compiler complexity, such as the Diamond Problem, where a class inherits conflicting implementations of the same method from two different parent classes.

**What is method overloading?** *Answer:* Having multiple methods in the same class with the same name but different parameter signatures (compile-time polymorphism).

**What is method overriding?** *Answer:* Providing a specific implementation in a subclass for a method that is already defined in its superclass (runtime polymorphism).

**What is compile-time (static) polymorphism?** *Answer:* Polymorphism resolved at compile time, typically achieved via method overloading. The compiler decides which method to call based on reference type and arguments.

**What is runtime (dynamic) polymorphism?** *Answer:* Polymorphism resolved at runtime, achieved via method overriding and dynamic method dispatch. The JVM decides which method to run based on the actual object type.

**What is encapsulation?** *Answer:* Encapsulation is the wrapping of data (variables) and code (methods) together as a single unit, keeping fields private and providing public getters/setters to validate access.

**What is abstraction?** *Answer:* Abstraction is the process of hiding implementation details and showing only the essential features of an object, achieved via abstract classes and interfaces.

**What is an abstract class?** *Answer:* A class declared with the `abstract` keyword that cannot be instantiated. It can contain both abstract methods (without bodies) and concrete methods (with bodies).

**What is an interface?** *Answer:* A blueprint of a class that contains public static final constants and abstract/default/static methods. It represents a contract that implementing classes must fulfill.

**What is the default access modifier in Java?** *Answer:* If no modifier is specified, it is package-private (default). It allows access only to classes within the same package.

**What does the `final` keyword mean for a class?** *Answer:* A `final` class cannot be subclassed (inherited). Example: `java.lang.String` is final.

**What does the `final` keyword mean for a method?** *Answer:* A `final` method cannot be overridden by any subclass.

**What is the root class of all classes in Java?** *Answer:* `java.lang.Object` is the root class of the entire Java class hierarchy.

---

## B. Intermediate Questions (40 Questions)

**What is the difference between a default constructor and a no-arg constructor?**

*Answer:* A default constructor is automatically created by the compiler if no constructors are defined. A no-arg constructor is written explicitly by the developer and takes no arguments.

**Can we overload the main method in Java?** *Answer:* Yes. You can define other `main` methods with different parameters, but the JVM will only call the standard `public static void main(String[] args)` method.

**Can we override static methods?** *Answer:* No. Static methods belong to the class, not the object. Declaring a static method in a subclass with the same signature is called **method hiding**, not overriding.

**Can we override private methods?** *Answer:* No. Private methods are not visible to subclasses, so they cannot be inherited or overridden.

**What is the difference between `==` and `equals()` in Java?** *Answer:* `==` is a binary operator that compares memory addresses (reference equality) for objects or values for primitives. `equals()` is a method in the `Object` class that can be overridden to compare object states (content equality).

**What is the contract between `equals()` and `hashCode()`?** *Answer:* If two objects are equal according to the `equals(Object)` method, they must return the same integer value from their `hashCode()` method.

**What happens if you override `equals()` but not `hashCode()`?** *Answer:* It violates the contract. Hash-based collections (like `HashMap` or `HashSet`) will fail to retrieve or store objects correctly because equal objects might have different hashcodes, landing in different buckets.

**What is covariant return type?** *Answer:* An overriding method in a subclass can return a subtype of the return type declared in the parent method. This is only valid for reference types.

**What is method hiding in Java?** *Answer:* If a subclass defines a static method with the same signature as a static method in the superclass, the subclass method hides the superclass method. The method called is determined by the reference type.

**What is field hiding in Java?** *Answer:* If a subclass defines a variable with the same name as a variable in the superclass, the subclass variable hides the parent variable. Fields are not polymorphic, and resolution is based on the reference type.

**What is upcasting? Is it safe?** *Answer:* Upcasting is casting a subclass object reference to a superclass reference type. It is implicit, safe, and handled automatically by the compiler.

**What is downcasting? When is it used?** *Answer:* Downcasting is casting a superclass reference back to a subclass reference type. It must be explicit and can throw a `ClassCastException` at runtime if the object is not of the target subtype.

**What is the role of the `instanceof` operator?** *Answer:* It tests whether an object reference points to an instance of a specific class or interface type. It returns a boolean and is safe to use with null (returns false).

**What is pattern matching for `instanceof`?** *Answer:* Introduced as a standard feature in Java 16, it allows casting an object implicitly if the type check passes: `if (obj instanceof String s) { System.out.println(s.length()); }`.

**What is the difference between an abstract class and an interface?** *Answer:*

Abstract classes can have instance variables, constructors, and non-public members.

Interfaces can only have public static final fields, cannot have constructors, and support multiple inheritance.

**What are default methods in interfaces? Why were they introduced?** *Answer:*

Introduced in Java 8, default methods have a body inside an interface and are declared with the `default` keyword. They allow adding new methods to interfaces without breaking existing implementations.

**What is the difference between Association, Aggregation, and Composition?**

*Answer:* Association is a general relationship. Aggregation is a weak "has-a" relationship where the child can outlive the parent. Composition is a strong "has-a" relationship where the child's lifetime is bound to the parent.

**What is delegation?** *Answer:* Delegation is an alternative to inheritance where an object

routes a request to a helper object: `class Printer { void print() { helper.print(); } }`.

**What is an inner class?** *Answer:* A non-static nested class defined inside another class.

It has access to all members of the outer class, including private ones, and requires an instance of the outer class to be instantiated.

**What is a static nested class?** *Answer:* A nested class declared as static. It does not require an outer class instance and can only access static members of the outer class.

**What is a local inner class?** *Answer:* A class defined inside a block or method. It is only visible inside that block and can access local variables only if they are final or effectively final.

**What is an anonymous inner class?** *Answer:* A class without a name defined and instantiated in a single expression, typically used to provide one-off implementations of interfaces or abstract classes.

**What is the difference between a shallow copy and a deep copy?** *Answer:* A shallow copy copies only the object references, meaning the copied object shares references to mutable fields. A deep copy creates new instances of all nested mutable objects, isolating the copy.

**Can we make an abstract class final?** *Answer:* No. Abstract classes must be extended to be useful, while final classes cannot be extended. Combining them is a compile-time error.

**Can an interface be final?** *Answer:* No. Interfaces must be implemented to be useful, while final prevents implementation. It is a compile-time error.

**What is the difference between private, default, protected, and public?** *Answer:* Private is same-class only. Default is same-package only. Protected is same-package + subclasses. Public is accessible everywhere.

**Can a subclass method restrict the access modifier of the overridden parent method?** *Answer:* No. The overriding method's modifier must be equal to or wider (more accessible) than the parent's. Narrowing access is a compile-time error.

**What checked exceptions can an overriding method throw?** *Answer:* The overriding method can only throw the same checked exception, a subclass of it, or no checked exception at all. It cannot throw broader or new checked exceptions.

**Are there any restrictions on unchecked exceptions when overriding?** *Answer:* No. Overriding methods can throw any runtime (unchecked) exceptions, regardless of what the parent method declares.

**What is the role of `super ( )` in constructors?** *Answer:* It invokes the parent class constructor. If not explicitly declared, the compiler inserts a call to `super ( )` automatically as the first line of the child constructor.

**Why is calling overridable methods in constructors dangerous?** *Answer:* If the parent constructor calls an overridden method, the subclass version will execute before the subclass fields have been initialized, leading to bugs or `NullPointerExceptions`.

**What is class loading in Java?** *Answer:* It is the process by which JVM ClassLoaders read `.class` files, load bytecode into memory (Metaspace), verify it, resolve symbols, and initialize static variables.

**What is the order of initialization in a class when an object is instantiated?** *Answer:* Static variables/blocks (textual order) -> Instance variables/blocks (textual order) -> Constructor body.

**What is the difference between a final field and an effectively final variable?** *Answer:* A final field is explicitly declared with the `final` keyword. An effectively final variable is not declared final, but its value is never changed after initialization.

**What is a marker interface?** *Answer:* An interface with no methods or fields, used to deliver metadata or instructions to the compiler or JVM (e.g., `Serializable`, `Cloneable`).

**What are the drawbacks of using inheritance?** *Answer:* Tight coupling between parent and child (breaking encapsulation), rigid structure determined at compile time, and



potential inheritance of unwanted behavior.

**How does composition improve testability?** *Answer:* By composing dependencies rather than inheriting them, you can easily mock or stub those dependencies during unit testing.

**Why are static methods not polymorphic?** *Answer:* Polymorphism relies on runtime dynamic dispatch (finding methods based on the heap object). Static methods are linked to the class at compile time (static binding).

**Can a subclass inherit constructors?** *Answer:* No. Subclasses do not inherit parent constructors. They must write their own or rely on default constructors, which chain to the parent constructor via `super()`.

**Why does Java allow multiple inheritance of interfaces but not classes?** *Answer:* Interfaces do not carry state (instance variables) and did not traditionally carry method implementations, which prevents state conflicts or structural inheritance complexity.

---

## C. Advanced Questions (40 Questions)

**How does dynamic method dispatch work under the hood in the JVM?** *Answer:* The JVM uses a method table called the **vtable** (virtual method table) for each class. When an overridden instance method is called, the JVM looks up the actual runtime object's vtable to dispatch the call.

**What is the difference between static binding and dynamic binding?** *Answer:* Static binding (compile-time) occurs for static, private, and final methods, which are resolved by the compiler. Dynamic binding (runtime) occurs for overridable instance methods, resolved by the JVM based on the object type.

**Why does Java have Integer caches? What are the implications?** *Answer:* To save memory, Java caches Integer objects for values between -128 and 127. Comparing cached Integers with `==` returns true, but values outside this range return false, which is a common interview trap.

**How do you design a thread-safe Singleton using double-checked locking?**

**Explain the role of `volatile`.** *Answer:* Use a volatile static field. Double-check `instance == null` before and after synchronization. `volatile` prevents instruction reordering during instantiation (allocating memory -> writing address -> running constructor).

**What are records in Java? How do they differ from normal classes?** *Answer:*

Records (introduced in Java 16) are final classes designed to be pure data carriers. They automatically generate final fields, constructor, accessors, `equals()`, `hashCode()`, and `toString()`. They cannot extend any class.

**What are sealed classes in Java? What is their benefit?** *Answer:* Sealed classes (introduced in Java 17) restrict which classes can extend or implement them using the `permits` clause. They enable compile-time exhaustiveness checks in switch expressions.

**What is the Liskov Substitution Principle (LSP)? Give an example of a violation.**

*Answer:* LSP states that subclasses must be substitutable for their superclasses. A classic violation is the `Rectangle` class extended by a `Square` where overriding `setWidth` changes the height, breaking `Rectangle`'s invariants.

**Explain the Dependency Inversion Principle (DIP). How does it relate to**

**Dependency Injection (DI)?** *Answer:* DIP states that high-level modules should depend on abstractions (interfaces), not concrete implementations. DI is the practice of passing these abstractions into classes (e.g., via constructor injection).

**How would you implement a deep copy using serialization?** *Answer:* Serialize the object into a byte array stream and immediately deserialize it back. This creates a fully independent clone of the object and all its reachable nested objects.

**Why is `clone()` deprecated in favor of copy constructors?** *Answer:* `clone()`

does not call constructors, is shallow by default, forces throwing `CloneNotSupportedException`, and relies on a magic JVM mechanism instead of standard Java code.

**What is defensive copying? Where is it critical?** *Answer:* Defensive copying is copying mutable inputs/outputs in constructors and getters to prevent external code from modifying the internal state of an immutable class.

**What are the rules for creating an immutable class in Java?** *Answer:* Class must be final; all fields must be private and final; no setters; perform defensive copying for all mutable fields; return unmodifiable views for collections.

**How does `String.intern()` interact with the String Pool?** *Answer:* It checks if the string exists in the String pool. If yes, it returns the pool reference. If no, it adds the string to the pool and returns that reference.

**What is the Diamond Problem? How is it resolved in Java 8+ with default methods?** *Answer:* The Diamond Problem is when a class implements two interfaces

that provide the same default method signature. The compiler rejects this and forces the implementing class to override and resolve the conflict.

**Why can't non-static inner classes define static members (before Java 16)?**

*Answer:* Inner classes are bound to an instance of the outer class. Having static fields would mean class-level state that violates their instance-dependent nature. Java 16+ relaxed this.

**What is the difference between `getClass()` and `instanceof` inside `equals()`?**

*Answer:* `instanceof` returns true for subclasses, which breaks symmetry if the subclass overrides `equals`. `getClass() == o.getClass()` ensures strict type equality, which is symmetric.

**How does the JVM garbage collector detect unreachable objects?** *Answer:* Using Root Reachability Analysis. It starts from GC Roots (thread stack frames, static variables, JNI references) and traces references. Objects not reachable from these roots are collected.

**What is an "Island of Isolation" in Java Garbage Collection?** *Answer:* A group of objects that reference each other, but have no references from any live GC Roots. The JVM detects that the entire group is unreachable and collects them.

**What is the difference between high-cohesion and low-coupling?** *Answer:* High-cohesion means a class does one cohesive thing. Low-coupling means classes do not depend on the implementation details of each other, communicating through interfaces instead.

**What is the Open/Closed Principle? Give a real-world coding scenario.** *Answer:* Open for extension, closed for modification. If you need a new shipping rate calculator, you write a new class implementing the `ShippingCalculator` interface instead of editing the existing one.

**What is the Interface Segregation Principle?** *Answer:* Clients should not be forced to implement interface methods they don't use. Split large interfaces into smaller, specialized ones (e.g., `Readable` and `Writable`).

**How does Constructor Injection prevent circular dependencies?** *Answer:* If Class A requires B in its constructor, and B requires A, the JVM cannot instantiate either without the other, causing a compile-time or startup error, forcing cleaner design.

**What is the strategy pattern? How does it differ from state pattern?** *Answer:* Strategy pattern configures a class with an algorithm at runtime. State pattern changes class behavior dynamically as its internal state changes.

**Explain the observer pattern. How do you implement it in modern Java without deprecated APIs?** *Answer:* Use custom listener interfaces or standard functional interfaces like `Consumer<Event>` instead of the deprecated `java.util.Observer` and `Observable`.

**What is the difference between Eager Initialization and Lazy Initialization in Singleton?** *Answer:* Eager creates the instance when the class is loaded. Lazy delays instance creation until `getInstance()` is called, saving memory but requiring synchronization.

**What is the Builder pattern? When is it preferred over factory?** *Answer:* Builder creates complex objects step-by-step using a fluent API. It is preferred when objects have many optional parameters, avoiding telescoping constructors.

**Can a nested class access the private variables of its outer class?** *Answer:* Yes. Both static nested classes and inner classes have full access to the private members of their enclosing outer class.

**Why are local variables accessed by local inner classes required to be final or effectively final?** *Answer:* Because the local class instance may outlive the method execution (heap vs stack). The JVM copies the local variable value into the inner class instance, requiring the value to be constant to maintain consistency.

**What is covariant exception throwing?** *Answer:* An overriding subclass method can declare to throw a subclass (more specific type) of the checked exceptions declared in the parent method signature.

**Does Java support pass-by-reference for arrays?** *Answer:* No. Arrays are objects. Java passes a copy of the reference pointing to the array on the heap. You can mutate array contents, but reassigning the array reference does not affect the caller's array.

---

## G. "Why?" Questions (30 Questions)

**Why is Java not a purely object-oriented language?** *Answer:* Because it supports primitive data types (`int`, `char`, `double`, etc.) which are not objects and do not inherit from `Object`.

**Why does Java not support multiple class inheritance but supports multiple interface inheritance?** *Answer:* Multiple class inheritance leads to the Diamond Problem where method implementation conflicts arise from having state and multiple concrete method bodies. Interfaces traditionally lacked state and method bodies, eliminating these conflicts.

**Why must `this()` or `super()` be the first statement in a constructor?** *Answer:* To ensure that parent class variables and state are fully constructed and initialized before the subclass executes any of its own initialization logic.

**Why cannot static methods access instance variables?** *Answer:* Static methods belong to the class and are loaded when the class is loaded. They can be run without any instances. Instance variables exist only inside specific objects on the heap.

**Why can't we override static methods?** *Answer:* Overriding relies on runtime dynamic dispatch (dynamic binding) based on the heap object. Static methods belong to the class, not instances, and are resolved at compile time (static binding).

**Why can't a constructor be static?** *Answer:* A constructor is used to initialize a new instance of an object, which implies an object-level context. `static` implies a class-level context without an instance.

**Why does Java use a String Pool?** *Answer:* To conserve memory. Strings are highly common. Storing literals in a pool allows identical string values to share the same memory slot on the heap.

**Why are Strings immutable in Java?** *Answer:* For security (network/DB credentials), caching `hashCode()` for hash collections, maintaining safety in the String Pool, and ensuring thread-safety without locks.

**Why are array declarations covariant in Java?** *Answer:* Covariant arrays (e.g., `Object[] objs = new String[5]`) were introduced early in Java to support writing generic algorithms (like sorting) before Generics were added in Java 5.

**Why does overriding method not allow throwing broader checked exceptions?**

*Answer:* To preserve polymorphism. A client holding a superclass reference expects to catch only the exceptions declared by the superclass. Throwing a broader checked exception would bypass compile-time catch checks.

**Why is composition preferred over inheritance?** *Answer:* It maintains encapsulation, reduces tight coupling between classes, allows changing helper behaviors at runtime (flexible strategies), and avoids complex hierarchies.

**Why is `equals()` and `hashCode()` contract necessary for HashMaps?** *Answer:* `HashMap` uses `hashCode()` to find the correct bucket, and `equals()` to find the specific key inside that bucket. Failing the contract prevents matching keys.

**Why does `null instanceof String` return false?** *Answer:* `instanceof` checks if the referenced object is of a specific type. Since `null` does not point to any object, it cannot be an instance of any class.

**Why can't abstract methods be final?** *Answer:* `abstract` means a method must be overridden in a subclass, while `final` prevents the method from being overridden. They are logically contradictory.

**Why can't we declare private methods inside an interface before Java 9?** *Answer:* Interfaces define public contracts. With default methods in Java 8, private methods became necessary to share common code between default methods without exposing them to the API.

**Why is `volatile` required in double-checked locking?** *Answer:* The compilation process can reorder instructions (allocating memory -> writing address -> running constructor). `volatile` prevents this reordering, ensuring threads don't see a partially constructed instance.

**Why do we need defensive copying for mutable fields in immutable objects?**

*Answer:* Because if you store a reference to a mutable object directly, external code can still modify the mutable object, thereby changing the state of your "immutable" object.

**Why do local inner classes need variables to be final or effectively final?** *Answer:* The local inner class object may outlive the stack frame of the method. The JVM copies the variables into the inner class instance; they must be final to avoid data inconsistency.

**Why is the `clone()` method protected in the `Object` class?** *Answer:* To force class developers to implement the `Cloneable` interface and override the `clone()` method explicitly, preventing random, potentially unsafe shallow copying of all objects.

**Why is `getClass() != o.getClass()` safer than `o instanceof Class in equals()`?** *Answer:* `instanceof` evaluates to true for subclasses, violating the symmetry requirement of the `equals()` contract if subclasses add new state fields.

**Why does calling `System.gc()` not guarantee garbage collection?** *Answer:* GC is a low-priority background thread managed by the JVM. `System.gc()` is only a hint/recommendation; the JVM determines when it is optimal to perform sweep phases.

**Why can't interfaces have instance variables?** *Answer:* Interfaces define behavior and contracts, not state. Supporting instance variables would bring state inheritance and duplicate field conflicts into multiple interface implementation.

**Why are records final by default?** *Answer:* Records are designed to be immutable data containers. Making them final prevents subclasses from adding mutable state or overriding accessors, preserving data integrity.

**Why can't you use `super` inside a static context?** *Answer:* `super` refers to the parent instance of the current object. Static context has no object instance, so there is no

`super` to refer to.

**Why are checked exceptions checked at compile-time while unchecked are checked at runtime?** *Answer:* Checked exceptions represent recoverable failures that the application should handle. Unchecked exceptions represent programming errors (bugs) that should be fixed.

**Why are static nested classes not called inner classes?** *Answer:* An inner class has a implicit link to an outer class instance. A static nested class is just a class defined inside another namespace, having no such link.

**Why does Java use late binding (dynamic binding) by default?** *Answer:* To support polymorphism and clean abstraction, allowing code to interact with abstract base classes or interfaces while invoking specific behavior at runtime.

**Why does default interface method collision require overriding?** *Answer:* If two implemented interfaces provide identical default methods, the compiler cannot automatically determine which implementation takes priority, requiring developer resolution.

**Why are covariant return types not supported for primitive types?** *Answer:* Primitives are values, not objects, and do not participate in inheritance hierarchies. There is no concept of a "subtype" relationship between `int` and `long`.

**Why is the Diamond Problem bad?** *Answer:* Because if two parents implement a method differently, a child inheriting both has no logical way to decide which implementation to execute, creating structural ambiguity.

---

## H. Design Questions (20 Questions)

**How do you design a system where only one instance of a class exists across the application?** *Answer:* Apply the Singleton Pattern: Make the constructor private, create a private static reference of the class, and provide a public static synchronized/double-checked method to return the instance.

**How do you design a database adapter system supporting MySQL, Oracle, and MongoDB?** *Answer:* Define a `DatabaseAdapter` interface with CRUD methods. Create class implementations for `MySQLAdapter`, `OracleAdapter`, and `MongoAdapter`. Inject the interface into client code.

**How do you design a Pizza ordering class that handles 20 different optional toppings without telescoping constructors?** *Answer:* Use the Builder Pattern. Create

an inner `Builder` class containing the same toppings. Chain topping methods on the builder, then call `.build()` to return the final immutable `Pizza`.

**How do you design a document conversion tool (PDF, HTML, DOCX) that can be extended easily?** *Answer:* Use the Strategy Pattern. Create a `ConverterStrategy` interface. Create `PDFConverter`, `HTMLConverter`, etc. The context class `DocumentProcessor` uses the strategy.

**How do you design a logging system that alerts developers via Email, SMS, and Slack simultaneously when a critical error occurs?** *Answer:* Use the Observer Pattern. The logger is the Subject. The notification channels are Observers. The logger notifies all registered observers when a critical log is processed.

**How do you design an API where subclassing is controlled and only specific packages can inherit your base class?** *Answer:* Use Sealed Classes (Java 17+) with the `permits` keyword to list only the allowed subclasses. Keep them in the same package.

**How do you design a payment system with transaction logging that adheres to Open/Closed Principle?** *Answer:* Define a `PaymentMethod` interface. Create separate classes for credit card, PayPal, UPI. Write a `PaymentService` that uses `PaymentMethod` through dependency injection.

**How do you design a class that acts as a pure data-holding object, ensuring it cannot be mutated or inherited?** *Answer:* Use a Java `record` (Java 16+) or declare a class `final` with `private final` fields, no setters, and a constructor initializing all fields.

**How do you design an undo/redo stack for a text editor?** *Answer:* Use the Command Pattern. Each action (type, delete, format) is a command class implementing a `Command` interface with `execute()` and `undo()` methods.

**How do you design a system where objects are created without exposing the instantiation logic to the client?** *Answer:* Use the Factory Method Pattern. The client requests an object from a `Factory` class by passing a string or enum, and the factory returns a concrete class instance through an interface reference.

**How do you design an immutable class containing a mutable `java.util.Date` field?** *Answer:* Clone the `Date` object in the constructor when assigning it to the field, and clone it again in the getter before returning it (defensive copying).

**How do you design a Cache system using SOLID principles?** *Answer:* Create a `Cache` interface with `get`, `put`, `remove`. High-level business services interact with the



interface. Implementation details (e.g., `RedisCache`, `InMemoryCache`) are hidden.

**How do you design a parking lot system that handles different vehicle sizes and spots?** *Answer:* Use an inheritance hierarchy for `Vehicle` (`Car`, `Motorbike`, `Truck`) and a `Spot` manager class containing `ParkingSpot` instances of different sizes. Use composition to link vehicles to spots.

**How do you design a File Compression system where users can zip/unzip files using different algorithms (Zip, Tar, Gzip)?** *Answer:* Use the Strategy Pattern. Declare a `CompressionStrategy` interface. Implement `ZipCompression`, `TarCompression`. The file manager uses the configured strategy.

**How do you design an employee payroll system that treats full-time, contract, and intern employees differently?** *Answer:* Create an abstract `Employee` class with an abstract `calculateSalary()` method. Each subclass overrides this method with its own payment structure.

**How do you design a configuration loader class that loads settings from a file only when they are first needed?** *Answer:* Use Lazy Initialization. Keep the config reference null initially. Instantiate the loader and read the file inside a thread-safe method block when it is first queried.

**How do you design an E-commerce system where customer checkout processes vary depending on user tier (Regular, Premium, VIP)?** *Answer:* Use Strategy Pattern. Introduce a `CheckoutStrategy` interface. Inject the appropriate strategy depending on the user tier during checkout.

**How do you design a custom class to be used safely as a key in a HashMap?** *Answer:* Make the class immutable (private final fields, no setters). Override `equals()` and `hashCode()` correctly using all fields, ensuring the hashcode never changes after creation.

**How do you design a system that dynamically decorates an object with additional responsibilities at runtime?** *Answer:* Use the Decorator Pattern (e.g., wrapping a plain `Coffee` with a `MilkDecorator` and a `SugarDecorator`).

**How do you design a media player that can play various audio formats, adapting to new codecs smoothly?** *Answer:* Use the Adapter Pattern or Strategy Pattern. Create a `MediaAdapter` class that implements the standard media player interface but delegates to specific decoding libraries.

---

## I. Scenario-Based Questions (20 Questions)

**Scenario:** Your class has a `List<String> tags` field. A junior developer writes a getter returning `this.tags`. Explain what can go wrong. *Answer:* The caller gets a direct reference to the internal list. They can call `.add()` or `.clear()` on it, violating encapsulation and modifying the class's internal state. Fix by returning `Collections.unmodifiableList(tags)` or a new copy.

**Scenario:** A database connection class is instantiated 100 times in a high-traffic app, crashing the DB connection pool. How do you resolve this? *Answer:* Apply the Singleton Pattern to the DB connection manager class, or use a dependency injection framework to manage it as a single instance.

**Scenario:** You are designing a square and rectangle system. You extend `Rectangle` to create `Square`. A test code calls `rect.setWidth(10); rect.setHeight(5);` and asserts the area is 50. Why does this test fail for `Square`? *Answer:* In `Square`, changing the width also changes the height to maintain square constraints. Substituting a `Square` for a `Rectangle` violates the Liskov Substitution Principle (LSP).

**Scenario:** You add a new method to an interface implemented by 100 classes. All classes now fail to compile. How do you fix this in Java 8+ without editing the 100 classes? *Answer:* Declare the new method as a `default` method in the interface and provide a default implementation body.

**Scenario:** You have a `HashMap<User, Profile>`. You modify a `User` object's `username` field (which is used in `hashCode()`). Now `map.get(user)` returns null. What happened? *Answer:* Mutating the username changed the hashCode of the `User` object. The map searches in a different bucket and cannot find the object, leaving the entry orphaned.

**Scenario:** A multi-threaded app creates multiple instances of a synchronized Singleton class, causing deadlocks. How do you rewrite the Singleton to be thread-safe without explicit synchronization blocks? *Answer:* Implement the Singleton as an enum or use Eager Initialization, where the JVM handles thread safety during class loading.

**Scenario:** You need to implement a deep clone of a complex structure containing cycles. A simple recursive clone causes a `StackOverflowError`. What is a robust workaround? *Answer:* Serialize the object graph into an `ObjectOutputStream` and deserialize it. The serialization stream handles cycles automatically by keeping track of object IDs.

**Scenario:** An API returns an abstract class `Report`. You need to call a subclass method `generateGraph()` which is only present in `PDFReport`. How do you

**safely write this?** *Answer:* Check type using `instanceof` and downcast safely: `if (report instanceof PDFReport pdf) { pdf.generateGraph(); }`.

**Scenario: A class `Engine` is inherited by `Car`. Later, you realize a car can change engines at runtime. How do you refactor this system?** *Answer:* Change inheritance (`extends Engine`) to composition. Give `Car` an `Engine` field and a setter `setEngine(Engine e)`.

**Scenario: You want to block all subclasses of `PaymentProcessor` from overriding the `authorize()` method. How do you configure it?** *Answer:* Mark the `authorize()` method with the `final` keyword in the base class.

**Scenario: An interface `Vehicle` has 20 methods. A `Bicycle` class implements it but leaves 15 methods empty. Which SOLID principle is violated? How do you fix it?** *Answer:* Interface Segregation Principle (ISP). Split the interface into smaller interfaces like `Motorized`, `PedalPowered`, and `Steerable`.

**Scenario: A constructor throws an exception before the object is fully initialized. What is the garbage collection status of this partially created object?** *Answer:* The object on the heap is immediately eligible for garbage collection because no live reference was ever returned and assigned to any variable.

**Scenario: You want to log all method calls of a service class without modifying the service class itself. How do you achieve this?** *Answer:* Use the Decorator Pattern or dynamic proxies (Aspect-Oriented Programming) to wrap the service class and log calls before delegating to the actual class.

**Scenario: You compare two Integer objects `a = 200` and `b = 200` using `a == b` and it returns false. You change their values to 100, and `a == b` returns true. Explain this behavior.** *Answer:* This is due to the Integer Cache. Integers between -128 and 127 are cached. 100 refers to the same cached object. 200 causes new object allocations on the heap, resulting in different references.

**Scenario: You implement a `ClassLoader` to load plugins dynamically. A plugin throws a `ClassCastException` when cast to your app's `PluginInterface`, even though it implements it. Why?** *Answer:* Two classes are considered identical by the JVM only if they have the same class name AND are loaded by the exact same `ClassLoader`. They were loaded by different loaders.

**Scenario: A method receives `final Student s`. A developer writes `s.setAge(20)`. The compiler does not complain. Why?** *Answer:* The `final` keyword prevents reassigning the reference variable `s` to a new `Student` object. It does not prevent modifying the internal state of the object `s` points to.

**Scenario: You need to create 10,000 game unit objects (like bullets). Creating each individually takes too much memory. How do you optimize this using design patterns?** *Answer:* Use the Flyweight Pattern. Share the intrinsic state (bullet sprite, velocity, damage) across all bullet instances, and store only the extrinsic state (coordinates) individually.

**Scenario: A method signature is `void print(Object o)` and `void print(String s)`. You call `print(null)`. Which method is called and why?**

*Answer:* `print(String s)` is called because the compiler always resolves overloaded calls to the most specific type matching the arguments.

**Scenario: A class needs to parse raw logs. Depending on the line prefix, it uses a different parser. How do you design this cleanly without large if-else blocks?**

*Answer:* Use a Map of prefix keys to Parser strategies. Query the map to get the strategy and execute it.

**Scenario: You want to prevent any external class from inheriting your class, but you still want your class to be instantiated. How do you achieve this?** *Answer:* Declare the class with the `final` keyword.

---

## D. Output-Based Questions (50 Questions)

### Question 1

```
class Parent {
    int x = 10;
    void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    int x = 20;
    void show() { System.out.println("Child"); }
}
public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        System.out.println(p.x);
        p.show();
    }
}
```

```
}
```

**Predict:**

Output **Answer:**

```
10  
Child
```

**Explanation:**

`p` is a reference of type `Parent` pointing to an object of type `Child`. In Java, fields are resolved based on the reference type (compile-time binding), while overridden methods are resolved based on the actual object type (runtime binding). Therefore, `p.x` gives `10` and `p.show()` executes the child's version.

**Concept Tested:** Reference type vs object type, Field Hiding vs Method Overriding.

**Interview Trap:** Assuming fields are polymorphic and that `p.x` would output `20`.

---

## Question 2

```
class A {  
    static void m() { System.out.println("A"); }  
}  
class B extends A {  
    static void m() { System.out.println("B"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.m();  
    }  
}
```

**Predict:**

Output **Answer:**

```
A
```

**Explanation:**

`m()` is a static method. In Java, static methods are resolved based on the reference type at compile time (static binding). Since the reference type of `obj` is `A`, `A.m()` is executed. This is method hiding, not overriding.

**Concept Tested:** Static method hiding.

**Interview Trap:** Thinking that static methods override each other and dispatch at runtime like instance methods.

---

**Question 3**

```
class Test {  
    void show(int x) { System.out.println("int"); }  
    void show(Integer x) { System.out.println("Integer"); }  
    void show(long x) { System.out.println("long"); }  
    void show(int... x) { System.out.println("varargs"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        new Test().show(5);  
    }  
}
```

**Predict:**

Output **Answer:**

```
int
```

**Explanation:**

The compiler searches for the closest match during overload resolution. `5` is an `int` literal, so the exact match `show(int)` is chosen.

**Concept Tested:** Overload resolution priority.

**Interview Trap:** Thinking widening, boxing, or varargs would override an exact match.

---

**Question 4**

```
class Test {
    void show(Integer x) { System.out.println("Integer"); }
    void show(long x) { System.out.println("long"); }
    void show(int... x) { System.out.println("varargs"); }
}

public class Main {
    public static void main(String[] args) {
        new Test().show(5);
    }
}
```

**Predict:**

Output **Answer:**

long

**Explanation:**

Since there is no exact `show(int)` match, the compiler looks for widening. `int` can be widened to `long` (primitive widening). Widening takes priority over boxing (`Integer`) and `varargs`.

**Concept Tested:** Overload priority: Widening vs Boxing.

**Interview Trap:** Expecting autoboxing to take precedence over widening.

---

## Question 5

```
class Test {
    void show(Integer x) { System.out.println("Integer"); }
    void show(int... x) { System.out.println("varargs"); }
}

public class Main {
    public static void main(String[] args) {
        new Test().show(5);
    }
}
```

**Predict:**

Output **Answer:**

Integer

**Explanation:**

Without an exact match or a widening option, the compiler falls back to autoboxing. `int` is boxed to `Integer`. Varargs has the lowest priority and is not chosen.

**Concept Tested:** Overload priority: Boxing vs Varargs.

**Interview Trap:** Believing varargs is preferred over boxing.

---

## Question 6

```
class Test {  
    void show(Long x) { System.out.println("Long"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        // new Test().show(5);  
    }  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Java does not support combined widening and boxing in a single step for method arguments. To match `show(Long)`, the `int` would need to be widened to `long` and then boxed to `Long`, which is illegal.

**Concept Tested:** Limits of overload resolution.

**Interview Trap:** Assuming `int` automatically promotes to `Long`.

---

## Question 7

```
class Test {  
    void show(Object x) { System.out.println("Object"); }  
}
```



```
}  
public class Main {  
    public static void main(String[] args) {  
        new Test().show(5);  
    }  
}
```

**Predict:**

Output **Answer:**

Object

**Explanation:**

The `int` value 5 is boxed to `Integer` (autoboxing), and then `Integer` is cast to `Object` via standard upcasting (widening reference type). Boxing followed by upcasting reference is permitted.

**Concept Tested:** Boxing then widening reference type.

**Interview Trap:** Thinking boxing cannot lead to reference upcasting.

---

## Question 8

```
class Test {  
    void show(String s) { System.out.println("String"); }  
    void show(Object o) { System.out.println("Object"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        new Test().show(null);  
    }  
}
```

**Predict:**

Output **Answer:**

String

**Explanation:**

Both `String` and `Object` can accept `null`. The compiler selects the most specific type in the hierarchy. Since `String` is a subclass of `Object`, `String` is more specific and is selected.

**Concept Tested:** Overload resolution with null and specific types.

**Interview Trap:** Assuming `null` matches `Object` first.

---

**Question 9**

```
class Test {
    void show(String s) { System.out.println("String"); }
    void show(Integer i) { System.out.println("Integer"); }
}
public class Main {
    public static void main(String[] args) {
        // new Test().show(null);
    }
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Both `String` and `Integer` can accept `null`. Since neither is a subclass of the other, they are at the same level of specificity. The compiler cannot choose between them, resulting in an ambiguity error.

**Concept Tested:** Ambiguous method overload resolution.

**Interview Trap:** Expecting `String` to take priority over `Integer` arbitrarily.

---

**Question 10**

```
class A {
    A() { System.out.print("A "); }
}
class B extends A {
    B() { System.out.print("B "); }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        new B();  
    }  
}
```

**Predict:**

Output **Answer:**

A B

**Explanation:**

When `new B()` is executed, B's constructor is called. The first line of B's constructor implicitly invokes `super()`, calling A's constructor. A's constructor runs first and prints "A ", then B's constructor prints "B ".

**Concept Tested:** Constructor chaining, implicit `super()`.

**Interview Trap:** Expecting only "B " to be printed.

---

## Question 11

```
class A {  
    A(int x) { System.out.print("A" + x + " "); }  
}  
class B extends A {  
    B() {  
        super(5);  
        System.out.print("B ");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        new B();  
    }  
}
```

**Predict:**

Output **Answer:**

A5 B

**Explanation:**

B's constructor explicitly calls `super (5)`, resolving A's parameterized constructor. A's constructor executes and prints "A5 ", then control returns to B's constructor which prints "B".

**Concept Tested:** Explicit parent constructor invocation.

**Interview Trap:** Forgetting that explicit `super ( )` suppresses implicit `super ( )`.

---

## Question 12

```
class A {
    static { System.out.print("A_static "); }
    { System.out.print("A_instance "); }
    A() { System.out.print("A_cons "); }
}
class B extends A {
    static { System.out.print("B_static "); }
    { System.out.print("B_instance "); }
    B() { System.out.print("B_cons "); }
}
public class Main {
    public static void main(String[] args) {
        new B();
    }
}
```

**Predict:**

Output **Answer:**

A\_static B\_static A\_instance A\_cons B\_instance B\_cons

**Explanation:**

First, class B is loaded, which loads parent class A. Static blocks run in parent-then-child order. Next, instance initializers and constructors run for A, then for B.

**Concept Tested:** Static vs Instance initialization order with inheritance.

**Interview Trap:** Mixing up static and instance execution sequences.

---

**Question 13**

```
class Test {
    static Test t = new Test();
    static { System.out.print("static "); }
    { System.out.print("instance "); }
    Test() { System.out.print("constructor "); }
}
public class Main {
    public static void main(String[] args) {
        // Just triggering class loading
    }
}
```

**Predict:**

Output **Answer:**

```
instance constructor static
```

**Explanation:**

Class loading executes static members in order of textual appearance. `static Test t = new Test();` is processed first. Instantiating `Test` triggers the instance block and constructor *before* the static block textually below it executes.

**Concept Tested:** Instantiation within static variable initialization.

**Interview Trap:** Assuming static blocks always print before instance blocks under all conditions.

---

**Question 14**

```
public class Main {
    static void change(int[] arr) {
        arr[0] = 50;
        arr = new int[]{100, 200};
    }
    public static void main(String[] args) {
        int[] nums = {1, 2, 3};
        change(nums);
        System.out.println(nums[0]);
    }
}
```

**Predict:**

Output **Answer:**

50

**Explanation:**

Java is pass-by-value. The value passed is a copy of the reference pointing to the array on the heap. `arr[0] = 50` modifies the shared array on the heap. Reassigning `arr` within `change` redirects the local parameter pointer, leaving the caller's reference `nums` untouched.

**Concept Tested:** Pass-by-value, Object reference mutation vs reassignment.

**Interview Trap:** Believing array reassignments propagate back to the caller.

---

## Question 15

```
public class Main {
    static void swap(Integer a, Integer b) {
        Integer temp = a;
        a = b;
        b = temp;
    }
    public static void main(String[] args) {
        Integer x = 10, y = 20;
        swap(x, y);
    }
}
```

```
        System.out.println(x + " " + y);
    }
}
```

**Predict:**

Output **Answer:**

```
10 20
```

**Explanation:**

The `swap` method swaps local copy references of variables `a` and `b`. The original reference variables `x` and `y` in the `main` stack frame remain unchanged.

**Concept Tested:** Pass-by-value with wrapper objects.

**Interview Trap:** Expecting wrapper objects to swap due to autoboxing.

---

## Question 16

```
public class Main {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = "Java";
        String s3 = new String("Java");
        String s4 = s3.intern();

        System.out.println(s1 == s2);
        System.out.println(s1 == s3);
        System.out.println(s1 == s4);
    }
}
```

**Predict:**

Output **Answer:**

```
true
false
```

```
true
```

**Explanation:**

`s1` and `s2` refer to the same literal inside the String Pool (`true`). `s3` is a separate heap object (`false`). `s3.intern()` returns the matching literal address from the String Pool, which equals `s1` (`true`).

**Concept Tested:** String Pool, `intern()` behavior, and `==` reference comparison.

**Interview Trap:** Assuming `new String()` references literals from the Pool directly.

---

**Question 17**

```
public class Main {  
    public static void main(String[] args) {  
        Integer a = 127;  
        Integer b = 127;  
        Integer c = 128;  
        Integer d = 128;  
        System.out.println(a == b);  
        System.out.println(c == d);  
    }  
}
```

**Predict:**

Output **Answer:**

```
true  
false
```

**Explanation:**

Java caches `Integer` objects for values from `-128` to `127`. `a` and `b` point to the same cached instance (`true`). `128` is outside this cache range, so `c` and `d` point to different heap objects (`false`).

**Concept Tested:** Integer cache.

**Interview Trap:** Expecting `==` to act as value comparison for all Wrapper objects.

---



## Question 18

```
class Parent {
    Parent() {
        show();
    }
    void show() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    int x = 42;
    void show() {
        System.out.println("Child: " + x);
    }
}
public class Main {
    public static void main(String[] args) {
        new Child();
    }
}
```

**Predict:**

Output **Answer:**

```
Child: 0
```

**Explanation:**

The constructor chain is: `new Child()` -> `Child()` -> `Parent()`. In `Parent()`, `show()` is executed. Due to runtime polymorphism, `Child.show()` is dispatched. However, `Child`'s instance variable `x` is initialized to `42` *after* the parent constructor returns. At this point, it holds its default value `0`.

**Concept Tested:** Subclass method dispatch during parent initialization.

**Interview Trap:** Expecting `x` to output `42`.

---

## Question 19

```
class A {  
    final void show() { System.out.println("A"); }  
}  
class B extends A {  
    // void show() { System.out.println("B"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        new B().show();  
    }  
}
```

**Predict:**

Output **Answer:**

A

**Explanation:**

The `final` method `show()` is inherited by B but cannot be overridden. Calling `new B().show()` safely executes the inherited parent method.

**Concept Tested:** Final method inheritance vs overriding.

**Interview Trap:** Thinking `final` prevents inheritance entirely.

---

## Question 20

```
interface A {  
    default void show() { System.out.println("A"); }  
}  
interface B {  
    default void show() { System.out.println("B"); }  
}  
class C implements A, B {  
    public void show() {  
        B.super.show();  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        new C().show();  
    }  
}
```

**Predict:**

Output **Answer:**

B

**Explanation:**

C resolves the default method conflict by overriding `show()` and explicitly delegating to B's version using `B.super.show()`.

**Concept Tested:** Default method conflict resolution.

**Interview Trap:** Believing default conflicts are resolved automatically.

---

## Question 21

```
interface A {  
    default void show() { System.out.println("A"); }  
}  
class B {  
    public void show() { System.out.println("B"); }  
}  
class C extends B implements A {}  
public class Main {  
    public static void main(String[] args) {  
        new C().show();  
    }  
}
```

**Predict:**

Output **Answer:**

B

**Explanation:**

Under Java's default method resolution rules, class methods take priority over interface default methods ("Class wins rule"). Thus, B's concrete method is chosen.

**Concept Tested:** Class method priority over default methods.

**Interview Trap:** Thinking a compiler conflict occurs between classes and interfaces.

---

**Question 22**

```
class A {  
    private void show() { System.out.println("A"); }  
    void test() { show(); }  
}  
class B extends A {  
    void show() { System.out.println("B"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        new B().test();  
    }  
}
```

**Predict:**

Output **Answer:**

A

**Explanation:**

`show()` in class A is private. Private methods are not visible to subclasses and are bound statically at compile time. Therefore, calling `test()` (which resides in A) calls A's private `show()`, not B's method.

**Concept Tested:** Private method static binding.

**Interview Trap:** Assuming B's `show()` overrides A's private `show()`.

---

### Question 23

```
class A {  
    int x = 10;  
}  
class B extends A {  
    int x = 20;  
}  
public class Main {  
    public static void main(String[] args) {  
        A a = new B();  
        System.out.println(((B)a).x);  
    }  
}
```

**Predict:**

Output **Answer:**

20

**Explanation:**

Fields are not polymorphic and are resolved based on the reference type. By explicitly casting reference `a` to type `B`, the compiler accesses the field `x` belonging to class `B`.

**Concept Tested:** Explicit casting and field resolution.

**Interview Trap:** Expecting casting to fail or throw `ClassCastException`.

---

### Question 24

```
class A {  
    void print() { System.out.println("A"); }  
}  
class B extends A {  
    void print() { System.out.println("B"); }  
}  
public class Main {  
    public static void main(String[] args) {
```

```
A obj = new B();
((A)obj).print();
}
}
```

**Predict:**

Output **Answer:**

B

**Explanation:**

`print()` is an instance method. Casting the reference back to `A` only changes the compile-time reference type. Dynamic dispatch still looks at the actual runtime object, which is `B`, executing `B`'s version.

**Concept Tested:** Casting reference vs dynamic method dispatch.

**Interview Trap:** Expecting casting to bypass method overriding.

---

## Question 25

```
class A {
    static int x = 10;
}
class B extends A {
    static int x = 20;
}
public class Main {
    public static void main(String[] args) {
        A obj = new B();
        System.out.println(obj.x);
    }
}
```

**Predict:**

Output **Answer:**

**Explanation:**

Static fields are resolved at compile time based on the reference type. Since `obj` has compile-time type `A`, `A.x` is accessed.

**Concept Tested:** Static variables and reference types.

**Interview Trap:** Expecting static variables to dynamically dispatch.

---

**Question 26**

```
class A {  
    A(int x) { System.out.print("A(" + x + ") "); }  
}  
class B extends A {  
    B() {  
        this(10);  
        System.out.print("B() ");  
    }  
    B(int x) {  
        super(x);  
        System.out.print("B(" + x + ") ");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        new B();  
    }  
}
```

**Predict:**

Output **Answer:**

```
A(10) B(10) B()
```

**Explanation:**

`new B()` calls the no-arg constructor `B()`. The first statement is `this(10)`, calling the

parameterized child constructor `B(int)`. The first statement of `B(int)` is `super(x)`, which executes A's parameterized constructor. Thus, A's constructor runs first ("`A(10)` "), followed by `B(10)` ("`B(10)` "), and finally control returns to `B()` ("`B()` ").

**Concept Tested:** Constructor delegation with `this()` and `super()`.

**Interview Trap:** Believing `this()` and `super()` can be executed in the same constructor body.

---

## Question 27

```
class Base {
    void show() { System.out.println("Base"); }
}
class Derived extends Base {
    @Override
    public void show() { System.out.println("Derived"); }
}
public class Main {
    public static void main(String[] args) {
        Base b = new Derived();
        b.show();
    }
}
```

**Predict:**

Output **Answer:**

```
Derived
```

**Explanation:**

Overriding methods can widen the access modifier. Class `Base` declares `show()` with default access, and subclass `Derived` overrides it with wider `public` access. This is valid, and runtime binding executes the subclass version.

**Concept Tested:** Access modifier widening in overriding.

**Interview Trap:** Expecting a compilation error because of different access modifiers.

---

## Question 28



```
class Base {  
    public void show() { System.out.println("Base"); }  
}  
class Derived extends Base {  
    // void show() { System.out.println("Derived"); }  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

When overriding a method, the subclass cannot narrow the access modifier. `show()` is public in `Base`. Overriding it with default (package-private) access in `Derived` is illegal because default is narrower than public.

**Concept Tested:** Access modifier narrowing restriction in inheritance.

**Interview Trap:** Thinking a subclass can restrict parent method visibility.

---

## Question 29

```
class A {  
    A create() { return new A(); }  
}  
class B extends A {  
    @Override  
    B create() { return new B(); }  
}  
public class Main {  
    public static void main(String[] args) {  
        A obj = new B();  
        System.out.println(obj.create().getClass().getSimpleName());  
    }  
}
```

**Predict:**

Output **Answer:**

B

**Explanation:**

Covariant return types are supported in Java. B overrides `create()` and changes the return type to B (which is a subclass of the parent return type A). Dynamic method dispatch executes `B.create()`, returning an instance of B.

**Concept Tested:** Covariant return types.

**Interview Trap:** Forgetting that primitive return types cannot be covariant.

---

**Question 30**

```
class A {
    static int x = 100;
}
public class Main {
    public static void main(String[] args) {
        A obj1 = new A();
        A obj2 = new A();
        obj1.x = 200;
        System.out.println(obj2.x);
    }
}
```

**Predict:**

Output **Answer:**

200

**Explanation:**

x is a static variable, meaning it belongs to class A and is shared among all instances. Accessing and writing to `obj1.x` updates the single shared location, so accessing `obj2.x` returns the updated value.

**Concept Tested:** Shared nature of static fields.

**Interview Trap:** Thinking static values remain distinct between instances.

---

**Question 31**

```
public class Main {  
    public static void main(String[] args) {  
        String s = "Hello";  
        s.replace('H', 'W');  
        System.out.println(s);  
    }  
}
```

**Predict:**

Output **Answer:**

Hello

**Explanation:**

Strings are immutable. The `replace()` method executes, but it returns a *new* String containing "Wello" and leaves the original variable `s` unchanged.

**Concept Tested:** String immutability.

**Interview Trap:** Forgetting to assign the return value back to a variable: `s = s.replace(...)`.

---

### Question 32

```
class Test {  
    int x;  
    Test(int x) { x = x; }  
}  
public class Main {  
    public static void main(String[] args) {  
        Test t = new Test(5);  
        System.out.println(t.x);  
    }  
}
```

**Predict:**

Output **Answer:**

0

**Explanation:**

The constructor parameter `x` shadows the instance variable `x`. `x = x` assigns the parameter's value back to the parameter itself, leaving the instance variable `x` at its default value 0.

**Concept Tested:** Parameter shadowing.

**Interview Trap:** Expecting JVM to auto-map parameter names to fields without `this.x = x`.

---

**Question 33**

```
public class Main {  
    public static void main(String[] args) {  
        final int[] arr = {1, 2, 3};  
        arr[0] = 99;  
        System.out.println(arr[0]);  
    }  
}
```

**Predict:**

Output **Answer:**

99

**Explanation:**

The `final` keyword prevents reassigning the reference variable `arr` to a new array instance. It does not make the referenced object (the array content) immutable. Thus, modifying `arr[0]` is completely valid.

**Concept Tested:** Final reference variables vs object state modification.

**Interview Trap:** Believing `final` locks down array indices.

---

**Question 34**

```
class Test {
    private int x = 10;
    class Inner {
        void show() { System.out.println(x); }
    }
}
public class Main {
    public static void main(String[] args) {
        new Test().new Inner().show();
    }
}
```

**Predict:**

Output **Answer:**

10

**Explanation:**

An inner class (non-static nested class) has an implicit link to the outer class instance that created it, giving it direct access to all members of the outer class, including private ones.

**Concept Tested:** Inner class scope and private access.

**Interview Trap:** Thinking inner classes cannot access private fields of outer classes.

---

### Question 35

```
class Test {
    static int x = 10;
    static class Nested {
        void show() { System.out.println(x); }
    }
}
public class Main {
    public static void main(String[] args) {
        new Test.Nested().show();
    }
}
```

**Predict:**

Output **Answer:**

10

**Explanation:**

A static nested class does not require an outer class instance. It behaves like a top-level class nested for namespace grouping and can access any static variables of the outer class.

**Concept Tested:** Static nested class syntax and access rules.

**Interview Trap:** Forgetting that static nested classes cannot access outer class instance variables.

---

### Question 36

```
class A {  
    void show() { System.out.println("A"); }  
}  
class B extends A {  
    void show() { System.out.println("B"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.show();  
    }  
}
```

**Predict:**

Output **Answer:**

B

**Explanation:**

This is standard dynamic method dispatch. The compiler verifies `show()` exists in class A, and the JVM dispatches the call to B's overridden version at runtime based on the actual object.

**Concept Tested:** Dynamic method dispatch.

**Interview Trap:** Thinking reference type A determines the method body executed.

---

### Question 37

```
class A {  
    void show() { System.out.println("A"); }  
}  
class B extends A {  
    void print() { System.out.println("B"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        A obj = new B();  
        // obj.print();  
    }  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Although the object on the heap is of type B, the reference type is A. The compiler verifies calls based on the reference type. Since A does not declare a `print()` method, compilation fails.

**Concept Tested:** Compile-time check based on reference type.

**Interview Trap:** Thinking compiler checks heap objects instead of references.

---

### Question 38

```
class A {  
    int x = 10;  
    { x = 20; }  
    A() { x = 30; }  
}  
public class Main {  
    public static void main(String[] args) {
```

```
        System.out.println(new A().x);
    }
}
}
```

**Predict:**

Output **Answer:**

30

**Explanation:**

Instance initialization occurs in order: field declaration (`x = 10`) -> instance initializer block (`x = 20`) -> constructor body (`x = 30`). The final value of `x` is 30.

**Concept Tested:** Instance field initialization order.

**Interview Trap:** Believing initializers overwrite constructors.

---

### Question 39

```
public class Main {
    public static void main(String[] args) {
        System.out.println(null instanceof Object);
    }
}
```

**Predict:**

Output **Answer:**

false

**Explanation:**

The `instanceof` operator checks if a reference variable points to an instance of the target type. Because `null` is a special literal indicating no object reference, it is not an instance of anything.

**Concept Tested:** `instanceof` with null references.

**Interview Trap:** Assuming `null` evaluates to true for `Object` type.

---



## Question 40

```
class Parent {
    static void print() { System.out.println("Parent"); }
}
class Child extends Parent {
    static void print() { System.out.println("Child"); }
}
public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.print();
    }
}
```

**Predict:**

Output **Answer:**

Parent

**Explanation:**

Static methods are bound statically by the compiler based on the reference type, not the runtime object type. The reference type of `p` is `Parent`, executing `Parent.print()`.

**Concept Tested:** Static method hiding.

**Interview Trap:** Thinking static methods dynamically dispatch at runtime.

---

## Question 41

```
class A {
    final int x;
    A() { x = 10; }
}
public class Main {
    public static void main(String[] args) {
        System.out.println(new A().x);
    }
}
```

```
}
```

**Predict:**

Output **Answer:**

```
10
```

**Explanation:**

A blank final field is a final field that is not initialized at its declaration. It must be initialized in all constructors of the class. This is valid, and the value is successfully initialized to 10.

**Concept Tested:** Blank final field initialization.

**Interview Trap:** Believing final variables must be initialized immediately at declaration.

---

## Question 42

```
class Test {  
    static int x = 10;  
    int y = 20;  
}  
public class Main {  
    public static void main(String[] args) {  
        Test t1 = new Test();  
        Test t2 = new Test();  
        t1.x = 30;  
        t1.y = 40;  
        System.out.println(t2.x + " " + t2.y);  
    }  
}
```

**Predict:**

Output **Answer:**

```
30 20
```

**Explanation:**

`x` is static and shared, so updating it via `t1` changes it for `t2` (30). `y` is an instance variable, so modifying `t1.y` has no effect on `t2.y` (20).

**Concept Tested:** Static vs Instance variables.

**Interview Trap:** Thinking both variables behave the same way.

---

**Question 43**

```
public class Main {  
    public static void main(String[] args) {  
        String s1 = "ab";  
        String s2 = "cd";  
        String s3 = "abcd";  
        String s4 = s1 + s2;  
        System.out.println(s3 == s4);  
    }  
}
```

**Predict:**

Output **Answer:**

false

**Explanation:**

`s1` and `s2` are variables. Therefore, `s1 + s2` is resolved at runtime using a `StringBuilder` or similar dynamic allocation, generating a new `String` instance on the heap. Thus, `s3 == s4` returns `false`.

**Concept Tested:** String concatenation at runtime vs compile time.

**Interview Trap:** Assuming all `String` concatenations resolve to pool values.

---

**Question 44**

```
public class Main {  
    public static void main(String[] args) {  
        final String s1 = "ab";
```

```
    final String s2 = "cd";
    String s3 = "abcd";
    String s4 = s1 + s2;
    System.out.println(s3 == s4);
}
}
```

**Predict:**

Output **Answer:**

```
true
```

**Explanation:**

Because `s1` and `s2` are final compile-time constants initialized with string literals, the expression `s1 + s2` is optimized by compiler constant folding into `"abcd"` at compile time. It references the same pool instance.

**Concept Tested:** Constant folding of final String variables.

**Interview Trap:** Expecting final variables to behave identical to standard variables during concatenation.

---

## Question 45

```
class Test {
    public boolean equals(Test t) {
        return true;
    }
}
public class Main {
    public static void main(String[] args) {
        Object t1 = new Test();
        Object t2 = new Test();
        System.out.println(t1.equals(t2));
    }
}
```

**Predict:**

Output **Answer:**

false

**Explanation:**

The method signature of `equals` inside `Test` is `equals(Test t)`. This *overloads* `equals` rather than overriding it, which requires `equals(Object o)`. Since reference types are `Object`, `Object.equals(Object)` is called, evaluating reference equality (`==`), which is `false`.

**Concept Tested:** Overloading vs Overriding `equals()`.

**Interview Trap:** Assuming `equals(Test t)` overrides the base `Object.equals` method.

---

## Question 46

```
public class Main {  
    public static void main(String[] args) {  
        String s1 = new String("Java");  
        String s2 = new String("Java");  
        System.out.println(s1 == s2);  
    }  
}
```

**Predict:**

Output **Answer:**

false

**Explanation:**

The `new` keyword explicitly allocates separate memory instances on the heap. `s1` and `s2` point to different addresses, making `s1 == s2` `false`.

**Concept Tested:** Reference comparison of new `String` instances.

**Interview Trap:** Expecting the compiler to auto-optimize `new String()` references to the pool.

---

## Question 47

```
class Parent {
    Parent(int x) { System.out.print("Parent "); }
}
class Child extends Parent {
    // Child() { }
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

If a parent class declares a parameterized constructor and no default constructor, subclasses must explicitly invoke it using `super(args)`. The default constructor generated for `Child` calls `super()`, which doesn't exist in `Parent`.

**Concept Tested:** Constructor dependencies under class inheritance.

**Interview Trap:** Expecting the compiler to automatically write parameterized `super` calls.

---

## Question 48

```
class A {
    A() { System.out.println("A"); }
}
class B extends A {
    B() {
        super();
        System.out.println("B");
    }
}
public class Main {
    public static void main(String[] args) {
        new B();
    }
}
```

**Predict:**

Output **Answer:**

A  
B

**Explanation:**

B's constructor explicitly calls `super ( )`, which runs the no-arg constructor of A. Then control returns and B's constructor prints "B". This is valid.

**Concept Tested:** Explicit super constructor call.

**Interview Trap:** Believing `super ( )` can only be implicit.

---

**Question 49**

```
class A {  
    static int x = 10;  
}  
public class Main {  
    public static void main(String[] args) {  
        A obj = null;  
        System.out.println(obj.x);  
    }  
}
```

**Predict:**

Output **Answer:**

10

**Explanation:**

`x` is a static variable. The compiler accesses static fields using the reference type (`A.x`) and translates `obj.x` into `A.x`. Since no object instance is required to access static variables, no `NullPointerException` is thrown.

**Concept Tested:** Static fields on null references.

**Interview Trap:** Expecting a `NullPointerException`.

---

**Question 50**

```
class A {  
    static void show() { System.out.println("A"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        A obj = null;  
        obj.show();  
    }  
}
```

**Predict:**

Output **Answer:**

A

**Explanation:**

Similar to static fields, calling static methods does not require an object instance. The compiler maps the call to `A.show()` using the reference type at compile time. No `NullPointerException` occurs.

**Concept Tested:** Static method invocation on null references.

**Interview Trap:** Expecting a `NullPointerException`.

---

## E. Compile-Time Error Questions (20 Questions)

### Question 1

```
abstract class A {  
    abstract final void show();  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

An `abstract` method must be overridden in a subclass, while a `final` method cannot be overridden. Combining `abstract` and `final` is contradictory and rejected by the compiler.



**Concept Tested:** Legality of access modifier combinations.

**Interview Trap:** Assuming a method can be abstract and final to enforce single-level implementation.

---

## Question 2

```
abstract final class Test {}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

An `abstract` class cannot be instantiated and is meant to be inherited. A `final` class cannot be inherited. Combining `abstract` and `final` at the class level is contradictory.

**Concept Tested:** Class declaration modifiers.

**Interview Trap:** Believing this creates a static-only class type.

---

## Question 3

```
interface Test {  
    private int x = 10;  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

All interface variables are implicitly `public static final`. Marking an interface variable as `private` is a compilation error.

**Concept Tested:** Interface fields modifiers.

**Interview Trap:** Trying to create private constants inside interfaces.

---

## Question 4

```
class Test {  
    final int x;  
    void init() {
```

```
        x = 10;
    }
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

A final instance field must be initialized either at its declaration, in an instance initializer block, or in every constructor of the class. It cannot be initialized inside a standard instance method.

**Concept Tested:** Final variable initialization scope.

**Interview Trap:** Expecting setter-like methods to initialize blank final fields.

---

## Question 5

```
class A {
    A(int x) {}
}
class B extends A {}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Class B's default constructor implicitly invokes `super()`. However, class A only has a parameterized constructor `A(int x)`. Since A has no no-arg constructor, B's default constructor fails to compile.

**Concept Tested:** Implicit constructor inheritance rules.

**Interview Trap:** Forgetting to write an explicit constructor in the child class when the parent has no default constructor.

---

## Question 6

```
class Test {
    Test() {
        System.out.println("Cons");
    }
}
```

```
        this(10);
    }
    Test(int x) {}
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

A constructor call using `this()` or `super()` must be the very first statement in a constructor body. Placing print statements before `this(10)` triggers a compile error.

**Concept Tested:** Constructor call ordering constraints.

**Interview Trap:** Trying to run logging statements before calling the delegate constructor.

---

### Question 7

```
class Test {
    Test() { this(); }
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

The compiler detects recursive constructor invocation (circular reference) and rejects it immediately during compilation.

**Concept Tested:** Circular constructor references.

**Interview Trap:** Expecting this to compile but cause `StackOverflowError` at runtime.

---

### Question 8

```
class A {
    void show() {}
}
class B extends A {
    int show() { return 10; }
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

When overriding a method, the return type must match or be a covariant (subtype) of the parent method's return type. Changing the return type from `void` to `int` is illegal.

**Concept Tested:** Method signature rules in overriding.

**Interview Trap:** Thinking return types can be overloaded in subclass overrides.

---

## Question 9

```
class A {  
    protected void show() {}  
}  
class B extends A {  
    void show() {}  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

An overriding method in a subclass cannot narrow the access modifier of the overridden method. `show()` has protected access in A. Overriding it with default (package-private) access in B is illegal.

**Concept Tested:** Access modifier constraints.

**Interview Trap:** Believing subclass methods are completely independent.

---

## Question 10

```
class A {  
    void show() throws IOException {}  
}  
class B extends A {  
    void show() throws Exception {}  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

An overriding method cannot throw broader or new checked exceptions. `Exception` is a superclass of `IOException` (broader checked exception), which is illegal.

**Concept Tested:** Exception constraints in method overriding.

**Interview Trap:** Assuming subclasses can expand exception signatures freely.

---

## Question 11

```
class Test {  
    static int x = 10;  
    void method() {  
        static int y = 20;  
    }  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

The `static` modifier is not allowed for local variables declared inside a method. Static members must be declared at the class level.

**Concept Tested:** Local variable declarations.

**Interview Trap:** Attempting to define thread-shared variables inside method scopes.

---

## Question 12

```
class Test {  
    static void show() {  
        System.out.println(this.toString());  
    }  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

The `this` keyword refers to the current instance of the class. Because static methods belong to the class and have no instance associated with them, referencing `this` inside a static context is illegal.

**Concept Tested:** Static vs Instance scoping.

**Interview Trap:** Expecting `this` to refer to the class meta-object.

---

### Question 13

```
class A {}
class B extends A {}
class C extends B, A {}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Java does not support multiple inheritance of classes. Declaring `C extends B, A` is a compilation error.

**Concept Tested:** Multiple class inheritance restrictions.

**Interview Trap:** Forgetting that multiple interface implementation is the only way to achieve multiple inheritance in Java.

---

### Question 14

```
interface Test {
    void m();
}
class Client {
    public static void main(String[] args) {
        // Test t = new Test();
    }
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Interfaces cannot be instantiated directly using `new Test()`. An implementing class or

anonymous class implementation is required.

**Concept Tested:** Interface instantiation limits.

**Interview Trap:** Forgetting that interface references can be declared but not instantiated directly.

---

### Question 15

```
class A {  
    final int x;  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Instance final variables must be initialized. Because `x` is declared final and no constructor initializes it, compilation fails with "variable x might not have been initialized".

**Concept Tested:** Uninitialized final variables.

**Interview Trap:** Assuming the compiler automatically initializes final variables to default values like `0`.

---

### Question 16

```
class Test {  
    void print(String... s) {}  
    void print(String[] s) {}  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Varargs are compiled as arrays under the hood. Therefore, `print(String... s)` and `print(String[] s)` resolve to the exact same signature in bytecode, causing a duplicate method declaration error.

**Concept Tested:** Varargs implementation in bytecode.

**Interview Trap:** Believing varargs and arrays can overload each other.

---

## Question 17

```
class Parent {  
    Parent(int x) {}  
}  
class Child extends Parent {  
    Child() {  
        this(10);  
        super(20);  
    }  
    Child(int x) {  
        super(x);  
    }  
}
```

### Predict:

Compile-time error **Answer:**

Compile-time error **Explanation:**

A constructor can call either `this()` or `super()`, but not both. They must be the first statement in the constructor, which is mutually exclusive.

**Concept Tested:** Constructor chaining limits.

**Interview Trap:** Expecting both calls to execute in order.

---

## Question 18

```
class Test {  
    final void print() {}  
}  
class Client extends Test {  
    void print() {}  
}
```

### Predict:

Compile-time error **Answer:**

Compile-time error **Explanation:**

Subclasses cannot override final methods of the parent class. Declaring `void print()` in `Client` is illegal.



**Concept Tested:** Final method overriding limits.

**Interview Trap:** Expecting method hiding to work for final instance methods.

---

### Question 19

```
class A {  
    A(int x) {}  
}  
class B extends A {  
    B() {  
        // compiler inserts super();  
    }  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

Since class A does not declare a default constructor, the implicit `super()` call inside B's constructor fails. An explicit `super(x)` call is mandatory.

**Concept Tested:** Parent constructor dependencies.

**Interview Trap:** Forgetting that declaring a parameterized constructor blocks the auto-generation of no-arg constructors.

---

### Question 20

```
class Test {  
    static abstract void m();  
}
```

**Predict:**

Compile-time error **Answer:**

Compile-time error **Explanation:**

A static method belongs to the class and cannot be overridden, whereas an abstract method exists to be overridden. Combining `static` and `abstract` is contradictory.

**Concept Tested:** Static and Abstract modifiers compatibility.

**Interview Trap:** Expecting subclasses to override static abstract behaviors.

---

## F. Runtime Exception Questions (20 Questions)

### Question 1

```
public class Main {  
    public static void main(String[] args) {  
        Object obj = new Integer(100);  
        String s = (String) obj;  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.ClassCastException` **Explanation:**

`obj` points to an `Integer` object on the heap. Casting an `Integer` to a `String` violates inheritance constraints, resulting in a `ClassCastException` at runtime.

**Concept Tested:** Safe downcasting.

**Interview Trap:** Believing that because the reference type is `Object`, it can be cast to any type.

---

### Question 2

```
public class Main {  
    public static void main(String[] args) {  
        String s = null;  
        System.out.println(s.length());  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.NullPointerException` **Explanation:**

The reference variable `s` is initialized to `null`, meaning it points to no object. Attempting to call the instance method `length()` on a null reference throws a `NullPointerException`.

**Concept Tested:** Null references.

**Interview Trap:** Expecting `null` to return a length of 0 or print "`null`".

---

### Question 3

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3};  
        System.out.println(arr[3]);  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.ArrayIndexOutOfBoundsException` **Explanation:**

The array `arr` has a size of 3, meaning valid indices are 0, 1, and 2. Accessing index 3 is out of bounds, throwing an `ArrayIndexOutOfBoundsException`.

**Concept Tested:** Array boundary checks.

**Interview Trap:** Expecting default or null values for out-of-bound indices.

---

### Question 4

```
public class Main {  
    public static void main(String[] args) {  
        Object[] names = new String[3];  
        names[0] = new Integer(5);  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.ArrayStoreException` **Explanation:**

Arrays in Java are covariant, meaning `Object[]` can point to `String[]` at compile time. However, at runtime, the JVM checks the actual type. Trying to store an `Integer` in a `String` array throws an `ArrayStoreException`.

**Concept Tested:** Array covariance type check.

**Interview Trap:** Assuming compile-time validation prevents array mismatch errors.

---

## Question 5

```
public class Main {  
    public static void main(String[] args) {  
        String s = "abc";  
        int num = Integer.parseInt(s);  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.NumberFormatException` **Explanation:**

`Integer.parseInt` attempts to parse a string into an integer. Because "abc" is not a valid representation of a number, a `NumberFormatException` is thrown.

**Concept Tested:** Parsing wrappers.

**Interview Trap:** Expecting it to return 0 or fail at compile time.

---

## Question 6

```
public class Main {  
    public static void main(String[] args) {  
        int result = 10 / 0;  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.ArithmeticException: / by zero` **Explanation:**

Dividing an integer by zero is mathematically undefined and throws an `ArithmeticException` in Java.

**Concept Tested:** Integer division safety.

**Interview Trap:** Thinking it returns `Infinity` like floating-point division (`10.0 / 0`).

---

## Question 7

```

class Test implements Cloneable {
    // missing clone override
}
public class Main {
    public static void main(String[] args) throws Exception {
        Object o = new Object();
        // o.clone(); // triggers CloneNotSupportedException if overridden
    }
}

```

**Predict:**

Runtime exception (if attempted via reflection/clone method call) **Answer:**

`java.lang.CloneNotSupportedException` **Explanation:**

If `clone()` is called on a class that does not implement the `Cloneable` interface, a `CloneNotSupportedException` is thrown.

**Concept Tested:** Cloneable contract rules.

**Interview Trap:** Expecting `clone()` to copy objects without checking interface markers.

---

## Question 8

```

public class Main {
    public static void main(String[] args) {
        Boolean b = null;
        if (b) {
            System.out.println("True");
        }
    }
}

```

**Predict:**

Runtime exception **Answer:**

`java.lang.NullPointerException` **Explanation:**

Inside the `if` check, the JVM attempts to unbox `b` (wrapper type `Boolean`) to the primitive `boolean` type. Since `b` is `null`, this throws a `NullPointerException` during unboxing.

**Concept Tested:** Autoboxing/unboxing null values.

**Interview Trap:** Expecting `null` to evaluate to `false` automatically.

---

## Question 9

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = Collections.emptyList();
        list.add("Java");
    }
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.UnsupportedOperationException` **Explanation:**

`Collections.emptyList()` returns an immutable list instance. Any attempt to modify its content (like calling `add()`) throws an `UnsupportedOperationException` at runtime.

**Concept Tested:** Immutable collection behaviors.

**Interview Trap:** Assuming all lists behave as standard mutable `ArrayLists`.

---

## Question 10

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(Arrays.asList("A", "B"));
        for (String s : list) {
            if (s.equals("A")) list.remove(s);
        }
    }
}
```

**Predict:**

Runtime exception **Answer:**

`java.util.ConcurrentModificationException` **Explanation:**

Modifying a collection directly while iterating over it using an enhanced `for` loop (which translates to an implicit iterator) triggers a `ConcurrentModificationException`.

**Concept Tested:** Fail-fast iterators.

**Interview Trap:** Expecting lists to allow direct element removal during loops.

---

## Question 11

```
public class Main {  
    static void recursive() {  
        recursive();  
    }  
    public static void main(String[] args) {  
        recursive();  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.StackOverflowError` **Explanation:**

Infinite recursion fills up the call stack frames, exceeding the stack memory size allocated by the JVM, which throws a `StackOverflowError`.

**Concept Tested:** Recursion limits.

**Interview Trap:** Thinking it will compile and run forever or throw `OutOfMemoryError`.

---

## Question 12

```
import java.util.*;  
public class Main {  
    public static void main(String[] args) {  
        List<Integer> list = Arrays.asList(1, 2, 3);  
        list.add(4);  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.UnsupportedOperationException` **Explanation:**

`Arrays.asList` returns a fixed-size list backed by the original array. Elements cannot be added or removed, so calling `add()` throws an `UnsupportedOperationException`.

**Concept Tested:** Fixed-size list characteristics.

**Interview Trap:** Thinking `Arrays.asList` returns a fully functional `ArrayList`.

---

### Question 13

```
public class Main {  
    public static void main(String[] args) {  
        String s = "Hello";  
        s.charAt(10);  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.StringIndexOutOfBoundsException` **Explanation:**

The string "Hello" has indices from 0 to 4. Requesting character at index 10 is out of bounds, throwing a `StringIndexOutOfBoundsException`.

**Concept Tested:** String boundaries checks.

**Interview Trap:** Expecting null or space character return values.

---

### Question 14

```
class Test {  
    static {  
        int x = 10 / 0;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        try {  
            new Test();  
        } catch (Exception e) {  
            System.out.println("Caught");  
        }  
    }  
}
```

**Predict:**

Runtime exception **Answer:**



`java.lang.ExceptionInInitializerError` **Explanation:**

Any exception that occurs inside a static initializer block is wrapped by the JVM and thrown as an `ExceptionInInitializerError`. Since this error is a subclass of `Throwable/Error`, catching `Exception` will not catch it, causing the program to terminate.

**Concept Tested:** Static block exception wrapping and Error handling.

**Interview Trap:** Expecting `catch (Exception e)` to catch all initialisation errors.

---

## Question 15

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = new int[-5];  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.NegativeArraySizeException` **Explanation:**

Declaring an array with a negative size parameter is illegal and throws a `NegativeArraySizeException` at runtime.

**Concept Tested:** Array instantiation parameters.

**Interview Trap:** Expecting a compile-time check or a size of 0.

---

## Question 16

```
import java.util.*;  
public class Main {  
    public static void main(String[] args) {  
        Set<Object> set = new TreeSet<>();  
        set.add(new Object());  
    }  
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.ClassCastException` **Explanation:**

`TreeSet` stores elements in sorted order. When adding an element, it attempts to cast it to `Comparable` to perform sorting checks. Since `Object` does not implement `Comparable`, a `ClassCastException` is thrown.

**Concept Tested:** Sorting requirements in `TreeSet`/`TreeMap`.

**Interview Trap:** Believing all classes can be stored in `TreeSets` without implementing `Comparable`.

---

## Question 17

```
public class Main {
    public static void main(String[] args) {
        Integer val = Integer.valueOf("12.34");
    }
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.NumberFormatException` **Explanation:**

`Integer.valueOf` attempts to parse a string representation of an integer. Since `"12.34"` represents a floating-point number rather than an integer, a `NumberFormatException` is thrown.

**Concept Tested:** Type matching during string parsing.

**Interview Trap:** Expecting the decimal portion to be truncated to `12`.

---

## Question 18

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.get(0);
    }
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.IndexOutOfBoundsException` **Explanation:**

The list is newly instantiated and empty (size 0). Accessing index 0 throws an `IndexOutOfBoundsException`.

**Concept Tested:** Collection boundary checks.

**Interview Trap:** Thinking index 0 is safe even in empty lists.

---

## Question 19

```
public class Main {
    public static void main(String[] args) {
        String s = "null";
        if (s == null) {
            // ...
        }
        System.out.println(s.substring(5));
    }
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.StringIndexOutOfBoundsException` **Explanation:**

The string is the literal value "null", which has a length of 4. Requesting `substring(5)` exceeds boundaries, throwing a `StringIndexOutOfBoundsException`.

**Concept Tested:** Lit Strings vs Null values.

**Interview Trap:** Confusing the string "null" with actual null values.

---

## Question 20

```
public class Main {
    public static void main(String[] args) {
        Object x[] = new String[3];
        x[0] = new Object();
    }
}
```

**Predict:**

Runtime exception **Answer:**

`java.lang.ArrayStoreException` **Explanation:**

Similar to Question 4, because the underlying array is a `String` array, trying to store a plain `Object` throws an `ArrayStoreException` at runtime.

**Concept Tested:** Dynamic array type safety checks.

**Interview Trap:** Believing reference assignment overrides heap array types.

---

## Rules You Must Know by Heart

1. Java is always pass-by-value. There is no pass-by-reference.
2. For objects, Java passes a copy of the reference (address) by value.
3. Every class in Java implicitly inherits directly or indirectly from `java.lang.Object`.
4. Constructors are never inherited by subclasses.
5. Constructors cannot be overridden because their names are specific to their class.
6. A constructor cannot be declared as `static`, `final`, `synchronized`, `abstract`, or `native`.
7. `super()` or `this()` must always be the first statement inside a constructor body.
8. If a class writes no constructors, the compiler automatically inserts a default no-argument constructor calling `super()`.
9. The moment any custom constructor is written in a class, the compiler stops generating the default constructor.
10. Instance initializer blocks are executed after the superclass constructor returns and before the child constructor body runs.
11. Static blocks and static variables are initialized once when the class is first loaded into memory by the `ClassLoader`.
12. Static blocks and variables are executed in the textual order they appear inside the class definition.
13. Instance initializers and instance variables are executed in the textual order they appear inside the class definition.
14. A static method cannot use the `this` or `super` keywords.

15. A static method cannot directly access instance variables or call instance methods without creating an object reference.
16. Static methods are hidden, not overridden, if redeclared in a subclass with the same signature.
17. Private methods are not visible to subclasses, so they are statically bound and cannot be overridden.
18. Final methods cannot be overridden by subclasses.
19. Final classes cannot be subclassed (inherited).
20. Instance variables (fields) are not polymorphic. Field access is resolved at compile time based on the reference type.
21. Static variables are not polymorphic. Access is resolved at compile time based on the reference type.
22. The return type of an overriding method can be a subtype of the parent method's return type (covariant return type).
23. The access modifier of an overriding method must be equal to or wider (more accessible) than the parent method.
24. Access modifier hierarchy from narrowest to widest: `private` -> default (package-private) -> `protected` -> `public`.
25. An overriding method cannot throw new or broader checked exceptions than the parent method.
26. An overriding method has no restrictions on throwing unchecked (runtime) exceptions.
27. All fields declared inside an interface are implicitly `public static final`.
28. All abstract methods inside an interface are implicitly `public abstract`.
29. Interface static methods are not inherited by implementing classes. They must be called directly on the interface name.
30. Implementing classes do not inherit default methods if a subclass method overrides them or a superclass method defines them.
31. Class methods always win over interface default methods in case of duplicate signatures ("Class wins rule").

32. More specific interface default methods take priority over less specific ones in a sub-interface hierarchy.
33. Multiple interface implementation conflicts must be resolved explicitly by overriding the conflicting method in the class.
34. Abstract classes cannot be instantiated using the `new` keyword.
35. Interfaces cannot be instantiated using the `new` keyword (unless declaring an anonymous class).
36. If a class implements `equals()`, it must override `hashCode()` to maintain the consistency contract.
37. Equal objects according to `equals(Object)` must return the exact same integer hashcode from `hashCode()`.
38. Objects with identical hashcodes are not guaranteed to be equal (hash collision).
39. Unequal objects according to `equals(Object)` can have the same or different hashcodes.
40. The `==` operator compares values for primitives, and memory addresses (reference references) for objects.
41. Comparing wrapper objects using `==` uses the Integer Cache for values between -128 and 127.
42. `null instanceof Type` always evaluates to `false` for any reference type.
43. Downcasting a superclass reference to a subclass type requires an explicit cast and is checked at runtime.
44. Downcasting to an invalid subclass type throws a `ClassCastException` at runtime.
45. Upcasting a subclass reference to a superclass type is implicit and safe.
46. The `instanceof` operator evaluates to `true` if the heap object is an instance of the target type or its subclasses.
47. Arrays are objects, and array references are passed by value.
48. Arrays in Java are covariant (e.g., `String[]` is assignable to `Object[]`), which can trigger `ArrayStoreException` at runtime.
49. `java.lang.Record` is the implicit parent of all records, and records cannot extend any other classes.

50. Abstract methods cannot have a body and cannot be declared private, final, or static.

---

## Top 50 Java OOP Interview Traps

### Trap 1: Field Polymorphism

*Wrong assumption:* Child field overrides Parent field of the same name, and accessing it through a Parent reference returns the Child value.

*Correct rule:* Fields are not polymorphic. Access is resolved at compile time based on the reference type.

*Example:*

```
Parent p = new Child();  
System.out.println(p.x); // Prints Parent's x, not Child's x.
```

### Trap 2: Static Method Overriding

*Wrong assumption:* Declaring a static method in a subclass with the same signature overrides the parent static method.

*Correct rule:* Static methods cannot be overridden; they are hidden. The reference type determines which method runs.

*Example:*

```
Parent p = new Child();  
p.staticMethod(); // Runs Parent's static method.
```

### Trap 3: Calling Overridable Methods in Constructors

*Wrong assumption:* Subclass overrides called in parent constructors access fully initialized subclass fields.

*Correct rule:* Subclass overrides run before subclass fields have been initialized (they are still default values).

*Example:*

```
class P { P() { show(); } void show() {} }  
class C extends P { int x = 10; void show() { System.out.println(x); } }  
// Instantiating C prints 0, not 10.
```

#### Trap 4: Null Reference static access

*Wrong assumption:* Accessing a static field or calling a static method through a null reference throws a `NullPointerException`.

*Correct rule:* Static accesses bypass runtime instances and use the reference type at compile time.

*Example:*

```
Parent p = null;  
p.staticMethod(); // Compiles and executes without throwing NPE.
```

#### Trap 5: Integer Caching and ==

*Wrong assumption:* Comparing two Integer objects using `==` always compares their value.

*Correct rule:* `==` compares references. Caching applies only from -128 to 127.

*Example:*

```
Integer a = 200, b = 200;  
System.out.println(a == b); // Prints false.
```

#### Trap 6: Final reference reassignment vs mutation

*Wrong assumption:* Declaring an object reference as `final` makes the object immutable.

*Correct rule:* `final` only prevents reassigning the reference to point to a new object.

*Example:*

```
final List<String> list = new ArrayList<>();  
list.add("Java"); // Works fine.  
// list = new ArrayList<>(); // Compile error.
```

#### Trap 7: Private method overriding



*Wrong assumption:* Declaring a method with the same signature as a parent private method overrides it.

*Correct rule:* Private methods are invisible to subclasses; the child method is a completely new method.

*Example:*

```
class P { private void m() { System.out.println("P"); } void test() { m(); }  
class C extends P { public void m() { System.out.println("C"); } }  
new C().test(); // Prints "P".
```

### Trap 8: String Literal concatenation at runtime

*Wrong assumption:* Concatenating a String variable with a literal results in a String pool object reference.

*Correct rule:* Runtime variable concatenation uses StringBuilder and allocates on the heap outside the pool.

*Example:*

```
String s1 = "ab";  
String s2 = s1 + "cd";  
String s3 = "abcd";  
System.out.println(s2 == s3); // Prints false.
```

### Trap 9: Empty/No-arg constructor auto-generation

*Wrong assumption:* The compiler always provides a default no-argument constructor for every class.

*Correct rule:* The compiler only provides it if NO other constructors are explicitly written.

*Example:*

```
class P { P(int x) {} }  
// new P(); // Compile error.
```

### Trap 10: Downcasting compile success

*Wrong assumption:* Explicit casting always works if the compiler does not flag it as an error.

*Correct rule:* The compiler allows downcasting if there is a subclass relationship, but the JVM checks type on the heap.

*Example:*

```
Parent p = new Parent();  
Child c = (Child) p; // Throws ClassCastException at runtime.
```

### Trap 11: instanceof with null

*Wrong assumption:* Checking `null instanceof Object` returns true because null represents an object state.

*Correct rule:* instanceof with null always returns false.

*Example:*

```
System.out.println(null instanceof Object); // Prints false.
```

### Trap 12: Overriding vs Overloading parameter type

*Wrong assumption:* Implementing `equals(MyClass obj)` overrides the base Object method `equals(Object obj)`.

*Correct rule:* This is method overloading, not overriding. The base method remains unchanged.

*Example:*

```
class Test { public boolean equals(Test t) { return true; } }  
Object o1 = new Test(), o2 = new Test();  
System.out.println(o1.equals(o2)); // Prints false (uses reference equality)
```

### Trap 13: Covariant Primitive returns

*Wrong assumption:* Subclass methods can narrow primitive return types during overriding (e.g., returning `int` instead of `long`).

*Correct rule:* Covariance applies only to reference types. Primitives must match exactly.

*Example:*

```
class P { long get() { return 1L; } }  
class C extends P { int get() { return 1; } } // Compile error.
```

### Trap 14: checked exceptions in overriding

*Wrong assumption:* An overriding subclass method can declare to throw any checked exception.

*Correct rule:* Subclass methods cannot throw broader or new checked exceptions.

*Example:*

```
class P { void show() throws IOException {} }  
class C extends P { void show() throws Exception {} } // Compile error.
```

### Trap 15: unchecked exceptions in overriding

*Wrong assumption:* If a parent method does not declare a runtime exception, the child cannot declare one.

*Correct rule:* Child methods can throw any runtime exception without restrictions.

*Example:*

```
class P { void show() {} }  
class C extends P { void show() throws NullPointerException {} } // Compile error.
```

### Trap 16: Abstract class constructors

*Wrong assumption:* Abstract classes cannot have constructors because they cannot be instantiated.

*Correct rule:* Abstract classes have constructors to initialize fields, called via `super()` in subclasses.

*Example:*

```
abstract class A { A() { System.out.println("A"); } }  
class B extends A { B() { super(); } }
```

### Trap 17: Abstract Class vs Interface field defaults

*Wrong assumption:* Declaring variables in interfaces behaves identical to abstract class variables.

*Correct rule:* Interface fields are implicitly `public static final` constants. Abstract class fields can be instance variables.

*Example:*

```
interface I { int x = 10; } // implicitly public static final.
```

### Trap 18: Interface static method inheritance

*Wrong assumption:* Subclasses or sub-interfaces inherit static methods declared inside interfaces.

*Correct rule:* Interface static methods are not inherited; they must be called using the interface name.

*Example:*

```
interface I { static void show() {} }  
class C implements I {}  
// C.show(); // Compile error.
```

### Trap 19: Covariant arrays assignment

*Wrong assumption:* Storing any object inside an `Object[]` pointing to `String[]` is safe because of compiler validation.

*Correct rule:* Array covariance is checked at runtime, throwing `ArrayStoreException` on mismatch.

*Example:*

```
Object[] arr = new String[2];  
arr[0] = 10; // Throws ArrayStoreException.
```

### Trap 20: default method priority conflict

*Wrong assumption:* A class implementing two interfaces with identical default methods defaults to the first interface listed.

*Correct rule:* The compiler rejects this as ambiguous, forcing the developer to override the method.

*Example:*

```
interface A { default void m() {} }  
interface B { default void m() {} }  
class C implements A, B { public void m() { A.super.m(); } }
```

## Trap 21: Local class variable mutation

*Wrong assumption:* Local inner classes can modify local variables declared in the surrounding method scope.

*Correct rule:* Local variables accessed by local classes must be final or effectively final.

*Example:*

```
int x = 10;
class Local { void show() { /* System.out.println(x); */ } }
// x = 20; // If x is modified, Local cannot access it (no longer effectively final)
```

## Trap 22: Record mutability

*Wrong assumption:* You can modify record fields after construction using setter-like accessors.

*Correct rule:* Records are implicitly final and their fields are private and final, having no setter methods.

*Example:*

```
record R(int x) {}
R r = new R(10);
// r.x = 20; // Compile error.
```

## Trap 23: Abstract class with zero abstract methods

*Wrong assumption:* Abstract classes must declare at least one abstract method.

*Correct rule:* An abstract class can have zero abstract methods, used purely to prevent direct instantiation.

*Example:*

```
abstract class A {} // Valid.
```

## Trap 24: `this` and `super` constructor chain combination

*Wrong assumption:* A constructor can call `super()` to construct the parent, and then `this(args)` to delegate within.

*Correct rule:* Only one constructor invocation statement is allowed, and it must be the first line.

*Example:*

```
class C { C() { super(); this(5); } } // Compile error.
```

## Trap 25: Static blocks and instance instantiation

*Wrong assumption:* Static block execution is always completed before any instance block runs in the program.

*Correct rule:* If an instance is instantiated inside a static block or field initializer, instance blocks execute during class loading.

*Example:*

```
class A { static A obj = new A(); { System.out.print("Instance "); } }  
// Prints "Instance" before class loading completes.
```

---

## Trap 26: Floating Point division by zero

*Wrong assumption:* Dividing any number by zero in Java throws an `ArithmeticException`.

*Correct rule:* Floating point division by zero does not throw exceptions; it returns `Infinity` or `NaN`.

*Example:*

```
System.out.println(10.0 / 0); // Prints Infinity.
```

## Trap 27: Static nested class outer instance access

*Wrong assumption:* Static nested classes can access outer class instance fields directly.

*Correct rule:* Static nested classes have no implicit reference to an outer class instance and can only access static members directly.

*Example:*

```
class Outer { int x; static class Nested { void m() { /* x = 10; */ } } }
```

## Trap 28: Overriding checks during Class Loading

*Wrong assumption:* `ClassCastException` or overriding signature checks are completed when classes are loaded.

*Correct rule:* Class loading checks byte structure. `ClassCastException` is checked at runtime when cast operations execute.

*Example:*

```
Object o = new Object();  
String s = (String) o; // Throws ClassCastException at runtime, not class
```

### Trap 29: Covariant Return with primitive casting

*Wrong assumption:* Covariant return permits returning `float` when the parent method returns `double`.

*Correct rule:* Covariance does not apply to primitive types. Primitive types must match exactly.

*Example:*

```
class P { double get() { return 1.0; } }  
class C extends P { float get() { return 1.0f; } } // Compile error.
```

### Trap 30: Array covariance storage verification

*Wrong assumption:* Storing a subclass reference in a covariant array is validated at compile time.

*Correct rule:* Compilation permits reference storage, but runtime JVM checks heap type and throws `ArrayStoreException`.

*Example:*

```
Object[] objs = new String[2];  
objs[0] = new Object(); // Throws ArrayStoreException at runtime.
```

### Trap 31: Abstract Class initialization bypass

*Wrong assumption:* Because abstract classes cannot be instantiated, their constructors are never executed.

*Correct rule:* When subclass instances are created, parent abstract constructors run as part of the normal chain.

*Example:*

```
abstract class A { A() { System.out.println("A"); } }
class B extends A { B() { super(); } }
new B(); // Prints "A".
```

### Trap 32: interface variable inheritance conflict

*Wrong assumption:* If a class implements two interfaces containing identical variable names, they merge or conflict immediately.

*Correct rule:* Variables are accessible via interface names. A conflict only occurs if you refer to the variable name directly without qualification.

*Example:*

```
interface A { int X = 1; }
interface B { int X = 2; }
class C implements A, B { void show() { /* System.out.println(X); */ } }
```

### Trap 33: Overloading method priority matching null

*Wrong assumption:* Passing null to overloaded methods matching `show(Object o)` and `show(String s)` is ambiguous.

*Correct rule:* Java selects the most specific type in the matching hierarchy. `String` is more specific than `Object`.

*Example:*

```
class Test { void show(Object o) {} void show(String s) {} }
new Test().show(null); // Resolves to show(String).
```

### Trap 34: Final method hiding attempts

*Wrong assumption:* Subclasses can define static methods with the same signature to hide final static parent methods.

*Correct rule:* Static methods cannot be overridden, but `final` prevents method hiding as well.

*Example:*

```
class P { final static void m() {} }
class C extends P { static void m() {} } // Compile error.
```



### Trap 35: blank final variable default values

*Wrong assumption:* Blank final variables automatically get initialized with default values (0, null) if omitted in constructor.

*Correct rule:* The compiler forces explicit initialization of all final fields inside every constructor.

*Example:*

```
class Test { final int x; Test() {} } // Compile error: x might not be ini
```

### Trap 36: static context referencing instance members

*Wrong assumption:* Static methods can access instance variables of the outer class if they are nested inside it.

*Correct rule:* Static context has no `this` reference. It must explicitly instantiate or reference an object.

*Example:*

```
class Test { int x; static void m() { /* x = 10; */ } } // Compile error.
```

### Trap 37: Checked exceptions and runtime inheritance

*Wrong assumption:* An overriding subclass method can throw any new checked exceptions.

*Correct rule:* Overriding subclass methods are restricted to throwing the same, narrower, or no checked exceptions.

*Example:*

```
class P { void show() {} }  
class C extends P { void show() throws IOException {} } // Compile error.
```

### Trap 38: Constructor Chaining order mismatch

*Wrong assumption:* Developer can execute child initialization tasks before parent construction starts.

*Correct rule:* Parent state is constructed first. `super()` must run before any child constructor statements.

*Example:*

```
class C extends P { C() { int x = 10; super(); } } // Compile error.
```

### Trap 39: package-private overriding visibility

*Wrong assumption:* A subclass in a different package can override a package-private (default) method.

*Correct rule:* Default access methods are invisible outside the package and cannot be inherited or overridden.

*Example:*

```
package p1; public class P { void m() {} }  
package p2; public class C extends p1.P { void m() {} } // This is a new m
```

### Trap 40: String literal intern auto-pool

*Wrong assumption:* Creating a String using `new String("abc")` automatically places the new object address in the pool.

*Correct rule:* Only literals go to the pool. `new` forces heap allocation. To pool it, you must call `intern()`.

*Example:*

```
String s1 = "abc";  
String s2 = new String("abc");  
System.out.println(s1 == s2); // Prints false.
```

### Trap 41: private fields in object heap representation

*Wrong assumption:* Private fields in parent classes are omitted from subclass heap allocations.

*Correct rule:* Subclass instances carry parent private fields in memory, though they are only accessible via parent methods.

*Example:*

```
class P { private int x; int getX() { return x; } }  
class C extends P {} // C objects hold memory space for 'x'.
```

### Trap 42: multiple default interface overrides resolution

*Wrong assumption:* If a class inherits a default method from an interface and a concrete class method, it must override to resolve.

*Correct rule:* Class methods take priority automatically. Override is only needed if conflicting default methods come from interfaces.

*Example:*

```
class Parent { public void m() {} }  
interface Intf { default void m() {} }  
class Child extends Parent implements Intf {} // Compiles without conflict
```

### Trap 43: final methods in abstract class

*Wrong assumption:* Abstract classes cannot declare final methods because abstract implies extension.

*Correct rule:* Abstract classes can declare final concrete methods to prevent subclasses from overriding core framework logic.

*Example:*

```
abstract class A { final void template() {} } // Valid.
```

### Trap 44: Cloneable marker interface methods

*Wrong assumption:* Implementing `Cloneable` provides the clone method body automatically.

*Correct rule:* `Cloneable` is a marker interface with zero methods. It only authorizes `Object.clone()` calls.

*Example:*

```
class Test implements Cloneable { /* must override clone() manually if pub
```

### Trap 45: custom Object.equals signature match

*Wrong assumption:* Overriding equals with `public boolean equals(MyClass other)` works with standard Collections checks.

*Correct rule:* Collections like `ArrayList` use `equals(Object)`, so matching specific types only overloads equals.

*Example:*

```
class Test { public boolean equals(Test t) { return true; } } // Overloaded
```

### Trap 46: static blocks throwing unchecked exceptions

*Wrong assumption:* If a static initializer throws an unchecked exception, it is caught by the calling constructor try-catch block.

*Correct rule:* It is wrapped in `ExceptionInInitializerError` and class loading fails permanently for the JVM session.

*Example:*

```
class A { static { int x = 1/0; } } // Throws ExceptionInInitializerError
```

### Trap 47: nested class naming in compiler

*Wrong assumption:* Anonymous inner classes compile into files matching the outer class name directly.

*Correct rule:* The compiler generates distinct `.class` files using numeric suffixes.

*Example:*

```
Outer$1.class, Outer$2.class
```

### Trap 48: local class final variable check timing

*Wrong assumption:* The compiler verifies effectively final status based only on the declaration line.

*Correct rule:* The compiler checks the entire scope of the local variable. If it is modified anywhere, it fails the check.

*Example:*

```
int x = 10;
class Local { void m() { System.out.println(x); } }
x = 20; // Causes compile error inside class Local.
```

### Trap 49: abstract method implementation inheritance hierarchy

*Wrong assumption:* An abstract subclass extending an abstract class must implement its parent's abstract methods.

*Correct rule:* Abstract subclasses can defer implementations. The first concrete subclass in the hierarchy must implement all.

*Example:*

```
abstract class A { abstract void m(); }  
abstract class B extends A {} // Valid.  
class C extends B { void m() {} } // Valid.
```

## Trap 50: primitive type casting vs object casting

*Wrong assumption:* Casting objects behaves exactly like casting primitives (e.g., casting `Integer` to `Double` works like `int` to `double`).

*Correct rule:* Primitive casting performs value transformation. Object casting only reinterprets references.

*Example:*

```
// Double d = (Double) new Integer(5); // Compile error or ClassCastException
```

---

## One-Day Revision Sheet

### Core Concepts Cheat Sheet

- **Polymorphism Rules:** Reference type controls what members can compile. Object type controls which overridden instance method executes. Fields and static methods never dispatch dynamically (they bind at compile time by reference type).
- **Initialization Order:** Static initializers (parent -> child) run once when classes load. Instance initializers and constructors (parent -> child) run on every object creation.
- **Overloading Priority:** Exact Match -> Widening -> Autoboxing -> Varargs. Java never performs widening and boxing together in a single step for parameter binding.
- **Interface Fields:** Automatically `public static final`. Methods are `public abstract` (or default/static/private).
- **Immutability Recipe:** `final` class, `private final` fields, no setters, defensive copy in constructor, defensive copy / unmodifiable view in getters.
- **Java Value Binding:** Java is strictly pass-by-value. References are passed as copied address values.

- **Dynamic Dispatch (vtable):** JVM looks up the vtable of the runtime object to call overridden methods.
- **equals() & hashCode() Contract:** Equal objects must have equal hashcodes. Always override `hashCode()` when overriding `equals()`.

## One-Week OOP Preparation Plan

| ONE-WEEK OOP STUDY PLAN |                               |                                                                 |
|-------------------------|-------------------------------|-----------------------------------------------------------------|
| Day                     | Topic Area                    | Target Focus                                                    |
| Day 1                   | Unit 1: Foundations & Objects | Class Anatomy, this, static, initialization orders.             |
| Day 2                   | Unit 2: Pillar Mastery        | Polymorphism, Dynamic Dispatch, method hiding, up/down casting. |
| Day 3                   | Unit 3: Abstraction & Design  | Interfaces, evolution, default conflicts, composition vs inh.   |
| Day 4                   | Unit 4: JVM & Traps           | Pass-by-value, Immutability, equals/hashCode contracts.         |
| Day 5                   | Unit 5: SOLID & Patterns      | 5 SOLID principles, Singletons, Strategy, Observer, Records.    |
| Day 6                   | Code practice & Dry runs      | Complete all 50 output and 40 error questions.                  |
| Day 7                   | Design Exercises & Mock       | Walk through the 10 design problems, practice SOLID.            |

- **Day 1 Focus:** Memorize the initialization order rules. Practice writing static blocks vs instance blocks and tracking them.
- **Day 2 Focus:** Master reference type vs object type rules. Draw reference pointer lines for upcasted and downcasted assignments.
- **Day 3 Focus:** Understand interface evolution changes. Practice resolving default interface method signature overlaps.

- **Day 4 Focus:** Trace pass-by-value heap copies. Memorize the 5 properties of the `equals()` contract.
- **Day 5 Focus:** Write bad SOLID examples and refactor them. Implement a thread-safe double-checked Singleton from scratch.
- **Day 6 Focus:** Work through the Output and Compile/Runtime error banks. Cover the code line-by-line.
- **Day 7 Focus:** Select 3 design problems (e.g., Parking Lot, Coffee Machine, Vending Machine) and write clean class models implementing SOLID and patterns.